UC for Gamers

AI: Claude Opus 5 (1M context)
Prompts: Pooya Farshim
Reviewers: Nobody

Date: August 14, 2026

Abstract

Universal composability is usually written in a mixture of prose and code, which makes UC research hard to falsify, and harder still to test or verify formally. Following the code-based style of property-based security, we set out a rigorous language for programming UC functionalities: identities and access control are data, wrappers enforce them, corruption is a register that every interface reads, and an execution is a handler passing a single token. Emulation is then defined over explicit machine classes with concrete bounds — no security parameter, no asymptotics — and the composition toolkit follows: substitution, transitivity, parallel composition, and a completeness theorem for the dummy adversary — all of them corollaries of one absorption lemma, which moves a protocol or an adversary into the environment and is an exact regrouping of a single execution rather than a comparison of two. Game-based properties turn out to be functionalities too, and they transfer along emulation. A signature functionality, a global clock and a bounded-delay network show the style at work. We hope this helps apply formal methods to UC, and ultimately give a formal UC proof.

Contents

1 Functionalities and Systems

We work in the universal composability framework of Canetti [2], several of whose conventions we make explicit below.

A process id names a running protocol instance, and it has three parts:

$$pid \;=\; (\mathsf{F},\, \mathtt{s},\, \mathtt{i}),$$

a *functionality name* F, a *session id* $\mathtt{s}$, and an *instance number* $\mathtt{i}$. The name says what code runs, the session which run of a protocol the instance belongs to, and the number distinguishes several instances of one name within a session. We write $\mathtt{id}.\mathsf{F}$, $\mathtt{id}.\mathtt{s}$ and $\mathtt{id}.\mathtt{i}$ for the three parts of $\mathtt{id}.pid$, and likewise $\mathcal{F}.\mathsf{F}$, $\mathcal{F}.\mathtt{s}$, $\mathcal{F}.\mathtt{i}$ for those of $\mathcal{F}.pid$; a bare $\mathtt{id}.pid = \mathcal{F}.pid$ means all three agree.

Names are elements of $\{0,1\}^* \cup \{\mathsf{A},\mathsf{Z},\mathsf{C}\}$ and party ids of $\{0,1\}^* \cup \{\mathsf{A},\mathsf{Z}\}$, where A, Z and C are reserved for the adversary, the environment and the corruption register. Being reserved, none of the three is an element of $\{0,1\}^*$, so a field declared to contain $\{0,1\}^*$ contains none of them; and C names no party, the register being a machine that parties call rather than one any party runs. Each of the three runs as a single instance in a single session, fixed to be session 0 and instance 0, so their process ids are $(\mathsf{A},0,0)$, $(\mathsf{Z},0,0)$ and $(\mathsf{C},0,0)$. We abbreviate these to the bare names A, Z and C throughout, the other two components carrying no information.

Tests are almost always on the name alone. Where we write $\mathtt{id}.\mathsf{F} = \mathsf{A}$ we mean the name component, never the whole triple, and the same for Z, for C, and for any functionality named in a membership test.

Two of the fields below collect names of this kind, and they collect different things. A party set holds *party ids*, elements of $\{0,1\}^* \cup \{\mathsf{A},\mathsf{Z}\}$; a caller set holds whole *process ids*, because who may call is a question about instances and not only about code. A caller set is therefore written with the abbreviations

$$\mathbf{A} := \{pid : pid.\mathsf{F} = \mathsf{A}\}, \qquad \mathbf{Z} := \{pid : pid.\mathsf{F} = \mathsf{Z}\}, \qquad \mathbf{S} := \{pid : pid.\mathsf{F} \in \{0,1\}^*\}$$

for the process ids of the adversary, of the environment, and of everything standard. Where an older notation would have admitted "A and Z" it now admits $\mathbf{A} \cup \mathbf{Z}$, of which only

$(A, 0, 0)$ and $(Z, 0, 0)$ ever occur, the execution supplying one of each; where it admitted "$\{0,1\}^*$" it now admits **S**, and there every session and instance genuinely does occur. Party sets keep the bare $\{A, Z\}$, those being party ids and not process ids, so a declaration that once read $\mathbf{P} := \mathbf{N}$ has to be split.

A functionality is named the same way, by its name component written as a subscript: $\mathcal{F}_{\mathsf{Sig}}$ is a functionality with $\mathcal{F}_{\mathsf{Sig}}.\mathsf{F} = \textsc{Sig}$, its session and instance left implicit, so the notation stands for a process id $(\textsc{Sig}, s, i)$ with s and i unspecified. Where a particular instance is meant they are supplied separately; where only the code matters, they are not written at all.

Little is lost by suppressing them, because a functionality's own code hardly ever reads them. Local or global, a core is written for the party it serves and the state it keeps; which session it belongs to, and which instance of its name it is, are the execution's business rather than its own. In this paper s occurs in three places only — line 3 of GUARD, the wrappers that invoke it, and the static check of Definition 1.3 — and i is never read at all. The instance number earns its place another way: it is what lets two instances of one name in one session carry the distinct process ids that Definition 1.2 asks for. Appendix B makes the observation into a statement: renaming sessions and instances is an isomorphism of executions, and an emulation proved at one process id holds at every other, under a hygiene condition on who may be called across sessions that every functionality here meets.

Definition 1.1 (Identities and functionalities). An *identity* is a pair $\mathrm{id} = (pid, \mathsf{P})$ of a process id and a party id, and an *identity set* is any set $\mathcal{I}$ of these — a process id being a triple, no longer a set of pairs of names. A *functionality* $\mathcal{F}$ carries the five fields

$\mathcal{F}.pid$	a process id	
$\mathcal{F}.\mathbf{P}$	set of *served* parties	the interfaces that exist
$\mathcal{F}.\mathbf{N}$	caller process ids it admits	who may call those interfaces
$\mathcal{F}.\mathbf{U}$	functionalities it *uses*	what must be there for it to run
$\mathcal{F}.par$	any further parameters	what the code reads besides the above

and *occupies* the identity set

$$\mathrm{IDs}(\mathcal{F}) \;:=\; \big\{\, (\mathcal{F}.pid,\ \mathsf{P}) \;:\; \mathsf{P} \in \mathcal{F}.\mathbf{P} \,\big\}.$$

Identity sets say two things: which identities a collection of functionalities occupies, and which a machine may claim. The five fields recur in the parameter line of every box below, written without the owner's name since the box heading supplies it. Only the first two fix $\mathrm{IDs}(\mathcal{F})$: the process id is shared by all its identities and the served parties supply the second component.

The fourth field records dependence. An entry of $\mathcal{F}.\mathbf{U}$ is a pair $(\mathcal{F}', \mathsf{R})$: $\mathcal{F}'$ names a functionality that must be present for $\mathcal{F}$ to run, and R is a predicate saying what $\mathcal{F}$ requires of whatever is found there. Naming a subroutine is rarely enough on its own — a subroutine that served none of its caller's parties, or admitted none of its callers, would be useless — so we write

$$\textsc{serves}(\mathcal{F}, \mathcal{G}) \;:\Longleftrightarrow\; \mathcal{G}.\mathbf{P} \supseteq \mathcal{F}.\mathbf{P} \;\wedge\; \mathcal{F}.pid \in \mathcal{G}.\mathbf{N}$$

for the requirement that comes up most: $\mathcal{G}$ serves every party $\mathcal{F}$ serves, and admits $\mathcal{F}$ among its callers. Without the second conjunct GUARD would refuse every call from $\mathcal{F}$ to $\mathcal{G}$.

What **U** records is what the *code* of $\mathcal{F}$ names. The calls its wrapper makes on its behalf

— to $\mathcal{C}$ on lines 2 and 6 of Section 7, and to $\mathcal{A}$ through MEDIATE — are common to every standard functionality and are left out, which costs nothing: both targets serve $\{0,1\}^* \cup \{A, Z\}$ and admit **S**, so SERVES$(\mathcal{F}, \mathcal{C})$ and SERVES$(\mathcal{F}, \mathcal{A})$ hold of every standard $\mathcal{F}$ whatever its own fields, and an entry for either could never fail.

Definition 1.2 (Systems). A functionality is *standard* if $\mathcal{F}.F \notin \{A, Z, C\}$, and each of its cores is wrapped as in Section 7 — save LEAK, which is wrapped as in Section 4, and INITIALIZE, which is a procedure and is not wrapped at all. A *system* (or *world*) π is a finite set of standard functionalities carrying distinct process ids: if $\mathcal{F}, \mathcal{F}' \in \pi$ and $\mathcal{F} \neq \mathcal{F}'$ then $\mathcal{F}.pid \neq \mathcal{F}'.pid$. It occupies whatever its members do,

$$\text{IDS}(\pi) \;:=\; \bigcup_{\mathcal{F} \in \pi} \text{IDS}(\mathcal{F}) \;=\; \big\{\, (\mathcal{F}.pid,\, P) \;:\; \mathcal{F} \in \pi,\; P \in \mathcal{F}.\mathbf{P} \,\big\}.$$

A system is what a protocol designer writes down — we use *protocol* and *system* interchangeably, *ideal* and *real* naming styles of writing one (below), not disjoint classes. Three machines are no part of it: the environment $\mathcal{Z}$, the adversary $\mathcal{A}$, and the corruption register $\mathcal{C}$ of Section 3. None is standard, so by Definition 1.2 no system contains any of them and IDS$(\mathcal{Z})$, IDS$(\mathcal{A})$ and IDS$(\mathcal{C})$ lie outside every system; the execution of Section 8 supplies all three instead. For $\mathcal{Z}$ and $\mathcal{A}$ that is what lets each claim its own identity while silenced on the system under test. For $\mathcal{C}$ it keeps the register clear of the set operations of Section 9 and Appendix A: no blocker is built for it, no replacement moves it, and every comparison runs over one and the same register. Reserving the three names also keeps them out of every $\mathcal{F}.\mathbf{N}$ declared as **S**, so no functionality admits the register among its callers — which is right, the register being a callee only.

Distinctness of process ids does real work. A system is determined by what it places at each id it uses, so membership is settled by the id alone. And every identity records that id in its first component, so distinct members have $\text{IDS}(\mathcal{F}) \cap \text{IDS}(\mathcal{F}') = \varnothing$: the union above is disjoint, and no identity of a system is occupied twice.

Definition 1.3 (Well-formed systems). Write $\pi^+ := \pi \cup \{\mathcal{Z}, \mathcal{A}, \mathcal{C}\}$ for a system together with the three machines every execution supplies. The system π is *well-formed*, written WF(π), if

$$\begin{aligned}\forall\, \mathcal{F} \in \pi^+ \;\; \forall\, (\mathcal{F}', R) \in \mathcal{F}.\mathbf{U} \;\; \exists\, \mathcal{G} \in \pi^+ \;:\; \mathcal{G}.F = \mathcal{F}'.F \;\wedge\; \big(\mathcal{G}.s = \mathcal{F}.s \;\vee\; \text{GLOBAL}(\mathcal{G})\big) \\ \wedge\; R(\mathcal{F}, \mathcal{G}).\end{aligned}$$

Three things about this. The conditions on $\mathcal{G}$ are exactly those GUARD imposes when the call is placed — name admitted, session matching unless the callee is shared, fields fitting — so WF(π) says precisely that the calls the members of π are built to make can succeed. Matching is by *name*, not by the functionality itself, which is what lets a dependence survive subsystem replacement: an entry naming φ_q is met by whatever sits at that name, ideal or real, so long as it meets the rest. And the requirement half ties the fields of caller and callee together; WF would say almost nothing without it.

Both quantifiers range over π^+, and each enlargement earns its place: on the right it makes an entry naming one of the three supplied machines satisfiable at all, none being a

member of any system, and on the left it checks their own entries — immediately here, since SERVES($\mathcal{Z}, \mathcal{A}$) and SERVES($\mathcal{A}, \mathcal{Z}$) hold and $\mathcal{C}$.**U** is empty. What WF does not do on its own is survive the constructions that follow: adding members could introduce obligations, and replacing a subsystem re-evaluates every requirement at the new occupant. Proposition A.4 (Appendix A) settles the first and Theorem 9.4 the second.

Conditions, collected. Well-formedness is one of several static conditions this paper puts on systems, cores and machines, and they are defined where each is first needed — which leaves a reader no way to see which result reads which. The table collects them once. None of them is read during an execution: EXEC reads fields, not conditions, as Proposition B.7 records for **U** in particular.

condition	constrains	read by
WF(π), Def. 1.3	a system: every **U** entry has a witness — the calls its members are built to *make* can succeed	Thm. 9.4, Prop. A.4, Def. 12.1
well-formed replacement, Def. 9.2	an incoming system against what stays: no process id in common	Prop. 9.3, Thm. 9.4, Prop. 10.11, Thm. 10.15, Thm. C.3, Prop. 12.10
compatible, Def. A.1	two systems: a shared process id serves and admits alike	Thm. A.3, second claim
session-uniform, Def. B.2	a system or environment: who can *reach* a member does not depend on its session	Lem. B.9, Prop. B.10, Cor. B.12, Cor. B.13
tight at I, Rem. A.5	a system: members outside I pin callers by process id	Def. 12.1
places no responsive call, Def. F.2	the cores of a system	Thm. D.4, Lem. 10.13 (ceding half), Prop. 12.9, Cor. 12.11
refusal-oblivious, Def. D.3	a machine in the adversary slot	the same four
identity-atomic, Def. B.4	all code; a standing convention rather than a per-system check	Lem. B.9, Prop. B.15
leakage-well-formed, Section 4	a realization	nothing: recorded, not used

Two things the table makes visible. The first two rows share a name and are different conditions: WF is about a system on its own, well-formed *replacement* about where an incoming system lands. And WF and session-uniformity are duals — the calls a member is built to make, against who may reach it — which invites folding them into one condition. They are kept apart because no result needs both, and because one definition could not carry both anyway: the game of Definition 12.1 is required to be well-formed and is deliberately *not* session-uniform, as Remark 12.6 explains.

We say $\mathcal{F}$ is *local* if $\mathcal{F}.\mathbf{N} = \{pid : pid.\mathsf{F} = \mathsf{F}^{\uparrow}\}$ for a designated parent (caller) name $\mathsf{F}^{\uparrow}$. (We leave it to the functionality to say whether **A** and **Z** may call it.) Intermediate notions are possible: a setup reachable from two named protocols admits $\{pid : pid.\mathsf{F} \in \{\mathsf{F}_1, \mathsf{F}_2\}\}$; and an **N** may pin whole process ids rather than names, listing its callers' occupied instances outright — Remark A.5 says why the private members of a realization should. At the other

end we say $\mathcal{F}$ is *global* if

$$\text{GLOBAL}(\mathcal{F}) \;:\Longleftrightarrow\; \mathbf{S} \subseteq \mathcal{F}.\mathbf{N} \;\vee\; \mathcal{F}.\mathsf{F} \in \{\mathsf{A}, \mathsf{Z}, \mathsf{C}\},$$

one notion and not two: the first disjunct is the ordinary case, any standard functionality may call it, and the second covers the three machines the execution supplies — and whatever stands in their slots — global for a different reason and treated so wherever GLOBAL is tested.

The distinction is what the session component is for. A local functionality is part of one run of one protocol, so a call may reach it only from its own session: line 3 of GUARD asks $\mathtt{id}'.\mathsf{s} = \mathtt{id}.\mathsf{s}$. The instance number is not checked, so one session may hold as many instances of a name as it likes and they may call one another. A global functionality is shared across runs — one session of it, reachable from every session — so that check is waived for it, and the sessions of $\mathcal{F}$ may all call one shared instance. We name a global ideal functionality with a leading G and a global real one with a leading γ, as in $\mathcal{F}_{\text{GSIG}}$; the calligraphic $\mathcal{G}$ stays what it has been, a second functionality alongside $\mathcal{F}$. The three machines the execution supplies are global: $\mathcal{A}$ and $\mathcal{C}$ already by the first clause, and $\mathcal{Z}$ by the second, which is there because $\mathcal{Z}.\mathbf{N} = \mathbf{A} \cup \mathbf{Z}$ is deliberately narrow and yet the environment must be reachable from every session.

Real vs. ideal specifications. Ideal functionalities and real protocols differ in several respects. First, ideal functionalities know the corruption status of parties in their party sets. Second, they can pass the execution to the simulator and may offer simulator-specific interfaces (see, e.g., $\mathcal{F}_{\text{Diffuse}}$). Finally, they may share state across parties — and sharing is the point: the guard of Section 2 ties every call to one party, so parties communicate only through functionalities that share state across them, and a real protocol talks to its peers through ideal subroutines and in no other way. Here and below, $\mathcal{F}_{\text{Diffuse}}$, $\mathcal{F}_{\text{Rand}}$, $\mathcal{F}_{\text{Store}}$ and $\mathcal{F}_{\text{IO}}$ name functionalities we do not give: a diffusion channel, a randomness source, a store, and an input–output relay. Nothing in this paper depends on their code.

The core interfaces of well-specified *real* protocols have none of these features, beyond those implicit in idealized building blocks and access-control wrappers; and real realizations do not specify how leakage is computed upon corruption, it being the contents of work tapes.[1]

Initialization. INITIALIZE is a procedure and not an interface: it runs once, when the functionality is first activated, and sets up the variables the rest of the code uses. It places no calls — the execution of Section 8 initializes every machine before its dispatcher is installed, so there would be nothing to serve one — no caller can reach it, it takes no claimed caller-id, and a functionality declaring none gets the empty one. Real protocols carry no global initializer of this kind, only ones specific to a party within a process, written $(\mathit{pid}, \mathsf{P}).\text{INITIALIZE}()$ and abusing notation: they are private, not interfaces anyone may address.

2 Guards, Silencers, and Mediators

This section fixes three pieces of machinery. The predicate GUARD constrains the calls that reach a machine; the wrapper SILENCE constrains the calls it makes; and the wrapper

[1] A more disciplined approach would model storage as an ideal functionality offering secure erasures; we avoid the extra layer to ease notation.

MEDIATE hands to the adversary a call that a corrupt party cannot answer for itself. The full interface of Section 7 is their composition.

Write OP for a generic core interface of $\mathcal{F}$. We specify only the cores; the wrapper below turns each into a full interface, and a caller reaches FULLOP, never OP. The access control checks are collected in the predicate GUARD, which is not a functionality but an algorithm: it takes the target identity $\mathtt{id}$, the claimed caller identity $\mathtt{id}'$ and the functionality $\mathcal{F}$, and returns a boolean. Two conventions of presentation. Boxes come in two colours: a mauve rule marks a functionality, which occupies identities, and a green one a predicate, wrapper or procedure, which occupies none. And a call is written $\mathtt{id}.\text{OP}(\mathtt{in})$ **as** $\mathtt{id}'$ where $\mathtt{id}'$ is the identity claimed in placing it, and received as $\mathtt{id}.\text{OP}(\mathtt{in})$ **from** $\mathtt{id}'$ where $\mathtt{id}'$ is the identity it was claimed under. Enforcement sits in the wrapper, where line 1 **require**s it, so a call it rejects never reaches a core. Line 1 checks that the interface exists; line 2, that the caller's name is admitted and that it acts for the right party; line 3, that caller and callee lie in one session, unless the callee is shared across sessions.

Predicate GUARD

GUARD$_{\mathcal{F}}(\mathtt{id}, \mathtt{id}')$:

1: **return** $\mathtt{id}.\mathit{pid} = \mathcal{F}.\mathit{pid} \wedge \mathtt{id}.\mathrm{P} \in \mathcal{F}.\mathbf{P}$ *// interfaces that exist*
2: $\quad \wedge\ \mathtt{id}'.\mathit{pid} \in \mathcal{F}.\mathbf{N} \wedge \mathtt{id}'.\mathrm{P} = \mathtt{id}.\mathrm{P}$ *// callers admitted*
3: $\quad \wedge\ (\mathtt{id}'.\mathrm{s} = \mathtt{id}.\mathrm{s} \vee \text{GLOBAL}(\mathcal{F}))$ *// one session, unless shared*

GUARD constrains the calls that reach a machine; the wrapper SILENCE below constrains the calls it *makes*. It takes an identity set $\mathcal{I}$ and a programme, runs it, and intercepts every call it issues. A call claiming an identity in $\mathcal{I}$ is refused with REJ without ever being placed; any other is placed and its answer handed back. So $\mathcal{I}$ is exactly where the wrapped programme is silenced, and it may speak as anything outside.

The scope is the wrapped programme and nothing further. A call it places passes the token to another machine, and what *that* machine says is governed by its own wrapper, not by this one. Were it otherwise a functionality would silence its subroutines transitively, and none could speak as itself. Every use of SILENCE below is of this shape: a full interface wrapping the core it serves.

It is convenient to read this as a handler. A call the programme issues is an operation of the ambient effect, and k is the continuation waiting for its answer. SILENCE decides what to resume with: REJ for a silenced claim, the answer of the call actually placed otherwise. The **return** clause passes the programme's own result through untouched, so silencing changes what the programme may say and nothing else. The analogy only fixes the shape of the wrapper.

Wrapper SILENCE
identity set $\mathcal{I}$, programme $\mathtt{id}.\text{OP}(\mathtt{in})$ from $\mathtt{id}'$

SILENCE$(\mathcal{I},\ \mathtt{id}.\text{OP}(\mathtt{in})$ from $\mathtt{id}')$:

1: **handle** $\mathtt{id}.\text{OP}(\mathtt{in})$ from $\mathtt{id}'$ **with**
2: $\quad$ **return** out $\mapsto$ out
3: $\quad \mathtt{id}''.\text{FULLOP}(\mathtt{in}'')$ as $\mathtt{id}'''$; $\mathrm{k} \mapsto \mathrm{k}(\text{REJ})$ if $\mathtt{id}''' \in \mathcal{I}$
4: $\qquad\qquad\qquad\qquad\quad \mapsto \mathrm{k}(\mathtt{id}''.\text{FULLOP}(\mathtt{in}'')$ as $\mathtt{id}''')$ otherwise

With SILENCE in hand we can say what happens to a call that a corrupt party cannot answer for itself. Every full interface below hands such a call to the adversary twice, on the way in and on the way out, and both times in the same shape: the wrapper MEDIATE. It takes a corruption set **C**, the operation id.OP being served, the value x to hand over, and the claimed caller id′ of the operation it stands in. Where the answer comes from is fixed inside it: whatever the operation, a corrupt party's answers come from (A, id.P). One thing about it is not an ordinary argument: both **require** and the **return** of MEDIATE return from the operation in whose code they stand, not merely from the line itself. When MEDIATE's test fails it does nothing and that operation carries on.

The hook is an ordinary call on $\mathcal{A}$'s full interface, claiming the identity id of the interface being served. It therefore meets $\text{GUARD}_{\mathcal{A}}$, which it passes because the claimed party id.P is the party of the target; and the answering adversary is confined by $\mathcal{A}$'s own silencer, which admits only (A, id.P). So while answering for id.P the adversary may speak as itself at id.P and as nothing else, and MEDIATE needs no silencing of its own.

Corruption wrapper MEDIATE
corruption set **C**, operation id.OP, value x, claimed caller id′

MEDIATE(**C**, id.OP, x, id′):
1: **if** (id′.F ≠ A ∧ id.P ∈ **C**) **then**
2: **return** (A, id.P).FULLA(id.OP, x) as id

3 Corruption

Corruption is not an interface that real protocols realize (what would a real protocol do when "instructed" to get corrupted?), but it must exist for emulation. We can assume real protocols always reject corruptions, but then they are never registered as corrupted.

We model corruptions as a global functionality $\mathcal{C}$. Several machines have their full interfaces given outright rather than obtained from cores — the environment, the adversary, and the leakage operation of Section 4 — but $\mathcal{C}$ is the only one that uses neither SILENCE nor MEDIATE, and the status report is why: that wrapper reads the corruption status on every call, which is what this interface implements, so wrapping it would be circular. Each interface below therefore carries by hand only the checks it needs, and FULLSTATUS in particular *always* returns the corrupted set without first asking whether the party is corrupt.

$\mathcal{C}$'s own leakage is constant, so the corruption test that Section 4 imposes on every other leakage operation would have nothing to protect; the guard is kept all the same, so the adversary reads the register at the party it is acting for and at no other.

Reading the register. **C** is $\mathcal{C}$'s own state, so no other machine holds a copy. Wherever **C** appears in the code of a functionality other than $\mathcal{C}$ itself, the understanding is that it was fetched first, by

$$\mathbf{C} \leftarrow (\text{C}, \text{id.P}).\text{FULLSTATUS}() \text{ as id},$$

and we suppress that line when it would only repeat what the surrounding code makes plain. Inside $\mathcal{C}$ there is nothing to fetch: **C** is read and written directly, and CORRUPT updates it in place.

Remark 3.1 (Why FULLSTATUS is not mediated)**.** Wrapped like the other operations, FULLSTATUS would be useless exactly when it matters. Suppose $\mathcal{Z}$ reaches out as id′

Corruption register $\mathcal{C}$
pid $:= C$, $\mathbf{P} := \{0,1\}^* \cup \{A, Z\}$, $\mathbf{N} := \mathbf{S} \cup \mathbf{A} \cup \mathbf{Z}$, $\mathbf{U} := \varnothing$, *par* $:= \perp$

INITIALIZE():

1: $\mathbf{C} \leftarrow \varnothing$

id.FULLCORRUPT() from id′

2: **require** GUARD$_{\mathcal{C}}$(id, id′)
3: **require** id′.F $\in \{A, Z\}$ // *the outside only, never a subroutine*
4: **require** id.P $\in \{0,1\}^*$ // *reserved names stay honest*
5: $\mathbf{C} \leftarrow \mathbf{C} \cup \{\text{id.P}\}$
6: **return** OK

id.FULLSTATUS() from id′

7: **require** GUARD$_{\mathcal{C}}$(id, id′)
8: **return** **C**

id.FULLLEAK() from id′

9: **require** GUARD$_{\mathcal{C}}$(id, id′)
10: **require** id′.F $= A$
11: **return** $\perp$ // *constant: the register keeps no secrets*

to ask the corruption status of id′.P. If id′.P is corrupt, the gate lets the call through and MEDIATE hands it to the adversary — so the answer to "is id′.P corrupt?" is whatever the adversary says, and it may say no. Since every caller reads the register to decide how to treat a corrupt party, an adversary-controlled answer would switch the mediation machinery off at will.

FULLSTATUS therefore mediates and silences nothing: past the guard it returns **C**, the register's own state. It is the one operation whose answer must come from the register, and being read-only it gives the adversary nothing it could not already infer. The core STATUS of earlier drafts goes with it, there being no wrapper left to keep out of.

Corruption is the outside's to do. Line 3 admits id′.F $\in \{A, Z\}$, so the adversary and the environment may each put a party into **C** and no standard functionality may put anything there at all. The guard above it binds the corrupting interface to the party it corrupts, asking id′.P $=$ id.P: to corrupt P the adversary must run at (A, P), and the environment must claim (Z, P), which line 3 lets its root do for any party. Both may read as well: $\mathbf{A} \cup \mathbf{Z} \subseteq \mathcal{C}.\mathbf{N}$, and FULLSTATUS returns all of **C**, so either may ask at any point and always learns the full set. And only genuine parties can be corrupted: FULLCORRUPT asks id.P $\in \{0,1\}^*$, so the reserved names stay honest by construction.

Commensurateness comes out right, and it is the *reading* that secures it rather than any division of who may write. The machine at (A, P) is $\mathcal{A}$ in one execution of an emulation experiment and $\mathcal{S}$ in the other, and nothing forces the two to corrupt the same parties. But $\mathcal{Z}$ is the same machine on both sides and reads **C** on both, so an occupant of the slot corrupting a different set is caught by an environment that simply asks. Commensurateness is thus a consequence, not an assumption: no admissible environment tells the two corruption sets apart with advantage beyond what Definition 10.7 allows the simulator. That the

environment may corrupt for itself only sharpens this, the corruptions it makes being its own on both sides; what is left to catch is the occupant's own initiative.

Corruption is indexed by party name alone, so P cannot be corrupt at one process id and honest at another.[2]

4 Leakage

Functionality $\mathcal{F}$, leakage operation
pid, **P**, **N**, **U**, *par*

id.FULLLEAK() from id′

1: **require** id.*pid* = $\mathcal{F}$.*pid* ∧ id.P ∈ $\mathcal{F}$.**P** // *the interface exists*
2: **require** id′.F = A ∧ id′.P = id.P // *$\mathcal{A}$, for this party*
3: **C** ← (C, id.P).FULLSTATUS() as id
4: **require** id.P ∈ **C** // *only a corrupt party leaks*
5: **return** SILENCE({id″ : id″ ≠ id}, id.LEAK() from id′)

The three requirements are those the general wrapper of Section 7 imposes, restated because leakage is not obtained from a core in the usual way. Line 1 is the existence half of GUARD; the caller half is deliberately weaker, since $\mathcal{A}$ must be able to leak from a functionality that does not admit it as a caller, so line 2 asks only that the adversary act for the party it is reading. Line 4 is the corruption gate. Without it an honest party's state would be readable at will, which is what line 3 of the full interface refuses everywhere else. The silencer is line 5's, so a leakage core that places calls claims its own identity like any other core.

For an ideal functionality, LEAK is part of the specification; the messages received so far and the contents of internal lists are typical. For real protocols it can be set to leak nothing, which models no leakage at all; ideal functionalities that say which parts of the state leak are the fix.

A protocol is *leakage-well-formed* if every machine body is pure between invocations, all sampled randomness is drawn through $\mathcal{F}_{\mathsf{Rand}}$ and all state persisting across invocations held in $\mathcal{F}_{\mathsf{Store}}$ rather than on a machine tape — each exposing it through its own LEAK — and inputs and outputs leak only where the protocol routes them through an input–output functionality $\mathcal{F}_{\mathsf{IO}}$, which is not the default. Work tapes are then never leaked, purity between invocations leaving nothing on them. No result below assumes any of this; it is the condition a realization must meet for its LEAK to mean anything, and we record it rather than use it.

$\mathcal{Z}$ cannot call this interface, so it need not be realized for indistinguishability. It must still be simulable, since the adversary can call it.

5 The Environment

We define a special functionality $\mathcal{Z}$, with a single per-id interface id.FULLZ that guards and silences the core id.Z. The field $\mathcal{Z}$.**N** = **A** ∪ **Z** deserves emphasis: it admits the adversary and

[2] One could instead corrupt an *identity* — **C** collecting identities, every test id.P ∈ **C** becoming id ∈ **C** — at the cost of "spreading" corruptions to subprotocols. The party-wide convention is cleaner, infecting a whole family at once.

the environment itself and nothing else, so no standard functionality may call $\mathcal{Z}$. Anything wider would let a protocol call the environment as one of its own subroutines — an "open protocol," whose subroutines are left undefined. Two distinct things rule that out. The narrowness of $\mathcal{Z}$.**N** refuses the call itself: a standard caller's process id lies in **S**, which meets $\mathbf{A} \cup \mathbf{Z}$ nowhere, so line 2 of GUARD rejects it whatever the caller declares. Definition 1.3 refuses the other half of openness, a member naming a dependence no member supplies. It is the first that does the work here: naming $\mathcal{Z}$ would not trip WF, since $\mathcal{Z} \in \pi^+$ makes such an entry satisfiable, so a system could be well-formed and still be open were the field any wider.

The **Z** in $\mathcal{Z}$.**N** is there for a different reason: it lets $\mathcal{Z}$ call itself. The Z interfaces of the environment at different parties are separate, and admitting **Z** is what lets them reach one another.

Which of them may reach which is settled by the silenced set, and there it does more than keep $\mathcal{Z}$ off the systems. Every other machine is tied to where it runs: a standard functionality may claim only its own identity, and $\mathcal{A}$ is silenced on $\{\mathtt{id}'' \neq \mathtt{id}\}$. Nothing would tie $\mathcal{Z}$ on its own, since SILENCE tests only the claimed $\mathtt{id}'$ and GUARD only the target, so (Z, P) and $(\mathsf{Z}, \mathsf{P}')$ would be interchangeable and the party component of a $\mathcal{Z}$ identity would carry no information. Line 3 supplies the tie, as loosely as applications allow: an interface at $\mathtt{id}$ may claim $\mathtt{id}'$ when $\mathtt{id}'.\mathsf{P} = \mathtt{id}.\mathsf{P}$, when $\mathtt{id}'.\mathsf{P} = \mathsf{Z}$, or when $\mathtt{id}.\mathsf{P} = \mathsf{Z}$. In words, *act for your own party, or for the root; and the root may act for anyone.*

Both escapes are needed. Without $\mathtt{id}.\mathsf{P} = \mathsf{Z}$ the root is stranded, and it is where EXEC begins and the only interface that can reach the others. Without $\mathtt{id}'.\mathsf{P} = \mathsf{Z}$ the leaves are: the adversary's only route back into the environment is $(\mathsf{A}, \mathsf{P}) \to (\mathsf{Z}, \mathsf{P})$, and MEDIATE wakes $\mathcal{A}$ at $(\mathsf{A}, \mathtt{id}.\mathsf{P})$ for the party it answers for, so an environment needing to compare what the adversary said at P against what it said at P' could never bring the two together — these interfaces are separate, and cannot pool it through shared state either — and the class of Definition 10.7 would genuinely shrink. With both escapes, connectivity is a two-way star rooted at (Z, Z), and we know of nothing an unrestricted environment could do that the star cannot. Line 2 restricts in the same spirit: it stops $\mathcal{Z}$ claiming (A, P), and so stops it passing the corruption gate of line 3 and driving an *honest* party's interface — a capability the adversary it would be impersonating does not have either, as Remark 10.4 records of a simulator silenced on less. What that line no longer has to do is keep $\mathcal{Z}$ away from FULLCORRUPT, which line 3 admits it to under its own name.

Together, $\mathbf{A} \cup \mathbf{Z}$ is what applications need and no more. **A** is necessary because the environment has no other way back into the execution: once $\mathcal{Z}$ passes the token on it runs again only when something calls it, and only the adversary may; that same channel is how $\mathcal{Z}$ hears whatever the adversary chooses to tell it. **Z** is necessary because the environment is one interface per party rather than one machine, and without it the root could reach none of them. Nothing beyond these two is: a functionality that had to call $\mathcal{Z}$ would treat it as a subroutine it does not define, and $\mathcal{C}$ reports corruptions to no one — the adversary knows what it corrupts, having done it — which is why $\mathcal{Z}$ must *ask* for the corruption set. So the environment is reachable only from the two machines outside every system.

6 The Adversary

The adversary $\mathcal{A}$ has the same shape: a single per-$\mathtt{id}$ interface $\mathtt{id}$.FULLA guarding and silencing the core $\mathtt{id}.\mathsf{A}$. What separates it from the environment is who may reach it. Where

Environment $\mathcal{Z}$
pid $:= Z$, $\mathbf{P} := \{0,1\}^* \cup \{A, Z\}$, $\mathbf{N} := \mathbf{A} \cup \mathbf{Z}$, $\mathbf{U} := \{(\mathcal{A}, \text{SERVES})\}$, *par* $:= \mathcal{I}$

id.FULLZ(in) from id′

1: **require** GUARD$_{\mathcal{Z}}$(id, id′)
2: $\bar{\mathcal{I}} \leftarrow \mathcal{I} \cup \{\text{id}'' : \text{id}''.\text{F} = \text{A}\}$ // $\mathcal{A}$ out of reach
3: $\quad \cup \{\text{id}'' : \text{id.P} \neq \text{Z} \wedge \text{id}''.\text{P} \notin \{\text{id.P}, \text{Z}\}\}$ // own party, or the root
4: out $\leftarrow$ SILENCE$(\bar{\mathcal{I}}$, id.Z(in) from id′$)$
5: **return** out

$\mathcal{Z}.\mathbf{N}$ is only $\mathbf{A} \cup \mathbf{Z}$, the field $\mathcal{A}.\mathbf{N} = \mathbf{S} \cup \mathbf{A} \cup \mathbf{Z}$ holds every process id that ever places a call. Only $\mathcal{C}$'s is missing, and $\mathcal{C}$ places none. That width is a requirement rather than a convenience: the mediation hooks of Section 2, and the calls ideal functionalities place through the abbreviation $\mathcal{A}(\cdot)$, are both calls placed on the adversary by a machine acting for some party.

These interfaces are what a backdoor looks like in this notation. The adversary holds no token of its own and cannot begin a call, so every attack it mounts starts from another machine handing it control: a party's interface through MEDIATE, or the environment directly, which is how an environment instructs an adversary at all. What it may say in return is narrow: the silenced set is $\{\text{id}'' : \text{id}'' \neq \text{id}\}$, so an interface of $\mathcal{A}$ may claim only the identity it runs at.

Inside an interface with identity id we write $\mathcal{A}(\text{in})$ for the call $(\text{A}, \text{id.P}).\text{FULLA}(\text{in})$ as id, and $\mathcal{Z}(\text{in})$ likewise. The claim must be id and no other, since line 5 silences a core on everything but its own identity; the adversary therefore learns the caller as the claimed id, and needs no separate field for it. Its core is entered as id.A(in) from id′, the shape SILENCE declares and the shape every other core is entered in. The target's party component is id.P, so the call also records on whose behalf it is made.

Adversary $\mathcal{A}$
pid $:= A$, $\mathbf{P} := \{0,1\}^* \cup \{A, Z\}$, $\mathbf{N} := \mathbf{S} \cup \mathbf{A} \cup \mathbf{Z}$, $\mathbf{U} := \{(\mathcal{Z}, \text{SERVES})\}$, *par* $:= \perp$

id.FULLA(in) from id′

1: **require** GUARD$_{\mathcal{A}}$(id, id′)
2: out $\leftarrow$ SILENCE$(\{\text{id}'' : \text{id}'' \neq \text{id}\}$, id.A(in) from id′$)$
3: **return** out

7 The Full Interface

Everything the wrapper needs is now in place: GUARD and MEDIATE and SILENCE from Section 2, and the register $\mathcal{C}$ of Section 3 that supplies $\mathbf{C}$. The full interface of a standard functionality is their composition.

Lines 2–4 and 6–7 handle corruption, and are the only place a full interface consults the register. It is read twice because $\mathbf{C}$ changes as the execution proceeds and a read is only a snapshot: the tests before line 6 use the first, the test on line 7 the second. Membership is monotone, CORRUPT only ever adding, so a party honest at the first read may be corrupt at the second but never the reverse.

Functionality $\mathcal{F}$
pid, **P**, **N**, **U**, *par*

id.FULLOP(in) from id′
1: **require** GUARD$_{\mathcal{F}}$(id, id′)
2: **C** ← (C, id.P).FULLSTATUS() as id
3: **require** ¬(id′.F = A ∧ id.P ∉ **C**) // $\mathcal{A}$ only at a corrupt party
4: MEDIATE(**C**, id.OP, in, id′)
5: out ← SILENCE({id″ : id″ ≠ id}, id.OP(in) from id′)
6: **C** ← (C, id.P).FULLSTATUS() as id
7: MEDIATE(**C**, id.OP, out, id′)
8: **return** out

Line 3 refuses one combination: the adversary at an honest party, which has no business there. Everything else passes, and MEDIATE settles the rest. So an honest party is addressable by everyone but $\mathcal{A}$ and runs its own core; a corrupt party is addressable by everyone, but only $\mathcal{A}$ reaches its core, every other caller being answered by $\mathcal{A}$ in the party's place. The environment needs no case of its own: whether it claims its own identity or an admitted external one, a corrupt party answers it through $\mathcal{A}$ either way.

That leaves a corrupt party addressed by anyone but $\mathcal{A}$, whose call cannot simply be run, so $\mathcal{A}$ answers in the party's place. Lines 4 and 7 are the same wrapper at two moments. The first catches a party already corrupt on arrival and hands over in, so the core never runs. The second catches one that turned corrupt while its core ran on line 5, and hands over out, no longer the party's to return. Monotonicity makes the two cover the cases exactly once: a party corrupt at the first has already returned, and none is corrupt at the first and honest at the second. Both hooks address $\mathcal{A}$ at (A, id.P) through its full interface, so the answering adversary is guarded and silenced like any other callee. Line 8 returns the core's own value, which happens in two cases: for a party honest throughout, and for a corrupt party addressed by $\mathcal{A}$ itself, which the gate admits and which neither hook catches, both testing id′.F ≠ A.

Remark 7.1 (Corruption mid-call is not preemptive)**.** Nothing interrupts a core that is already running. Suppose the core on line 5 calls the adversary — through $\mathcal{A}(\cdot)$, or through a subroutine that mediates — and the adversary corrupts id.P before returning a value. The register records that at once, **C** being its own state, but the core is neither told nor stopped: it resumes with whatever the adversary returned and runs to completion. Only on its return does line 6 read **C** again, find id.P in it, and hand out to MEDIATE on line 7 to be replaced.

So the adversary holds the token twice over such a call, in this order: once inside the core, for the call the core placed, and once after the core has returned, for the mediation of its output. No moment in between lets the corruption take effect, since a party that turns corrupt mid-call cannot un-run what it has already done. Nor is the call re-gated: line 3 was decided on the first read, and a call admitted while id.P was honest stays admitted. What the wrapper guarantees is therefore weaker than aborting, and sufficient: nothing a newly corrupt party computed reaches its caller unmediated. Calls arriving *after* the corruption are caught on the way in instead, by line 4.

8 The Execution Model

In an execution exactly one machine holds the activation token. A call $\mathtt{out} \leftarrow \mathtt{id}.\textsc{Op}(\mathtt{in})$ abbreviates suspending the caller with a continuation and passing the token to interface $\textsc{Op}$ of $\mathcal{F}$ at identity $\mathtt{id}$. Every call between machines addresses a full interface, and $\textsc{FullOp}$ is what $\textsc{Exec}$ dispatches; a core is invoked only by the wrapper that serves it, inside the machine that carries it, and never across a token transfer. A call addressed to an identity no machine occupies is answered with $\textsc{rej}$. An instance may hold several outstanding continuations and may be re-entered while suspended, in particular by $\mathcal{A}$.

Definition 8.1 (Machines and occupancy)**.** A *machine* is anything an execution runs: the members of the system, and whatever the execution supplies alongside them. A machine *occupies* a finite set of identities — for a functionality, the $\textsc{ids}(\mathcal{F})$ of Definition 1.1 — and $\textsc{Exec}$ dispatches to it exactly the calls addressed to identities it occupies. The occupants of one execution carry pairwise disjoint identity sets, and $\textsc{Exec}$ is defined for no other family. An *occupant of the adversary slot* is a machine occupying the identities of process id A and no others; the adversary of Section 6 is one, and so is every simulator of Section 10. An *activation* is a maximal interval during which one machine holds the token.

Every call carries a claimed id $\mathtt{id}'$. Standard functionalities may claim only the identity of the interface placing the call; $\mathcal{Z}$ and $\mathcal{A}$ may claim others, subject to the checks of Section 2. Those checks are the guard, which tests $\mathtt{id}'$ against the interface's identity $\mathtt{id}$ and against the functionality's parameters, and we specify them with **require** statements. A failed guard returns the distinguished value $\textsc{rej}$ to the sender, and $\textsc{rej}$ is a refusal rather than a value: no machine may return it as its own output. The rule is enforced rather than assumed — a core whose code would return $\textsc{rej}$, and a mediated answer of $\textsc{rej}$ from the adversary's slot, are delivered as $\perp$ instead — so a caller receiving $\textsc{rej}$ learns that a guard refused its call, and nothing else can produce one. In particular $\textsc{rej} \neq \perp$.

Randomness is per machine: each machine carries its own coin tape, infinite, uniform, and independent of every other machine's, read only when its code samples. Probabilities over an execution are over these tapes jointly. Bundling machines into one occupant — as Section 10 and Appendix D do — does not merge tapes: a hosted copy's coins are what they were before absorption.

Besides the system π, the execution carries three machines: the environment $\mathcal{Z}$, the adversary $\mathcal{A}$, and the corruption register $\mathcal{C}$, whose state $\mathbf{C}$ every wrapper reads. By Section 1 none is a member of π, which makes line 1 unambiguous — the register is initialized once, as an argument, not a second time as a member — and keeps π the object a designer wrote. The procedure initializes the three, then activates the environment at (Z, Z) as though it had called itself. That first activation goes through $\mathcal{Z}$'s full interface like any other, so the environment is silenced from the outset, and $\textsc{Guard}_{\mathcal{Z}}$ passes trivially, the claimed and target identities being the same. Whatever the environment returns is the outcome.

Read as a handler, the execution has exactly one operation: placing a call. Line 2 activates the environment, the only activation not placed by a machine. Line 4 handles every call thereafter, and the continuation $\mathtt{k}$ is where the token comes back: suspending the caller, passing the token to $\mathtt{id}$, and resuming the caller with the answer are precisely what invoking $\mathtt{k}$ on that answer means. Exactly one machine holds the token throughout.

Execution EXEC
system π, environment $\mathcal{Z}$, adversary $\mathcal{A}$, corruption register $\mathcal{C}$

EXEC$(\pi, \mathcal{Z}, \mathcal{A}, \mathcal{C})$:

1: $\mathcal{C}$.INITIALIZE(); $\mathcal{F}$.INITIALIZE() for every $\mathcal{F} \in \pi$
2: **handle** (Z, Z).FULLZ() from (Z, Z) **with** *// $\mathcal{Z}$ is activated first and holds the token*
3: **return** out $\mapsto$ out
4: id.FULLOP(in) from id′; k $\mapsto$ k(id.FULLOP(in) from id′)

The **return** clause is reached when the environment's program halts, and its output is the execution's.

9 Subsystem Replacement

A protocol is normally written against a subroutine later supplied by another protocol. Replacing one by the other is a set operation on systems.

Definition 9.1 (Subsystem replacement)**.** Let ρ be a system and $\varphi \subseteq \rho$ a subsystem of it. For a system π, the *replacement of φ by π in ρ* is

$$\rho[\pi/\varphi] \;:=\; (\rho \setminus \varphi) \cup \pi.$$

Nothing so far stops the incoming π from landing on a process id the surrounding protocol already uses, which would leave a call ambiguously addressed. We rule this out rather than repair it.

Definition 9.2 (Well-formed replacement)**.** The replacement of φ by π in ρ is *well-formed* if no process id occurs both in $\rho \setminus \varphi$ and in π, that is, if

$$\{\mathcal{F}.pid : \mathcal{F} \in \rho \setminus \varphi\} \cap \{\mathcal{F}.pid : \mathcal{F} \in \pi\} \;=\; \varnothing.$$

Proposition 9.3 (Replacement)**.** *Let ρ and π be systems, let $\varphi \subseteq \rho$, and let the replacement of φ by π in ρ be well-formed. Then $\rho[\pi/\varphi]$ is a system. If moreover* $\mathrm{IDS}(\pi) = \mathrm{IDS}(\varphi)$*, then* $\mathrm{IDS}(\rho[\pi/\varphi]) = \mathrm{IDS}(\rho)$.

Proof. The process ids of $\rho[\pi/\varphi]$ are those of $\rho \setminus \varphi$ together with those of π. Each system carries distinct ids internally and well-formedness makes the two collections disjoint, so no id repeats; the members are standard and finitely many, both inherited from ρ and π, so $\rho[\pi/\varphi]$ is a system. That first claim rests on the well-formedness of the replacement alone, $\mathrm{IDS}(\pi) = \mathrm{IDS}(\varphi)$ entering nowhere in it. Identities need no separate hypothesis: an identity carries its process id in the first component, so members with distinct ids have disjoint identity sets. For the identities themselves,

$$\mathrm{IDS}(\rho[\pi/\varphi]) \;=\; \mathrm{IDS}(\rho \setminus \varphi) \cup \mathrm{IDS}(\pi) \;=\; \mathrm{IDS}(\rho \setminus \varphi) \cup \mathrm{IDS}(\varphi) \;=\; \mathrm{IDS}(\rho),$$

using $\mathrm{IDS}(\pi) = \mathrm{IDS}(\varphi)$ in the middle step and $\varphi \subseteq \rho$ in the last. □

The two halves of the proposition are used differently below. That $\rho[\pi/\varphi]$ is a system lets Section 10 speak of it at all, and rests on well-formedness alone. That its identities are those of ρ needs $\mathrm{IDS}(\pi) = \mathrm{IDS}(\varphi)$, and no result in Section 10 assumes it: admissibility there is stated for a pair of systems through the union of their identity sets, and Proposition 10.11 and Theorem 10.15 say outright that no relation between the two is used. Where the equality is wanted, blocking supplies it.

Definition 9.2 settles process ids, and Proposition 9.3 identities. Neither addresses dependence, and dependence is where replacement can go wrong: the names $\rho \setminus \varphi$ relies on are still occupied after φ gives way to π, but every requirement is re-evaluated at the new occupant. Two conditions close the gap.

Informally, Theorem 9.4 below says that a well-formed replacement preserves WF when the entries of the incoming π can be discharged in the new world, and some member of π stands in for φ wherever an entry of $\rho \setminus \varphi$ was witnessed inside φ. Hypothesis (ii) is what it means for π to be a drop-in for φ, and emulation does not supply it.

Theorem 9.4 (Replacement preserves well-formedness)**.** *Let ρ and π be systems, let $\varphi \subseteq \rho$, let the replacement of φ by π in ρ be well-formed in the sense of Definition 9.2, and let $\mathrm{WF}(\rho)$. Suppose*

(i) every entry of every $\mathcal{F} \in \pi$ has a witness in $\big(\rho[\pi/\varphi]\big)^+$, in the sense of Definition 1.3; and

(ii) whenever $\mathcal{F} \in \rho \setminus \varphi$ has an entry $(\mathcal{F}', \mathsf{R}) \in \mathcal{F}.\mathbf{U}$ for which some *witness in the sense of Definition 1.3 lies in φ, some $\mathcal{G} \in \pi$ satisfies $\mathcal{G}.\mathsf{F} = \mathcal{F}'.\mathsf{F}$, $\mathcal{G}.\mathsf{s} = \mathcal{F}.\mathsf{s} \vee \mathrm{GLOBAL}(\mathcal{G})$, and $\mathsf{R}(\mathcal{F}, \mathcal{G})$.*

Then $\mathrm{WF}\big(\rho[\pi/\varphi]\big)$.

Proof. Write $\rho' := \rho[\pi/\varphi] = (\rho \setminus \varphi) \cup \pi$, a system by the first claim of Proposition 9.3 — the second is not available here, $\mathrm{IDS}(\pi) = \mathrm{IDS}(\varphi)$ being no hypothesis of this theorem. Take $\mathcal{F} \in \rho'^+$ and an entry $(\mathcal{F}', \mathsf{R}) \in \mathcal{F}.\mathbf{U}$; we must place a witness in ρ'^+. There are three cases, by where $\mathcal{F}$ comes from.

If $\mathcal{F} \in \pi$, hypothesis (i) is exactly the claim.

If $\mathcal{F}$ is one of $\mathcal{Z}, \mathcal{A}, \mathcal{C}$, its entries are settled once and for all, independently of the system: $\mathcal{C}.\mathbf{U}$ is empty, and the entries of $\mathcal{Z}$ and $\mathcal{A}$ name each other, with $\mathcal{A}, \mathcal{Z} \in \rho'^+$ and SERVES holding between them as Section 1 records. All three are global, so the session conjunct is waived.

If $\mathcal{F} \in \rho \setminus \varphi$, then $\mathcal{F} \in \rho^+$, so $\mathrm{WF}(\rho)$ supplies a witness $\mathcal{G} \in \rho^+$ for this entry. Either $\mathcal{G} \notin \varphi$, in which case $\mathcal{G}$ lies in $(\rho \setminus \varphi) \cup \{\mathcal{Z}, \mathcal{A}, \mathcal{C}\} \subseteq \rho'^+$ and serves again: it is the same machine with the same fields, so $\mathcal{G}.\mathsf{F}$, $\mathcal{G}.\mathsf{s}$ and whatever R reads are unchanged, and $\mathcal{F}$ is unchanged too. Or $\mathcal{G} \in \varphi$, and hypothesis (ii) supplies a member of π meeting the same three conditions in its place. Either way the entry has a witness in ρ'^+. □

Hypothesis (ii) is what it means for π to be a drop-in for φ, and emulation does not supply it. Definition 10.7 compares behaviour at the boundary; it says nothing about the declared fields of the machines behind that boundary, and a π that emulates φ perfectly may still fail to admit a caller φ admitted, or serve a party φ served. The check is cheap — it ranges over the entries of $\rho \setminus \varphi$ that pointed into φ, and nothing else — but it is a check, and belongs beside well-formedness of the replacement rather than inside the composition

theorem.

10 UC Emulation and Composition

Everything an execution needs is now fixed, so we can say when one system is at least as secure as another. The two are placed under one and the same environment, so what it may claim must be settled first; that is the role of $\mathcal{Z}.\mathit{par}$ below.

The value of $\textsc{Exec}(\pi, \mathcal{Z}, \mathcal{A}, \mathcal{C})$ is whatever the environment returns. We take an environment to return a bit at the root — its guess as to which system it was placed in — and read the execution as returning 1 exactly when the environment does. Nothing else in the framework constrains that value, so this is a condition on the environments quantified over, not a consequence. The gap between two executions is then a number, not an informal resemblance.

Definition 10.1 (UC advantage). Let π and φ be systems. For an environment $\mathcal{Z}$, an adversary $\mathcal{A}$ and a simulator $\mathcal{S}$,

$$\textsc{Adv}^{\textsc{uc}}_{\pi,\varphi}(\mathcal{Z}, \mathcal{A}, \mathcal{S}) \;:=\; \Big|\, \Pr\big[\textsc{Exec}(\pi, \mathcal{Z}, \mathcal{A}, \mathcal{C}) = 1\big] \;-\; \Pr\big[\textsc{Exec}(\varphi, \mathcal{Z}, \mathcal{S}, \mathcal{C}) = 1\big] \,\Big|,$$

the probabilities being over the coins of every machine in the two executions, and $\textsc{Exec} = 1$ abbreviating the event that the execution halts with the environment returning 1.

The two executions differ in the system and in the machine that occupies the adversary's identities: $\mathcal{A}$ on one side, $\mathcal{S}$ on the other. Both are typed by the same pair of definitions.

Definition 10.2 (Adversaries). An *adversary* is a machine that occupies $\textsc{ids}(\mathcal{A})$ and no other identities and carries the adversary's interfaces — the wrapper of Section 6, guard and silencer, and any further interface that section's machine is given (Appendix F adds one) — around an arbitrary core at each of them. The machine of Section 6 is one; the dummy of Appendix D is another.

Definition 10.3 (Simulators). A *simulator* is an adversary that admits at least $\mathbf{S} \cup \mathbf{Z}$.

Every simulator is thus an adversary, which is what lets one stand in an adversary class: Proposition 10.16 and Theorem C.3 read simulators as adversaries, and the budget containments of Section 11 compare the two classes directly. Occupancy and the admitted callers are what standing in the adversary's slot means, so everything Section 6 settles about a machine at (A, P) holds of a simulator unchanged: it begins no call of its own, and its silencer lets it claim only the identity it runs at. Proposition 10.14 asks only occupancy of whatever it is given; Theorem D.4 uses the wrapper and the admitted callers too, and Remark 10.4 explains why neither is a restriction.

The metric takes the three machines as arguments, so emulation can be stated by quantifying over classes of them and bounding the result. Which environments may be used at all is settled by the pair being compared.

Remark 10.4 (What emulation fixes about the adversary slot)**.** Definition 10.6 below constrains $\mathcal{Z}.par$ and says nothing about the machines in the adversary slot; Definitions 10.2 and 10.3 say everything needed, and it is less than the five fields. *pid* and **P** come free with occupancy: $\text{IDs}(\mathcal{F}) = \{(\mathcal{F}.pid, \mathsf{P}) : \mathsf{P} \in \mathcal{F}.\mathbf{P}\}$, so occupying $\text{IDs}(\mathcal{A})$ pins both, giving $\mathcal{S}.pid = (\mathsf{A}, 0, 0)$ at line 1 and $\text{GLOBAL}(\mathcal{S})$ at line 3; and pinning **P** is what is wanted, a slot occupant serving fewer parties leaving an A-identity unoccupied on one side alone, answered REJ where the other answers. *par* is never read — the silenced set is the literal $\{\mathtt{id}'' : \mathtt{id}'' \neq \mathtt{id}\}$ of line 2, carried as code by the required wrapper, and it is no formality: a simulator silenced on *less* could claim a party's identity, pass the corruption gate $\mathtt{id}'.\mathsf{F} = \mathsf{A}$ of line 3, and drive an honest party's interface, a capability the adversary it stands in for lacks. **U** is inert, read only by WF, where π^+ names the reserved $\mathcal{A}$ rather than the slot's occupant.

That leaves **N**, which is why Definition 10.3 requires it outright. Widening is impossible, $\mathcal{A}.\mathbf{N}$ already holding every process id that places a call; narrowing gains a simulator nothing and costs both theorems below. Dropping **S** turns every mediation hook at a corrupt party into the deemed $\perp$ of Section 8 — and Theorem 10.15 quantifies over every ρ, so a simulator admitting only φ's process ids meets others in the first outer world that has them. Dropping **Z** shuts the environment out on the ideal side alone, line 2 at the slot refusing the very instructions the real adversary answers. And nothing in any of this mentions the pair being simulated for, which is what lets Theorem 10.15 hand the hypothesis's simulator on unchanged.

Definition 10.5 (Environments)**.** An *environment* is a machine of the execution that occupies $\text{IDs}(\mathcal{Z})$ together with the identities of finitely many standard functionalities it *hosts* — none for the machine of Section 5, copies of the surrounding protocol for the absorbed environments of Definition 10.10 below. It carries the interface of Section 5, silencer and parameter *par* included, at each Z-identity, and each hosted functionality's own full interface at that functionality's identities; EXEC dispatches to it at every identity it occupies, and its output is what its $\mathcal{Z}$ part returns.

Definition 10.6 (Admissible environments)**.** For a system π, write $\langle\pi\rangle := \{\,\mathtt{id} : \mathtt{id}.pid.\mathsf{F} = \mathcal{F}.\mathsf{F} \text{ for some } \mathcal{F} \in \pi\}$ for its *name closure*: the identities of every process id carrying a name π uses, occupied or not, in every session and every instance. For systems π and φ, the environments *admissible* for the pair are

$$\mathbb{Z}_{\pi,\varphi} := \{\, \mathcal{Z} : \mathcal{Z}.par \supseteq \langle\pi\rangle \cup \langle\varphi\rangle \text{ and } \mathcal{Z} \text{ hosts at no identity of } \text{IDs}(\pi) \cup \text{IDs}(\varphi) \,\}.$$

The second condition is occupancy hygiene: an execution is defined only for occupants with pairwise disjoint identity sets (Definition 8.1), so an environment hosting at an identity one of the systems occupies could not be placed beside it, and admissibility rules it out for both worlds at once.

The containment is justified below. A statement of emulation then fixes three classes: a class $\mathbb{A}$ of adversaries (Definition 10.2), a class $\mathbb{S}$ of simulators (Definition 10.3), and a class $\mathbb{Z} \subseteq \mathbb{Z}_{\pi,\varphi}$ of environments (Definition 10.5). Taking $\mathbb{Z} = \mathbb{Z}_{\pi,\varphi}$ gives the strongest statement of the three, and is the default reading.

Definition 10.7 (UC emulation)**.** Let π and φ be systems, let $\varepsilon \geqslant 0$, and let $\mathbb{Z} \subseteq \mathbb{Z}_{\pi,\varphi}$. We say that π *UC-emulates* φ *within* ε *against* $(\mathbb{A}, \mathbb{S}, \mathbb{Z})$ if

$$\forall \mathcal{A} \in \mathbb{A}\ \ \exists \mathcal{S} \in \mathbb{S}\ \ \forall \mathcal{Z} \in \mathbb{Z}\ :\qquad \mathrm{ADV}^{\mathrm{UC}}_{\pi,\varphi}(\mathcal{Z}, \mathcal{A}, \mathcal{S})\ \leqslant\ \varepsilon.$$

The order of the quantifiers is the substance of the definition: the simulator may depend on the adversary but not on the environment, so one $\mathcal{S}$ must work against every $\mathcal{Z} \in \mathbb{Z}$ at once. Otherwise the two executions are one experiment with a single system swapped, $\mathcal{Z}$ being literally the same machine on both sides. Weakening it to $\forall \mathcal{A}\ \forall \mathcal{Z}\ \exists \mathcal{S}$ — a simulator built for the environment it faces — gives a notion Definition 10.7 implies and whose converse we do not prove, the two reading as $\max_{\mathcal{A}} \min_{\mathcal{S}} \max_{\mathcal{Z}}$ and $\max_{\mathcal{A}} \max_{\mathcal{Z}} \min_{\mathcal{S}}$ of one quantity in the concrete form of Section 11. The composition results survive the weakening, each proof below instantiating the hypothesis at an environment fixed by the conclusion's; what the order buys is therefore meaning rather than composition, since only under it is $(\varphi, \mathcal{S})$ one system behaving like $(\pi, \mathcal{A})$ whoever is watching — an ideal world one can name and hand on.

Definition 10.6 is what makes the comparison meaningful. Since $\mathcal{Z}.\mathit{par}$ silences $\mathcal{Z}$, it may claim no identity inside it, and three kinds must be kept out of reach. An identity in $\mathrm{IDS}(\pi) \triangle \mathrm{IDS}(\varphi)$ is occupied in one world and free in the other, so an environment allowed to speak as one is told which world it is in before anything happens, and no simulator can repair that. An identity internal to *both* separates nothing, but an environment claiming it speaks as a component of the system rather than from outside, which an environment is not for. And an *unoccupied* identity of a used name is as good as an occupied one, because GUARD admits callers by name: a local callee admits every instance of its parent's name, line 2 asking no more, so an environment claiming a fresh instance of that name drives internal subroutines exactly as their callers do — and against a blocked pair (Appendix A) it tells the worlds apart outright, the blocker refusing a call the genuine occupant admits. Silencing the closure disallows all three at once. What stays claimable is fresh *names*, and those reach only what admits all of **S**: the global functionalities, shared between the two worlds in any comparison where they are not themselves at issue.

Smaller sets being less restrictive, the least restricted admissible environments are those whose *par* is the closure exactly — and the blocking of Appendix A delivers them for a compatible pair: $\mathrm{IDS}(\overline{\pi}) = \mathrm{IDS}(\overline{\varphi}) = \mathrm{IDS}(\pi) \cup \mathrm{IDS}(\varphi)$ by the second claim of Theorem A.3, and a blocker copies its name from the member it stands in for, so $\langle\overline{\pi}\rangle = \langle\overline{\varphi}\rangle = \langle\pi\rangle \cup \langle\varphi\rangle$. So $\mathcal{Z}.\mathit{par} := \langle\overline{\pi}\rangle$ is not merely a value the two blocked systems can share, but the weakest restriction the pair admits.[3]

Three constructions below — the absorbed environment here, and the two of Appendix D — fold several machines into one occupant of the execution. They share a shape, which we name once.

[3] Quantifying over all environments regardless of *par* is coherent but too strong to be useful: the simulator would have to reproduce φ's internal traffic call by call. Letting the simulator *spawn functionalities* at identities φ does not occupy would relax this, leaving only $\mathrm{IDS}(\varphi) \setminus \mathrm{IDS}(\pi)$ to be withheld — empty for the usual pairing; what spawning costs in composition we leave to future work.

Definition 10.8 (Bundle)**.** A *bundle* runs several machines together as a single occupant of the execution, passing the token among them as EXEC does and initialising each as EXEC would. It occupies a declared set of identities and takes its output and parameters from a designated member. A call between two hosted machines stays inside; a call arriving from outside is served by the member carrying the target identity, through that member's full interface, guard included; a call a hosted machine places outward is placed on the execution when its target lies in a declared *outward set*, and answered REJ otherwise. An instance may in addition route particular calls between its members by hand — interposing one before another, even when the target lies outside every member's identities — and such an explicit routing governs the calls it names, leaving only the rest to the target rule above.
How a hosted machine's outward call is claimed turns on occupancy, since a silencer wraps the core of one interface and reaches no further (Section 2). A member *whose identities the bundle occupies* leaves its calls under its own identity, meeting its own silencer alone; a member *whose identities the bundle does not occupy* has none to claim, so the bundle rewrites each of its outward calls to leave under an identity it does hold, and the rewritten claim meets that interface's silencer. Which case a member is in is what keeps a bundle faithful to the execution it stands for.

Definition 10.8 already says that a hosted machine comes in two kinds, according to whether the bundle occupies its identities, and the difference is the whole of what follows. A machine whose identities the bundle keeps disappears from the execution's dispatch and is served inside; one whose identities the bundle cedes must leave them to somebody, and its calls must be placed by that somebody on its behalf. Both moves come up — the first absorbs a protocol, the second an adversary — so we define them together and prove them right once.

Definition 10.9 (Absorption)**.** Let $\mathcal{Z}$ be an environment and H a finite set of machines of an execution, split as $H = H_{\text{KEEP}} \uplus H_{\text{CEDE}}$, the members of H_{CEDE} being occupants of the adversary slot. The *absorption* of H into $\mathcal{Z}$ is the bundle (Definition 10.8) $\mathcal{Z}^H$ of $\mathcal{Z}$ with one copy of each member of H; it occupies $\text{IDS}(\mathcal{Z}) \cup \text{IDS}(H_{\text{KEEP}})$, takes its output and *par* from $\mathcal{Z}$, and has as outward set the identities the surrounding execution still serves. A member of H_{KEEP} keeps its identities, so its calls leave under its own claim and calls addressed to it are served inside the bundle. A member of H_{CEDE} keeps none, so the bundle rewrites each outward call $\mathtt{id}''.\text{FULLOP}(x)$ it places while running at (A, P) as the instruction

$$(A, P).\text{FULLA}\big(\mathtt{id}''.\text{OP},\ x\big) \ \text{ as } (Z, P),$$

and the identities it cedes are occupied in the new execution by a *relay* for it, which throughout is the dummy $\mathcal{D}$ of Appendix D.

We state the ceding case for the adversary slot alone. It is the only one used below, and the only one whose interface has an instruction shape to rewrite calls into — a standard functionality serves what its own code names and could not stand in for another machine's calls.

Definition 10.10 (Absorbed environment)**.** Let $\varphi \subseteq \rho$, let π be a system to be put in place of φ, and let $\mathcal{Z}$ be an environment. The *absorbed* environment $\mathcal{Z}^{\rho\setminus\varphi}$ is the absorption (Definition 10.9) of $\rho \setminus \varphi$ into $\mathcal{Z}$ with every member kept and none ceded, so it is the bundle of $\mathcal{Z}$ with one copy of each $\mathcal{F} \in \rho \setminus \varphi$, every copy keeping its identities; it occupies $\mathrm{IDs}(\mathcal{Z}) \cup \mathrm{IDs}(\rho \setminus \varphi)$, takes its output and *par* from $\mathcal{Z}$, and has outward set $\mathrm{IDs}(\pi) \cup \mathrm{IDs}(\varphi) \cup \{\mathtt{id} : \mathtt{id}.\mathsf{F} \in \{\mathsf{A}, \mathsf{C}\}\}$. It is an environment (Definition 10.5), not a functionality, since it spans several process ids. The superscript records the hosted shell only — the outward set reads π as well — so $\mathcal{Z}^{\rho\setminus\varphi}$ is fixed relative to a replacement given by context, and two replacements sharing $\rho \setminus \varphi$ but differing in π name different machines.

Both halves of the outward set are forced by what the absorbed execution has to reproduce. The first is the slot, whose identities go to π or to φ according to which world the absorbed environment sits in. We take the union rather than $\mathrm{IDs}(\varphi)$ alone so the same machine serves in both, which matters now that the two need not occupy the same identities. The second is the two reserved ids A and C, named as ids rather than as machines because the adversary slot holds $\mathcal{S}$ on the ideal side while Proposition 10.14 needs one and the same absorbed machine on both. They are there because the hosted machines are *full* interfaces rather than bare cores: each reads the register on lines 2 and 6 of Section 7, and each hands a call at a corrupt party to the adversary through MEDIATE, which hooks to $(\mathsf{A}, \mathtt{id}.\mathsf{P})$. Neither id belongs to any system. Without this half a call to a hosted functionality would be refused at its first line, and a corrupt party's hook would be refused instead of reaching the adversary: the absorbed execution would stop being the outer one regrouped exactly when **C** met $\rho \setminus \varphi$. Everything else is answered with REJ, nothing serving it in the outer execution either.

The parameter is carried over untouched, and it must be. It is the silenced set of $\mathcal{Z}$'s own interface, so any change to it changes which calls $\mathcal{Z}$ may place — and $\mathcal{Z}$ is the one machine that has to behave identically on both sides. One might think the hosted functionalities need $\mathrm{IDs}(\rho \setminus \varphi)$ removed from it to speak as themselves, but they do not: each is a full interface, and line 5 of its own wrapper is what licenses its claim, $\mathcal{Z}$'s reaching no further than $\mathcal{Z}$'s own core (Definition 10.8). Removing those identities would licence nothing new for them, and would un-silence $\mathcal{Z}$ on exactly the identities it was silenced on before absorption.

Absorbing thus turns an environment for the outer pair into one for the inner pair, preserving admissibility. The register needs no special treatment: being a member of no system it lies in neither φ nor $\rho \setminus \varphi$, so the absorbed environment does not host it and blocking builds no blocker for it. Like $\mathcal{Z}$ and $\mathcal{A}$ it is supplied by EXEC, the same single machine on both sides of every comparison. Its hosted copies keep their identities, so by Definition 10.8 they leave their calls under their own claims and meet their own silencers — not $\mathcal{Z}$'s — which is what the regrouping below needs; the constructions of Appendix D are the opposite case, hosting an adversary whose identities the bundle does not occupy.

Proposition 10.11 (Absorption preserves admissibility)**.** *Let ρ and π be systems, let $\varphi \subseteq \rho$, and let the replacement of φ by π in ρ be well-formed. Then*

$$\mathcal{Z} \in \mathbb{Z}_{\rho[\pi/\varphi],\,\rho} \qquad \Longrightarrow \qquad \mathcal{Z}^{\rho\setminus\varphi} \in \mathbb{Z}_{\pi,\varphi}.$$

Proof. For the *par* half, absorption carries the field over untouched, so it is enough that $\langle \rho[\pi/\varphi] \rangle \cup \langle \rho \rangle \supseteq \langle \pi \rangle \cup \langle \varphi \rangle$. Every name φ uses is used by ρ, since $\varphi \subseteq \rho$, and every name π uses is used by $\rho[\pi/\varphi]$, since $\pi \subseteq \rho[\pi/\varphi]$. For the hosting half, $\mathcal{Z}^{\rho \setminus \varphi}$ hosts at $\mathrm{IDs}(\rho \setminus \varphi)$ beyond whatever $\mathcal{Z}$ hosted: the latter avoids $\mathrm{IDs}(\rho[\pi/\varphi]) \cup \mathrm{IDs}(\rho)$, which contains $\mathrm{IDs}(\pi) \cup \mathrm{IDs}(\varphi)$; and $\mathrm{IDs}(\rho \setminus \varphi)$ meets $\mathrm{IDs}(\varphi)$ nowhere, the two being disjoint subsystems of ρ, and meets $\mathrm{IDs}(\pi)$ nowhere by well-formedness of the replacement — the one place the hypothesis is used, and it is used for typing alone. No relation between $\mathrm{IDs}(\pi)$ and $\mathrm{IDs}(\varphi)$ enters. One thing the statement does not say: the absorbed machine *claims* identities inside its own *par*, each hosted copy claiming its own. That is no contradiction — by Definition 10.8 a hosted copy's claims meet its own silencer and never the environment's, and admissibility is a condition on the field, read where the field applies. □

What makes the theorem work is that absorption regroups an execution without changing it. Since the same correspondence serves Appendix D, we set it down before the lemma.

Configurations. By a *configuration* of an execution we mean its state between activations: the state of every machine, the unread part of every coin tape, which machine holds the token and where in its code, and the callers suspended awaiting answers. Calls nest, and Section 8 lets an instance hold several outstanding continuations, so the last of these is a chain rather than a set: one entry per call placed and not yet answered, in the order placed, recording the caller and the call it waits on.

The two executions do not run the same machines — that is what absorbing does — but they run the same ones once $\mathcal{Z}^{\mathsf{H}}$ is counted as the machines it bundles rather than as one, and but for the relays a ceded identity needs. Definition 10.8 passes the token among those as EXEC does, so each holds it in the same sense on either side. Counted so, the machines of the two executions correspond one to one apart from the relays, and we write ι for the bijection: the hosted copy of each member of H to that member, the hosted $\mathcal{Z}$ to $\mathcal{Z}$, and everything the execution still runs, $\mathcal{C}$ included, to itself.

Definition 10.12 (Regrouping correspondence). A configuration c of the left execution and a configuration $\hat{c}$ of the right *correspond*, written $c \approx \hat{c}$, when

- (C1) every machine of $\hat{c}$ and its image under ι are in the same state — $\mathcal{C}$ included, so the two agree on $\mathbf{C}$;
- (C2) every machine and its image have the same unread coin tape;
- (C3) the same machine holds the token in both, under ι, at the same point of the same operation and with the same local values; and
- (C4) the two chains of suspended callers agree under ι: equal length, and for each k the k-th entries name machines ι identifies, waiting on the same call — same target identity, operation, input and claimed identity.

Where the right-hand family carries a relay, ι is a bijection between the machines of the left and those of the right *other than* the relay, and (C4) is read with the relay's own frames elided: a relayed call contributes one frame the left does not have, resumed exactly once with the answer the left's call returned — save that a REJ the left reads may arrive as $\perp$ on the right, which is the one place refusal-obliviousness is spent.

Only (C4) is at risk from the bracketing. On the left every suspension is recorded by

EXEC; on the right they divide between EXEC and the bundle, which records those of the calls internal to it. The clause asks that the two chains agree once the bundle's own are counted where they were placed. Regrouping is exactly that: one chain of suspensions, kept by two dispatchers.

The first activation. The two executions begin in correspondence, and it is worth saying why, since this is the one activation the lemma's case analysis does not reach: it is placed by no machine. Line 1 initialises $\mathcal{C}$ and everything the execution still runs on both sides, Definition 10.9 initialises each hosted copy as EXEC would, and a relay carries no state, so every machine and its image start in the same state, which is (C1). No tape has been read, giving (C2). INITIALIZE places no calls, so nothing is suspended and both chains are empty, giving (C4) — and with it the reason the order in which the bundle initialises its copies cannot be observed: that order can be read off nothing in the configuration. Line 2 then activates $(Z, Z).\text{FULLZ}()$ from (Z, Z). On the left the token goes to $\mathcal{Z}$; on the right it goes to $\mathcal{Z}^H$, where (Z, Z) is carried by the hosted $\mathcal{Z}$, which Definition 10.9 makes the bundle occupy. Both enter FULLZ at the same identity on the same input, which is (C3).

Thereafter every activation is a call placed or a value returned, and the lemma comes to showing that correspondence survives one of them on either side, and that corresponding configurations halt together with the same output. Its paragraphs take those activations case by case.

Lemma 10.13 (Absorption). *Let an execution run the occupants $\{\mathcal{Z}\} \cup H \cup R$, with $\mathcal{C} \in R$ and $\mathcal{Z}$ an environment, let $H = H_{\text{KEEP}} \uplus H_{\text{CEDE}}$, and let $\mathcal{Z}^H$ be the absorption of Definition 10.9. Ask of each member of H_{CEDE}, and of nothing in H_{KEEP}, that*

(a) it be refusal-oblivious (Definition D.3);

(b) no core of R place a responsive call on the identities it cedes (Definition F.2); and

(c) $\mathcal{Z}$.par contain no Z-identity.

Then, the two families being legal occupant families in the sense of Definition 8.1,

$$\text{EXEC}\big(\{\mathcal{Z}\} \cup H \cup R\big) \qquad \textit{and} \qquad \text{EXEC}\big(\{\mathcal{Z}^H\} \cup R \cup \{\textit{a relay at each ceded identity}\}\big)$$

are identically distributed — equal as distributions, not merely close.

Proof. Two conventions of Sections 1 and 8 are used throughout and are worth restating. Distinct machines draw from independent coins, and bundling machines into one does not merge their tapes, so a hosted copy's randomness is what it was before absorption. And INITIALIZE places no calls, so the order in which the bundle initialises its hosted copies is unobservable. The proof is an induction on activations: the initial configurations correspond, as shown before the lemma; corresponding configurations produce the same next activation on both sides and step to corresponding configurations, each machine and its image reading the same coins in the same order, which is what (C2) of Definition 10.12 preserves; and corresponding halting configurations output alike, the output being the hosted $\mathcal{Z}$'s by Definition 10.9. The paragraphs below discharge the inductive step: which machines exist and start alike, (C1); where a call goes and what it claims, (C4); and how the token moves, (C3). The first two paragraphs are common to both kinds of member; the two after them are what a ceded member costs.

Machines. The left runs $\{\mathcal{Z}\} \cup H \cup R$; the right runs $\{\mathcal{Z}^H\} \cup R$ together with a relay at each

ceded identity, and $\mathcal{Z}^{H}$ is $\mathcal{Z}$ with one copy of each member of H. Counted as the machines it bundles, the two collections coincide but for the relays, so ι pairs each member of H with its copy, each member of R and $\mathcal{C}$ with itself, and leaves the relays over, which is the reading Definition 10.12 gives them. Line 1 initialises R on both sides, Definition 10.9 initialises the hosted copies as EXEC would, and a relay is stateless, so every machine and its image start alike. Occupancy is disjoint on both sides by hypothesis, so each call has a unique target: on the right the bundle holds $\mathrm{IDs}(\mathcal{Z}) \cup \mathrm{IDs}(H_{\mathrm{KEEP}})$ and the relays hold what H_{CEDE} let go.

Routing. Take a call and ask where it lands. With both endpoints in $\{\mathcal{Z}\} \cup H$ it is dispatched by EXEC on the left and served inside $\mathcal{Z}^{H}$ on the right, by the same machine either way. Leaving that group it is dispatched by EXEC on the left and placed outward on the right, its target lying in the outward set, which Definition 10.9 makes the identities the execution still serves. Entering the group at an identity of H_{KEEP}, it is dispatched by EXEC on the left and served on the right by the hosted copy carrying that identity, which the bundle occupies. Entering at an identity of H_{CEDE}, it is dispatched to the ceded member on the left and to the relay on the right, which is the one place the two families differ and the subject of the two paragraphs below. Addressed anywhere else it has no machine on either side: EXEC finds no target and $\mathcal{Z}^{H}$ answers REJ. Guards are evaluated at the callee from the claimed id, so they agree once the claims do.

Claims, for a member that keeps its identities. Each such call carries the same claimed id on both sides, and the two silencers that could change one are the same silencers on both. A hosted interface is governed by its own silencer and not by the environment's, by Definition 10.8: it keeps its identities, so SILENCE at $\mathcal{Z}$ does not reach it. Being a full interface it claims exactly its own identity on either side, line 5 silencing its core on $\{\mathrm{id}'' \neq \mathrm{id}\}$; and the calls its wrapper places, on $\mathcal{C}$ and through MEDIATE, stand outside even that silencer on both sides. The environment's own interface is governed by *par*, which absorption carries over unchanged, so $\bar{\jmath}$ is computed from the same set at the same identity and admits exactly the same claims. Neither machine gains nor loses a claim.

Claims, for a member that cedes them. Such a member has no identity of its own to claim inside the bundle, so Definition 10.9 rewrites its outward calls, and the rewriting has to survive three tests. First, where the bundle may speak from. Whenever the hosted member runs on behalf of party P — that is, where the left has it running at (A, P) — the bundle is running at (Z, P) or at (Z, Z), or else $P = Z$: it is entered either by $\mathcal{Z}$ calling an A-identity, served internally with $\textsc{Guard}_{\mathcal{A}}$ forcing the claimed party to be the target's, and line 3 letting an interface at (Z, Q) claim party P only when $Q \in \{P, Z\}$ or $P = Z$; or by the relay, whose inward call targets (Z, P) exactly. Second, whether the claim (Z, P) is silenced: not by the party-tie just settled, not by line 2 since its name is Z, and not by *par*, which hypothesis (c) empties of Z-identities. Third, whether it passes GUARD at the relay: $P \in \mathcal{A}.\mathbf{P}$ at line 1, $\mathbf{Z} \subseteq \mathcal{A}.\mathbf{N}$ with the parties equal at line 2, and GLOBAL at line 3. The relay then places the very call the left placed, under the very claim the left claimed, so both sides meet the same conjuncts at the same callee. A call arriving *at* the ceded identity travels the other way by the same three tests: it reaches the relay under the guard it passed on the left, is handed to $\mathcal{Z}$ at (Z, P) with its caller in the payload — admitted by the relay's own silencer and by $\textsc{Guard}_{\mathcal{Z}}$, $\mathbf{A} \subseteq \mathcal{Z}.\mathbf{N}$ and $\textsc{Global}(\mathcal{Z})$ — and reaches the hosted member's interface at (A, P) through that interface's guard, the guard that passed on the left.

Refusals. One value can differ. Where a call the ceded member places is refused, the left reads REJ from its own call, while on the right the relay returns that REJ and Section 8 delivers it to the hosted member as $\perp$. Hypothesis (a) is exactly what closes this: a refusal-

oblivious machine answers alike to the two, so its distribution is unchanged. Hypothesis (b) is what stops the other direction of the same problem, a responsive call on a ceded identity: RESPOND refuses every call the responder places, and the relay must place one to answer at all, so a responsive call on a relayed slot would be answered $\perp$ where the left answered with substance — which is why no core of R may place one. Nothing else the relay does is visible: CORRUPT survives the detour, its gate $\mathtt{id}'.\mathrm{F} = \mathrm{A}$ met by the relay's claim, which is the one call the rewriting must place under an A-identity and the bundle cannot.

Token. EXEC has one operation, placing a call, and its continuation is where the token comes back. On the right the calls internal to $\mathcal{Z}^{\mathrm{H}}$ no longer reach that handler, but Definition 10.8 passes the token among the hosted machines as EXEC does, so each internal caller is suspended on its own continuation and resumed with the same answer; and each relayed call adds one frame on the right, resumed exactly once, which is the elision Definition 10.12 allows. Two properties of EXEC are used, and Section 8 grants both to every instance: a machine may hold several outstanding continuations, and may be re-entered while suspended. The second is what a call arriving from R needs while $\mathcal{Z}^{\mathrm{H}}$ is itself waiting on an outward call. Exactly one machine holds the token at every moment on either side.

Every machine therefore sees the same inputs in the same order and answers from the same tape, so correspondence is preserved at every activation and the two executions agree as distributions — in particular on the probability that $\mathcal{Z}$ returns 1. □

The two halves of the lemma are used apart. Absorbing a protocol cedes nothing, so all three hypotheses fall away and what is left is a pure rebracketing; that is the case Theorem 10.15 runs on, and it is worth stating separately for the emphasis that it costs nothing at all.

Proposition 10.14 (Regrouping). *Let ρ and π be systems, let $\varphi \subseteq \rho$, let the replacement of φ by π in ρ be well-formed, and let $\mathcal{Z}$ be an environment hosting at no identity of* $\mathrm{IDS}(\rho) \cup \mathrm{IDS}(\pi)$. *Then for σ either π or φ, and for any occupant $\mathcal{X}$ of the adversary slot (Definition 8.1),*

$$\mathrm{EXEC}\big(\rho[\sigma/\varphi],\, \mathcal{Z},\, \mathcal{X},\, \mathcal{C}\big) \qquad \textit{and} \qquad \mathrm{EXEC}\big(\sigma,\, \mathcal{Z}^{\rho\setminus\varphi},\, \mathcal{X},\, \mathcal{C}\big)$$

are identically distributed — equal as distributions, not merely close.

Proof. Lemma 10.13 at $\mathrm{H} = \rho \setminus \varphi$ with $\mathrm{H}_{\mathrm{CEDE}} = \varnothing$, so that $\mathcal{Z}^{\mathrm{H}}$ is the absorbed environment of Definition 10.10 and no hypothesis of the lemma applies: (a), (b) and (c) are asked of ceded members and there are none. Note $\rho[\varphi/\varphi] = \rho$, so the case $\sigma = \varphi$ is the ideal side. The two occupant families are legal: the left-hand collection is a system in both cases, so no identity is occupied twice — for $\sigma = \pi$ because well-formedness says $\rho \setminus \varphi$ and π share no process id, and for $\sigma = \varphi$ because that collection is ρ itself — while on the right the hosting hypothesis on $\mathcal{Z}$ keeps the bundle clear of $\mathrm{IDS}(\sigma)$. □

The theorem now says that substituting one subsystem for another inside a protocol costs nothing in advantage.

Informally, Theorem 10.15 says: if π UC-emulates φ within ε, then $\rho[\pi/\varphi]$ UC-emulates ρ within the same ε. The adversary and simulator classes carry over unchanged; the environments of the conclusion are those admissible for the pair $(\rho[\pi/\varphi], \rho)$ whose absorbed form $\mathcal{Z}^{\rho\setminus\varphi}$ lies in the class of the hypothesis.

Theorem 10.15 (Composition). *Let ρ and π be systems, let $\varphi \subseteq \rho$, and let the replacement of φ by π in ρ be well-formed in the sense of Definition 9.2. Suppose*

$$\pi \text{ UC-emulates } \varphi \text{ within } \varepsilon \text{ against } (\mathbb{A}, \mathbb{S}, \mathbb{Z}).$$

Then

$$\rho[\pi/\varphi] \text{ UC-emulates } \rho \text{ within } \varepsilon \text{ against } (\mathbb{A}', \mathbb{S}', \mathbb{Z}'),$$

where

$$\mathbb{A}' = \mathbb{A}, \qquad \mathbb{S}' = \mathbb{S}, \qquad \mathbb{Z}' = \left\{ \mathcal{Z} \in \mathbb{Z}_{\rho[\pi/\varphi],\rho} \,:\, \mathcal{Z}^{\rho\setminus\varphi} \in \mathbb{Z} \right\}.$$

Explicitly,

$$\forall \mathcal{A} \in \mathbb{A}' \; \exists \mathcal{S} \in \mathbb{S}' \; \forall \mathcal{Z} \in \mathbb{Z}' : \qquad \mathrm{ADV}^{\mathrm{UC}}_{\rho[\pi/\varphi],\rho}(\mathcal{Z}, \mathcal{A}, \mathcal{S}) \leqslant \mathrm{ADV}^{\mathrm{UC}}_{\pi,\varphi}\left(\mathcal{Z}^{\rho\setminus\varphi}, \mathcal{A}, \mathcal{S}\right) \leqslant \varepsilon.$$

Proof. The three relations are of three kinds. The adversaries carry over unchanged because the reduction hands on the very same machine: an $\mathcal{A} \in \mathbb{A}'$ attacking $\rho[\pi/\varphi]$ is read as an $\mathcal{A} \in \mathbb{A}$ attacking π. Any $\mathbb{A}' \subseteq \mathbb{A}$ would serve, and equality is the largest such choice. The simulators carry over for the mirror reason: the $\mathcal{S} \in \mathbb{S}$ the hypothesis supplies is returned as the simulator for the conclusion, so any $\mathbb{S}' \supseteq \mathbb{S}$ would serve, and equality is the smallest. That it may be returned at all is Remark 10.4: nothing typing a simulator mentions the pair it simulates for, so one for (π, φ) already stands in the slot for $(\rho[\pi/\varphi], \rho)$.

The environments are the one place something is computed rather than copied. $\mathbb{Z}$ and $\mathbb{Z}'$ are classes for two different pairs of systems, and $\mathbb{Z}'$ is the preimage of $\mathbb{Z}$ under absorption,

$$\mathbb{Z}' = \left((\cdot)^{\rho\setminus\varphi}\right)^{-1}(\mathbb{Z}) \cap \mathbb{Z}_{\rho[\pi/\varphi],\rho},$$

so an outer environment counts exactly when what it becomes on absorption is one the hypothesis covers. It is a genuine class, since $\mathbb{Z}' \subseteq \mathbb{Z}_{\rho[\pi/\varphi],\rho}$ by construction. When $\mathbb{Z}$ is the largest admissible class $\mathbb{Z}_{\pi,\varphi}$, Proposition 10.11 makes the second condition redundant and the preimage is all of $\mathbb{Z}_{\rho[\pi/\varphi],\rho}$: emulation against the largest class gives the largest class again.

Proposition 10.11 also makes the right-hand advantage defined, needing no closure hypothesis on the environments and no relation between $\mathrm{IDs}(\pi)$ and $\mathrm{IDs}(\varphi)$. The first inequality is Proposition 10.14 applied twice, with $\sigma = \pi$ and $\mathcal{X} = \mathcal{A}$ on the real side and $\sigma = \varphi$ and $\mathcal{X} = \mathcal{S}$ on the ideal one. Subtracting,

$$\mathrm{ADV}^{\mathrm{UC}}_{\rho[\pi/\varphi],\rho}(\mathcal{Z}, \mathcal{A}, \mathcal{S}) = \mathrm{ADV}^{\mathrm{UC}}_{\pi,\varphi}\left(\mathcal{Z}^{\rho\setminus\varphi}, \mathcal{A}, \mathcal{S}\right),$$

so it is in fact an equality: the two experiments are one experiment bracketed differently, and the same $\mathcal{S}$ serves on both sides because the ideal execution is regrouped by the very same absorption. The second inequality is the emulation hypothesis applied to $\mathcal{Z}^{\rho\setminus\varphi}$, which lies in $\mathbb{Z}$ because $\mathcal{Z}$ lies in $\mathbb{Z}'$. The theorem displays the first relation as an inequality only for uniformity with the second. □

Proposition 10.16 (Transitivity). *Let π, φ and ψ be systems and let $\mathbb{Z} \subseteq \mathbb{Z}_{\pi,\varphi} \cap \mathbb{Z}_{\varphi,\psi}$. If π UC-emulates φ within ε_1 against $(\mathbb{A}, \mathbb{S}, \mathbb{Z})$ and φ UC-emulates ψ within ε_2 against*

$(\mathbb{S}, \mathbb{S}', \mathbb{Z})$, *then* π *UC-emulates* ψ *within* $\varepsilon_1 + \varepsilon_2$ *against* $(\mathbb{A}, \mathbb{S}', \mathbb{Z})$.

Proof. First, the conclusion is a legal statement of emulation. A $\mathcal{Z}$ in both classes has $\mathcal{Z}.par \supseteq \langle\pi\rangle \cup \langle\varphi\rangle$ and $\mathcal{Z}.par \supseteq \langle\varphi\rangle \cup \langle\psi\rangle$, hence $\mathcal{Z}.par \supseteq \langle\pi\rangle \cup \langle\psi\rangle$; and it hosts at no identity of $\mathrm{IDs}(\pi) \cup \mathrm{IDs}(\varphi)$ nor of $\mathrm{IDs}(\varphi) \cup \mathrm{IDs}(\psi)$, hence at none of $\mathrm{IDs}(\pi) \cup \mathrm{IDs}(\psi)$ — so both clauses of Definition 10.6 hold and $\mathbb{Z} \subseteq \mathbb{Z}_{\pi,\psi}$; the three systems need not be related for this. Now let $\mathcal{A} \in \mathbb{A}$. The first emulation supplies $\mathcal{S} \in \mathbb{S}$, and since the second quantifies its adversaries over $\mathbb{S}$, reading $\mathcal{S}$ as an adversary against φ yields $\mathcal{S}' \in \mathbb{S}'$. For every $\mathcal{Z} \in \mathbb{Z}$ the triangle inequality gives

$$\mathrm{Adv}^{\mathrm{UC}}_{\pi,\psi}(\mathcal{Z}, \mathcal{A}, \mathcal{S}') \leqslant \mathrm{Adv}^{\mathrm{UC}}_{\pi,\varphi}(\mathcal{Z}, \mathcal{A}, \mathcal{S}) + \mathrm{Adv}^{\mathrm{UC}}_{\varphi,\psi}(\mathcal{Z}, \mathcal{S}, \mathcal{S}') \leqslant \varepsilon_1 + \varepsilon_2,$$

since the middle term $\Pr[\textsc{Exec}(\varphi, \mathcal{Z}, \mathcal{S}, \mathcal{C}) = 1]$ cancels. As $\mathcal{S}'$ depends on $\mathcal{A}$ alone, it is the simulator required. □

Corollary 10.17 (Composition with a specification). *Let* π, ρ *and* ψ *be systems, let* π *UC-emulate* φ *within* ε_1 *against* $(\mathbb{A}, \mathbb{S}, \mathbb{Z})$, *let* $\varphi \subseteq \rho$, *and let the replacement of* φ *by* π *in* ρ *be well-formed. Write* $\mathbb{Z}'$ *for the class of Theorem 10.15. If* ρ *UC-emulates* ψ *within* ε_2 *against* $(\mathbb{S}, \mathbb{S}', \mathbb{Z}')$, *then* $\rho[\pi/\varphi]$ *UC-emulates* ψ *within* $\varepsilon_1 + \varepsilon_2$ *against* $(\mathbb{A}, \mathbb{S}', \mathbb{Z}')$.

Proof. By Theorem 10.15, $\rho[\pi/\varphi]$ UC-emulates ρ within ε_1 against $(\mathbb{A}, \mathbb{S}, \mathbb{Z}')$, and $\mathbb{Z}' \subseteq \mathbb{Z}_{\rho[\pi/\varphi],\rho}$. The second hypothesis is a statement of emulation against that same class, so $\mathbb{Z}' \subseteq \mathbb{Z}_{\rho,\psi}$ as well. Both containments needed by Proposition 10.16 therefore hold, and it applies to the two statements with ρ in the middle. □

This is the form in which composition is used. One analyzes ρ with the idealized subroutine φ in place, proving it emulates whatever specification ψ is wanted — against the classes the corollary names, whose adversaries are the first step's simulators; the corollary transfers that conclusion to the system that runs. One hypothesis is easy to miss: emulation against $\mathbb{Z}'$ asserts, as Definition 10.7's side condition, that $\mathbb{Z}' \subseteq \mathbb{Z}_{\rho,\psi}$ — so ψ's names must lie under the *par* of every environment the class retains, and the corollary is vacuous where they do not.

11 Concrete Security

Definition 10.7 is stated against arbitrary classes and says nothing about what they contain. Filling them with machines of bounded resources turns each statement of the previous section into a concrete one, and the relations *between* classes into arithmetic on budgets.

Definition 11.1 (Budgets and bounded classes). A *budget* is a pair $\mathbf{a} = (t, r)$ of naturals. A machine is $\mathbf{a}$-*bounded* if it takes at most t steps in each invocation and answers at most r calls in all; so it runs for at most $t\,r$ steps in any execution. A machine meeting both bounds in every execution is counted $\mathbf{a}$-bounded whether or not the halting rule is written into it — the bundles built from bounded machines below are bounded in this sense. Write $\mathbb{A}(\mathbf{a})$, $\mathbb{S}(\mathbf{a})$ and $\mathbb{Z}_{\pi,\varphi}(\mathbf{a})$ for the classes of $\mathbf{a}$-bounded adversaries, of $\mathbf{a}$-bounded simulators, and of $\mathbf{a}$-bounded environments lying in $\mathbb{Z}_{\pi,\varphi}$. Budgets are

ordered componentwise; machines that share an execution — a bundle of several, or a simulator running an adversary — combine by

$$(t, r) \oplus (t', r') \;:=\; \big(\max(t, t'),\; r + r'\big),$$

one machine running per activation, so the per-invocation time takes the larger bound while the invocation counts add.

The budget is part of the machine, not a promise about it. Bounding the three slot machines is not yet enough to bound an execution — the members of a system are machines too, and a core that recurses on itself, or two members calling one another forever, would carry the token away from every budgeted machine with nothing to reclaim it. Concrete statements therefore read over *budgeted systems*: every member of every system compared carries a budget of its own, in the sense of this definition applied verbatim. An execution then runs finitely many machines, each taking finitely many steps, and exactly one machine steps at any moment, so the execution halts within the total step bound $\sum_i t_i\, r_i$ of its machines and EXEC is defined outright. Halting at a budget is given a value: a machine that halts mid-call resumes each caller suspended on it with $\perp$, and an environment that halts away from its root return is read as returning 0. The classes are monotone in the budget, $\mathbb{A}(\mathbf{a}) \subseteq \mathbb{A}(\mathbf{a}')$ whenever $\mathbf{a} \leqslant \mathbf{a}'$, and likewise for $\mathbb{S}$ and $\mathbb{Z}_{\pi,\varphi}$. That makes Lemma C.2 a statement about numbers: shrinking the adversary budget, growing the simulator budget, or shrinking the environment budget each preserve emulation.

Definition 11.2 (Concrete UC advantage)**.** Let π and φ be budgeted systems, let $\mathbf{a}$ and $\mathbf{s}$ be budgets for the adversary and the simulator, and let $\mathbb{Z} \subseteq \mathbb{Z}_{\pi,\varphi}$ be a class of environments. Set

$$\mathrm{ADV}^{\mathrm{UC}}_{\pi,\varphi}[\mathbf{a}, \mathbf{s}, \mathbb{Z}] \;:=\; \max_{\mathcal{A} \in \mathbb{A}(\mathbf{a})} \; \min_{\mathcal{S} \in \mathbb{S}(\mathbf{s})} \; \max_{\mathcal{Z} \in \mathbb{Z}} \; \mathrm{ADV}^{\mathrm{UC}}_{\pi,\varphi}(\mathcal{Z}, \mathcal{A}, \mathcal{S}),$$

and abbreviate $\mathrm{ADV}^{\mathrm{UC}}_{\pi,\varphi}[\mathbf{a}, \mathbf{s}, \mathbf{z}]$ for the case $\mathbb{Z} = \mathbb{Z}_{\pi,\varphi}(\mathbf{z})$, the largest class of that budget.

The third argument is a class rather than a budget because the largest class depends on the pair of systems, and the statements below compare advantages taken over *different* pairs. Where a common class is needed the notation must be able to say so.

The three operators are the three quantifiers of Definition 10.7 read as a value, in the same order and for the same reason: the simulator is chosen after the adversary and before the environment. The extrema are attained, though the reason takes a moment, and it is finiteness rather than compactness. An $\mathbf{a}$-bounded machine takes at most $t\, r$ steps in all, so it reads at most $t\, r$ symbols of any input and at most $t\, r$ coins; up to behaviour it is a map from what it read to what it did, and there are finitely many such maps. Its coins weight them with dyadic probabilities of denominator 2^{tr}, so a bounded class induces finitely many behaviour distributions in all. With the members of the systems budgeted as well, $\Pr[\mathrm{EXEC} = 1]$ is a well-defined number at each choice of behaviours, so each advantage of Definition 10.1 takes one of finitely many values, and a maximum or minimum over finitely many values is attained. (Over an arbitrary class $\mathbb{Z}$ the outer max is read as a supremum.)

Therefore

$$\pi \text{ UC-emulates } \varphi \text{ within } \varepsilon \text{ against } \big(\mathbb{A}(\mathbf{a}), \mathbb{S}(\mathbf{s}), \mathbb{Z}_{\pi,\varphi}(\mathbf{z})\big)$$

holds exactly when that quantity is at most ε — the attained minimum is what makes the reverse direction exact at the boundary. A concrete security claim is then a single inequality relating three budgets and a probability, with no security parameter and no asymptotics in sight.

12 Properties

Emulation compares two systems. A *property* asks something of one: that no adversary win a game played against it — which Proposition 12.9 will reduce to the dummy's not winning, the slot absorbing into the environment. Little new machinery is wanted for either. A game is a functionality like any other, the environment is the adversary that plays it, and the plumbing of the previous sections already settles who may call what — so a property is a system, an environment class, and an event, and the fields of Definition 1.1 do most of the work that a game's rules do on paper. What has to be added is small, and is named as it comes: one hygiene condition on the game system, and the three conditions Appendix D already asks of anything it moves.

Definition 12.1 (Property functionality, game system). A *property functionality* for $\mathcal{F}$, or a *game*, is a standard functionality $\mathcal{G}\mathfrak{m}$ with

$$\mathcal{G}\mathfrak{m}.\mathit{pid} := (\mathcal{G}\mathfrak{m}.\mathsf{F}, 0, 0), \qquad \mathcal{G}\mathfrak{m}.\mathbf{P} := \mathcal{F}.\mathbf{P} \cup \{\mathsf{Z}\},$$
$$\mathcal{G}\mathfrak{m}.\mathbf{N} := \mathbf{Z}, \qquad \mathcal{G}\mathfrak{m}.\mathbf{U} := \{(\mathcal{F}, \textsc{serves}_{\mathcal{G}\mathfrak{m}})\},$$

carrying beside its own interfaces a distinguished FINALIZE, served at the root and returning a bit, and no interface of which is served once FINALIZE has been. Here

$$\textsc{serves}_{\mathcal{G}\mathfrak{m}}(\mathcal{G}\mathfrak{m}, \mathcal{F}) \;:\Longleftrightarrow\; \mathcal{F}.\mathbf{P} \supseteq \mathcal{G}\mathfrak{m}.\mathbf{P} \setminus \{\mathsf{Z}\} \;\wedge\; \mathcal{G}\mathfrak{m}.\mathit{pid} \in \mathcal{F}.\mathbf{N}$$

is SERVES with the root exempted. A *game system* is $\gamma := \{\mathcal{G}\mathfrak{m}\} \cup \sigma$ for a system σ containing $\mathcal{F}$ and whatever $\mathcal{F}$ uses and not containing $\mathcal{G}\mathfrak{m}$, with $\textsc{wf}(\gamma)$ and with γ *tight at* $\{\mathcal{G}\mathfrak{m}\}$ in the sense of Remark A.5; we write $\textsc{Adv}_{\sigma}^{\mathcal{G}\mathfrak{m}}$ below and abbreviate it $\textsc{Adv}_{\mathcal{F}}^{\mathcal{G}\mathfrak{m}}$ when $\sigma = \{\mathcal{F}\}$.

Each field is forced, and it is worth saying by what.

The process id. There is one game per experiment, so the session and instance carry no information and we fix both at 0. Session 0 is not a convention but a requirement: the environment reaches $\mathcal{G}\mathfrak{m}$ by claiming its own Z-identity, which line 2 admits because $\mathbf{Z} \subseteq \mathcal{G}\mathfrak{m}.\mathbf{N}$, and line 3 then asks $\mathcal{G}\mathfrak{m}.s = \mathcal{Z}.s = 0$ — $\mathcal{G}\mathfrak{m}$ not being global. Remark B.17 is that observation in general. By Definition 1.3 a local $\mathcal{F}$ shares the caller's session, so the whole experiment sits in session 0, which is where the machines the execution supplies live too.

The served parties. A core claims only its own identity (line 5) and GUARD asks $\mathtt{id}'.\mathsf{P} = \mathtt{id}.\mathsf{P}$, so $\mathcal{G}\mathfrak{m}$ at party P reaches $\mathcal{F}$ at party P and at no other: the game exercises $\mathcal{F}$ one party at a time, and the environment chooses the party by choosing which identity of $\mathcal{G}\mathfrak{m}$ to address, its root being free to act for any by line 3. Hence $\mathcal{F}.\mathbf{P}$. The root is added for

Property functionality $\mathcal{G}\mathfrak{m}$ for $\mathcal{F}$
pid $:= (\mathcal{G}\mathfrak{m}.\mathsf{F}, 0, 0)$, $\mathbf{P} := \mathcal{F}.\mathbf{P} \cup \{\mathsf{Z}\}$, $\mathbf{N} := \mathbf{Z}$, $\mathbf{U} := \{(\mathcal{F}, \text{SERVES}_{\mathcal{G}\mathfrak{m}})\}$, *par* $:= \perp$

INITIALIZE():

1: DONE $\leftarrow$ FALSE
2: ... the game's own variables

id.OP(in) from id′

3: **require** ¬DONE *// closed once finalized*
4: ... the game's own code, which may call $\mathcal{F}$ at party id.P
5: **return** out

id.FINALIZE() from id′

6: **require** ¬DONE *// once only*
7: **require** id.P $= \mathsf{Z}$ *// at the root, which stays honest*
8: DONE $\leftarrow$ TRUE
9: $w \leftarrow$ the game's verdict, read off its own state
10: **return** w

id.LEAK() from id′

11: **return** $\perp$ *// a game keeps no party's secrets*

FINALIZE, and the next paragraph says why.

The admitted callers. **Z** and nothing else. Not **S**, which would make $\mathcal{G}\mathfrak{m}$ global and let any standard functionality finalize the game; not **A**, which would hand it to the adversary. With **Z** alone, line 2 refuses every caller but the environment.

The name. Arbitrary, and Corollary B.16 says so: renaming the game and $\mathcal{F}$.**N** together changes no probability, so a property proved against a game of one name holds against it under every other. This matters because line 5 lets the game claim only its own identity, so $\mathcal{F}$.**N** has to name the game while the $\mathcal{F}$ that ships inside a protocol names that protocol; the corollary is what makes the two the same property. What is not arbitrary is the caller's *code*, which no renaming touches.

The dependence. $\text{SERVES}_{\mathcal{G}\mathfrak{m}}$ exempts the root because the root is the game's own bookkeeping identity: $\mathcal{F}$ need not serve a party the game never calls it at. A game whose verdict must *query* $\mathcal{F}$ — verifying a claimed forgery, say — has two options, and they are the usual ones: record during the oracle calls whatever the verdict needs, or ask $\mathcal{F}$ to serve the root as well, when $\text{SERVES}_{\mathcal{G}\mathfrak{m}}$ becomes SERVES.

Tightness. The fields above do not suffice on their own, and the gap is Remark A.5's. An admissible environment is silenced on $\langle\gamma\rangle$, so it may *claim* no identity of a name γ uses — but Definition 10.6 forbids it only from *hosting* at identities γ occupies, and a hosted functionality is a full interface claiming its own identity through its own silencer (Definition 10.8). So an environment may host a machine at an unoccupied process id of a used name: at $(\mathcal{G}\mathfrak{m}.\mathsf{F}, 0, 1)$, say, an invented second instance of the game's own name. A member of σ that admits its caller by *name* — the local pattern of Section 1 — admits that machine, and its calls pass every line of GUARD: the name lies in **N**, the parties agree, the sessions agree, and MEDIATE does not fire at an honest party. The environment would then

hold an oracle on $\mathcal{F}$ that the game records nothing of, and $\Pr[\textsc{Win}] = 1$ for any game whose verdict is that the adversary produced something it was not given.

Tightness at $\{\mathcal{G}\mathfrak{m}\}$ refuses that, and refuses it completely. Every member but the game admits whole process ids the system occupies, so an invented instance fails line 2; and Remark A.5's party condition makes every identity a pinned caller could speak from occupied, where Definition 10.6 forbids hosting. The one route tightness leaves open — to a member of I itself, which here is the game — is closed by $\mathcal{G}\mathfrak{m}.\mathbf{N} = \mathbf{Z}$: a hosted functionality is standard and claims a standard process id, which $\mathbf{Z}$ excludes, while the environment's own Z-identities are not hosted but its own. The game is thus the one member that may admit a bare class of process ids, and it is safe in doing so exactly because Z is no standard name.

Remark 12.2 (Why FINALIZE is served at the root)**.** Party ids in $\{0,1\}^*$ can be corrupted, and line 4 of the full interface hands a corrupt party's call to the adversary *before* the core runs. An FINALIZE served at such a party would therefore be answered by the adversary whenever that party is corrupt, and the answer to "have I won?" would be whatever the adversary says — Remark 3.1's problem exactly, and no check inside the core can repair it, mediation happening in the wrapper above. The root is the fix, and the framework already supplies it: FULLCORRUPT asks $\mathrm{id}.\mathsf{P} \in \{0,1\}^*$, so Z never enters $\mathbf{C}$, so MEDIATE at $(\mathcal{G}\mathfrak{m}.\mathit{pid}, \mathsf{Z})$ can never fire and the bit line 9 computes is always the game's own. The reserved party ids were introduced to keep the three supplied machines honest; they keep the game honest at no extra cost. Line 7 then confines FINALIZE to that identity, so no other is a place to win.

Remark 12.3 (Once, and last)**.** That FINALIZE is called once and is the last call the game answers is enforced rather than assumed. Line 8 sets a flag the wrapper cannot bypass, line 6 refuses a second FINALIZE, and line 3 refuses every other interface thereafter; a refused **require** returns REJ by Section 8, so the environment learns that the game is closed and nothing else — at an honest party, at least. At a corrupt one line 4 intercepts before the core is entered, so the caller is answered by the adversary rather than refused; that changes nothing, the slot being unable to write the game's state and WIN being fixed already. Nothing stops the environment calling a global functionality afterwards either, which is right — the game is over, and what it does then cannot change WIN. The order of the lines matters for the reason $\mathcal{F}_{\mathsf{Sig}}$ re-tests its tables: line 9 may place calls, and Section 8 lets a suspended instance be re-entered, so the flag is set before the verdict is computed rather than after. Were it the other way about, two nested FINALIZE calls could each find the game open.

Remark 12.4 (What corruption buys, and what it costs)**.** At a party $\mathsf{P} \in \{0,1\}^*$ the game's own interfaces are wrapped like any other functionality's, so once $\mathsf{P} \in \mathbf{C}$ line 4 intercepts a call to $\mathcal{G}\mathfrak{m}$ at P: the core does not run, the game records nothing, and the caller is answered by the adversary. That gains an environment nothing — MEDIATE replaces an answer rather than running a core, so the game's state is never written by the adversary — and it costs it the oracle at that party. Since corruption is indexed by party alone (Section 3), corrupting a party of $\mathcal{F}$ closes the game's oracle at that party too. A game that wants an oracle at a corrupt party must therefore serve it somewhere the corruption does not reach: at the root, or at a party of its own outside $\mathcal{F}.\mathbf{P}$, which it may do whenever that oracle needs no call on $\mathcal{F}$.

What keeps the adversary out of the game's cores is not either of the two lines one would reach for. Line 3 *admits* $\mathcal{A}$ at a corrupt party, refusing it only at honest ones; and MEDIATE tests $\mathrm{id}'.\mathsf{F} \neq \mathsf{A}$, so it does not intercept a call the adversary itself places. A call from the slot at a corrupt party would therefore run $\mathcal{G}\mathfrak{m}$'s core, exactly as Section 7 says of a corrupt party addressed by $\mathcal{A}$. What refuses it is $\mathcal{G}\mathfrak{m}.\mathbf{N} = \mathbf{Z}$ at line 2, and nothing else does. The narrowness of that field is not tidiness but the only thing between the adversary and the game's own code.

And $\mathcal{G}\mathfrak{m}$'s LEAK returns $\perp$ because leakage does not go through GUARD (Section 4): the adversary can read it at a corrupt party whatever $\mathcal{G}\mathfrak{m}.\mathbf{N}$ says, so a game must keep nothing there it is not willing to hand over.

Remark 12.5 (Several instances of $\mathcal{F}$)**.** A game may address as many instances of $\mathcal{F}$ as it likes, and the two ways it can differ. Instances in the game's own session need nothing extra, line 3 comparing sessions and ignoring the instance number, so $(\mathcal{F}.\mathsf{F}, 0, i)$ is reachable for every i. Instances in other sessions need $\textsc{global}(\mathcal{F})$, which is what waives that line. Either way the claim is $\mathcal{G}\mathfrak{m}$'s own identity, line 5 silencing its cores on everything else, so every instance sees one and the same caller and access control is decided once. And the instances really are copies: by Appendix B they run the same code and their fields differ in the process id alone, while Section 8 gives each its own state and its own coins. So a multi-instance property — n keys, n sessions — is written by having the game count them, with no extra convention about what a second instance means.

What counting them does not do is make transfer cheaper. Proposition 12.10 replaces one subsystem, so a game over n instances every one of which is to be replaced costs n applications and $n\varepsilon$, by Theorem C.3 along a chain of hybrids — exactly as Remark B.18 records on the relocation side. One application transfers the property for the instances φ contains and says nothing of the others.

Remark 12.6 (A game is not session-uniform, and need not be)**.** $\mathcal{G}\mathfrak{m}.\mathbf{N} = \mathbf{Z}$ meets $\mathbf{Z}$ while $\mathcal{G}\mathfrak{m}$ is not global, so Definition B.2 fails of $\mathcal{G}\mathfrak{m}$ and the results of Appendix B do not apply to a game system. That is as it should be. A game is pinned to session 0 by the paragraph above, there is one of it per experiment, and relocating it is not something anyone wants; what the appendix applies to is the $\mathcal{F}$ under test, whose instances Remark 12.5 counts.

Which environments may play is settled by the class Section 10 already defines, taken at a single system.

Definition 12.7 (Property environments)**.** The environments *admissible for* a game system γ are $\mathbb{Z}_{\gamma,\gamma}$ of Definition 10.6: those with $\mathcal{Z}.\mathit{par} \supseteq \langle\gamma\rangle$ that host at no identity of $\mathrm{IDs}(\gamma)$. Write $\mathbb{Z}^{\circ}_{\gamma}$ for those among them that silence no Z-identity, the ones the absorption of Proposition 12.9 can move.

The two clauses say what a game needs them to say. Silenced on the name closure, an environment may claim no identity of a name γ uses, so it cannot speak as $\mathcal{F}$, as one of $\mathcal{F}$'s subroutines, or as a further instance of any of them; what it may claim is its own Z-identities, which reach $\mathcal{G}\mathfrak{m}$, and fresh names, which reach only what admits all of $\mathbf{S}$. So the environment plays the game through $\mathcal{G}\mathfrak{m}$'s interfaces and touches $\mathcal{F}$ directly only when $\mathcal{F}$ is global — and then it touches it as anyone may, which is what being global means and

what Definition 10.6 already grants in the emulation setting. A property of a global $\mathcal{F}$ is therefore a property against an adversary that shares it, and the game must be written knowing that. The hosting clause is the occupancy hygiene of Definition 8.1, and it also stops the environment from installing a second game.

Definition 12.8 (Win, property advantage)**.** Let $\gamma = \{\mathcal{G}\mathfrak{m}\} \cup \sigma$ be a game system, let $\mathcal{Z}$ be admissible for γ (Definition 12.7), and let $\mathcal{X}$ occupy the adversary slot. Write WIN for the event that the interface $(\mathcal{G}\mathfrak{m}.\mathit{pid}, \mathsf{Z})$.FULLFINALIZE returns 1 in $\textsc{Exec}(\gamma, \mathcal{Z}, \mathcal{X}, \mathcal{C})$. The *advantage* of $\mathcal{Z}$ in the property $\mathcal{G}\mathfrak{m}$ of σ is

$$\textsc{Adv}^{\mathcal{G}\mathfrak{m}}_{\sigma}(\mathcal{Z}, \mathcal{X}) \;:=\; 2\Pr[\,\textsc{Win}\,] - 1, \qquad \textsc{Adv}^{\mathcal{G}\mathfrak{m}}_{\sigma}(\mathcal{Z}) \;:=\; \textsc{Adv}^{\mathcal{G}\mathfrak{m}}_{\sigma}(\mathcal{Z}, \mathcal{D}),$$

the probability being over the coins of every machine in the execution. For $\varepsilon \geqslant 0$, a class $\mathbb{Z} \subseteq \mathbb{Z}_{\gamma,\gamma}$ of environments and a class $\mathbb{X}$ of slot occupants, we say σ has the property $\mathcal{G}\mathfrak{m}$ *within* ε *against* $(\mathbb{Z}, \mathbb{X})$ if $\textsc{Adv}^{\mathcal{G}\mathfrak{m}}_{\sigma}(\mathcal{Z}, \mathcal{X}) \leqslant \varepsilon$ for every $\mathcal{Z} \in \mathbb{Z}$ and every $\mathcal{X} \in \mathbb{X}$, and *within* ε *against* $\mathbb{Z}$ when $\mathbb{X} = \{\mathcal{D}\}$.

The advantage is signed, where that of Definition 10.1 is not, and the asymmetry is the point: an environment cannot flip WIN as it can flip a guess, WIN being the game's verdict rather than its own, so a game it cannot win scores -1 and there is nothing to take an absolute value of.

WIN is an event of the execution rather than its output, and that is deliberate: an execution's output is whatever the environment returns (Section 10), and a property is not a guess to be reported but a thing that happened. Nothing is lost by the distinction, because Remark 12.2 makes the two agree where it matters. The bit line 9 computes is the bit FINALIZE hands the environment, unmediated, so we may name the environment that reports it: write $\mathcal{Z}^\star$ for $\mathcal{Z}$ with its return at the root replaced by "return 1 if any call I placed on $(\mathcal{G}\mathfrak{m}.\mathit{pid}, \mathsf{Z})$.FULLFINALIZE was answered with 1, and 0 otherwise". Every part of that earns its place. The *identity* is named because FINALIZE addressed at another party fails line 7 where that party is honest and is answered by the *adversary* where it is corrupt, so the value coming back need not be the game's; at the root neither happens, by Remark 12.2. The *operation* is named because $\mathsf{Z} \in \mathcal{G}\mathfrak{m}.\mathbf{P}$, so line 1 serves every interface of $\mathcal{G}\mathfrak{m}$ at the root, the game's own oracles included. The test is *against* 1 rather than a reading of the answer as a bit, because a second FINALIZE comes back REJ by Remark 12.3, and REJ is no bit (Section 8). And it is *any* call rather than the first, because the first one placed may be refused — an environment claiming (Z, P) for $\mathsf{P} \neq \mathsf{Z}$ fails line 2's party test, leaving DONE unset — while a later one is served; nothing is weakened by quantifying, since at most one call is ever answered with 1, the game serving one FINALIZE and refusing the rest. So EXEC returns 1 exactly when WIN occurs, and $\mathcal{Z}^\star$ can compute it, every call on that interface being one it placed itself: $\mathcal{G}\mathfrak{m}.\mathbf{N} = \mathbf{Z}$ admits no other caller, and a functionality the environment hosts claims a standard id, which $\mathbf{Z}$ excludes. Then

$$\Pr\big[\textsc{Exec}(\gamma, \mathcal{Z}^\star, \mathcal{X}, \mathcal{C}) = 1\big] \;=\; \Pr\big[\textsc{Win} \text{ in } \textsc{Exec}(\gamma, \mathcal{Z}, \mathcal{X}, \mathcal{C})\big],$$

the two machines placing the same calls and differing only in the value they halt with. That also puts $\mathcal{Z}^\star$ in every class $\mathcal{Z}$ is in whose membership turns on *par* and hosting alone — $\mathbb{Z}_{\gamma,\gamma}$ among them, and the budgeted classes of Definition 11.1 up to the one further step of reading a bit back. Proposition 12.10 is where all of this is used.

Why the dummy. The default slot occupant is the dummy of Appendix D, and the reason is that a game has one adversary and it is the environment. The dummy relays in both directions and keeps no strategy, so an environment driving it holds everything the slot can do; and where $\mathcal{F}$'s own cores reach the adversary — the $\mathcal{A}(\cdot)$ of Section 6, which is how an ideal functionality leaves a value unpinned — it is then the environment that answers, which is exactly the freedom a game means to give it. Writing $\textsc{Adv}^{\mathcal{G}\mathfrak{m}}_{\sigma}(\mathcal{Z})$ for the dummy's case is thus no loss of generality but a choice of who plays; the two-argument form is kept because Proposition 12.10 needs a simulator in that slot. That the abbreviation costs nothing is not a matter of taste, and the reason is the construction of Appendix D: the slot absorbs into the environment.

Proposition 12.9 (The slot absorbs into the environment)**.** *Let $\gamma = \{\mathcal{G}\mathfrak{m}\} \cup \sigma$ be a game system whose cores place no responsive calls (Definition F.2), let $\mathcal{X}$ be a refusal-oblivious occupant of the adversary slot (Definition D.3), and let $\mathcal{Z} \in \mathbb{Z}^{\circ}_{\gamma}$. Then $\mathcal{Z}^{\mathcal{X}}$ of Definition D.1 again lies in $\mathbb{Z}^{\circ}_{\gamma}$, and*

$$\textsc{Adv}^{\mathcal{G}\mathfrak{m}}_{\sigma}(\mathcal{Z}, \mathcal{X}) \;=\; \textsc{Adv}^{\mathcal{G}\mathfrak{m}}_{\sigma}(\mathcal{Z}^{\mathcal{X}}).$$

So if σ has the property $\mathcal{G}\mathfrak{m}$ within ε' against $\mathbb{Z}^{\circ}_{\gamma}$, it has it within ε' against $(\mathbb{Z}^{\circ}_{\gamma}, \mathbb{X})$ for every class $\mathbb{X}$ of refusal-oblivious occupants. Read concretely, an $\mathbf{z}$-bounded environment and an $\mathbf{x}$-bounded occupant give an absorbed environment that is $\mathbf{z} \oplus \mathbf{x}$-bounded.

Proof. Lemma 10.13 at $\mathsf{H} = \{\mathcal{X}\}$ with $\mathsf{H}_{\textsc{keep}} = \varnothing$ gives $\textsc{Exec}(\gamma, \mathcal{Z}, \mathcal{X}, \mathcal{C}) \equiv \textsc{Exec}(\gamma, \mathcal{Z}^{\mathcal{X}}, \mathcal{D}, \mathcal{C})$, its three hypotheses being the three assumed here, one for one. $\textsc{Win}$ is the event that the interface at $(\mathcal{G}\mathfrak{m}.\mathit{pid}, \mathsf{Z})$ returns 1, and absorption touches the adversary slot alone — $\mathcal{G}\mathfrak{m}$ occupies that identity on both sides — so the event has one probability, and Definition 12.8 gives one number. Admissibility is Appendix D's observation that $\mathcal{Z}^{\mathcal{X}}$ occupies what an environment occupies and carries $\mathcal{Z}$'s *par*, so it is admissible exactly when $\mathcal{Z}$ is; the budget is the $\mathbf{z} \oplus \mathbf{a}$ of Appendix E. The final sentence is the display read at each $\mathcal{X}$ in turn. □

The hypotheses are Theorem D.4's and no more: a game system placing responsive calls is outside this, by Remark F.4, and so is an environment that silences its own Z-identities, the instruction of Definition D.1 leaving under a Z-claim. Neither is a condition on what the occupant *does*, which is the point — a game need not care whether the machine in the slot plays fairly, only that it can be moved.

One scope condition comes with all of this, and it is the same one throughout. The proposition above rests on the regrouping of Theorem D.4, which holds only for systems whose cores place no responsive calls, Remark F.4 showing where the relay fails otherwise; the same condition governs Theorem D.4 itself, which is what supplies the emulation hypothesis Proposition 12.10 wants from a wider adversary class, and Corollary 12.11 inherits it on both sides at once. A game system whose $\mathcal{F}$ reaches the slot through $\mathcal{A}^{!}$ therefore sits outside all three and must have its dummy-only hypothesis established directly. What a responsive theory of games would look like we leave where Remark F.4 leaves it.

The calibration. The factor $2\Pr[\textsc{Win}] - 1$ reads the game as a *decision*: an environment that guesses wins with probability $\frac{1}{2}$ and scores 0, and one that always wins scores 1. A *search* property — unforgeability, say, where winning is meant to be hard outright — is read off the same quantity as $\Pr[\textsc{Win}] = (1 + \textsc{Adv}^{\mathcal{G}\mathfrak{m}}_{\sigma})/2$, and a bound ε on the advantage is a

bound $(1+\varepsilon)/2$ on the winning probability, which for small ε is the wrong scale. Games of the second kind are better stated with $\Pr[\textsc{Win}]$ itself; we keep one definition and one convention, and say which reading is meant where a game is given.

Finally, the reason to define properties this way rather than beside the framework: emulation carries them.

Proposition 12.10 (Properties transfer along emulation). *Let $\gamma = \{\mathcal{GM}\} \cup \sigma$ be a game system, let $\varphi \subseteq \sigma$, let the replacement of φ by π in γ be well-formed in the sense of Definition 9.2, and let $\gamma[\pi/\varphi]$ be a game system as well. Suppose*

$$\pi \text{ UC-emulates } \varphi \text{ within } \varepsilon \text{ against } (\{\mathcal{D}\}, \mathbf{S}, \mathbf{Z}).$$

Then there is $\mathcal{S} \in \mathbf{S}$ such that for every $\mathcal{Z} \in \mathbf{Z}_{\gamma[\pi/\varphi],\gamma}$ whose reporting form has $\mathcal{Z}^{\star\,\gamma\setminus\varphi} \in \mathbf{Z}$ — a condition Proposition 10.11 makes vacuous whenever $\mathbf{Z}$ is the largest admissible class, though not when it is the largest class at a budget*, absorption costing $\mathbf{c}_{\gamma\setminus\varphi}$ and the reporting form one step more —*

$$\textsc{Adv}^{\mathcal{GM}}_{\sigma[\pi/\varphi]}(\mathcal{Z}) \;\leqslant\; \textsc{Adv}^{\mathcal{GM}}_{\sigma}(\mathcal{Z},\mathcal{S}) \;+\; 2\varepsilon,$$

and, writing $\textsc{Win}$ and $\textsc{Win}'$ for the winning events of the two executions the display above compares — the real one and the ideal one —

$$\Pr[\,\textsc{Win}\,] \;\leqslant\; \Pr[\,\textsc{Win}'\,] \;+\; \varepsilon.$$

Proof. First, the two advantages are defined and are about the systems the statement names. Since $\mathcal{GM} \notin \sigma$ we have $\mathcal{GM} \notin \varphi$, so $\gamma[\pi/\varphi] = \{\mathcal{GM}\} \cup \sigma[\pi/\varphi]$; and $\mathbf{Z}_{\gamma[\pi/\varphi],\gamma}$ lies in both $\mathbf{Z}_{\gamma,\gamma}$ and $\mathbf{Z}_{\gamma[\pi/\varphi],\gamma[\pi/\varphi]}$, each clause of Definition 10.6 weakening when a union of two closures shrinks to one. That the left-hand side is a property advantage at all is where the hypothesis on $\gamma[\pi/\varphi]$ is spent, and it is not free: emulation supplies neither $\mathcal{GM}.\mathbf{P} = \mathcal{F}'.\mathbf{P} \cup \{\mathcal{Z}\}$ at the new occupant $\mathcal{F}'$ nor $\mathcal{GM}.\mathit{pid} \in \mathcal{F}'.\mathbf{N}$, the second being hypothesis (ii) of Theorem 9.4 and no part of Theorem 10.15. Nor is that all it buys: tightness is part of Definition 12.1, so the hypothesis asks in addition that π's members pin their callers by process id where φ's did. Emulation supplies that least of all, comparing behaviour at the boundary and saying nothing of the declared fields behind it.

Let $\mathcal{S} \in \mathbf{S}$ be the simulator the hypothesis supplies for $\mathcal{D}$; it depends on $\mathcal{D}$ alone, so one $\mathcal{S}$ serves every $\mathcal{Z}$. Fix such a $\mathcal{Z}$ and take its reporting form $\mathcal{Z}^\star$ from the paragraph after Definition 12.8, which gives

$$\Pr\big[\textsc{Exec}(\delta, \mathcal{Z}^\star, \mathcal{X}, \mathcal{C}) = 1\big] \;=\; \Pr\big[\textsc{Win} \text{ in } \textsc{Exec}(\delta, \mathcal{Z}, \mathcal{X}, \mathcal{C})\big]$$

for either game system δ and either occupant $\mathcal{X}$; the hypothesis places $\mathcal{Z}^\star$ in the class $\mathbf{Z}'$ that Theorem 10.15 attaches to this replacement, and $\mathcal{Z}^\star \in \mathbf{Z}_{\gamma[\pi/\varphi],\gamma}$ because $\mathcal{Z}$ is and the two agree on *par* and on what they host. Theorem 10.15 applied to the replacement of φ by π in γ, at the adversary $\mathcal{D}$ and the environment $\mathcal{Z}^\star$, bounds the difference of the two left-hand probabilities — with $\delta = \gamma[\pi/\varphi]$ and $\mathcal{X} = \mathcal{D}$ against $\delta = \gamma$ and $\mathcal{X} = \mathcal{S}$ — by ε. Hence $\Pr[\textsc{Win}]$ differs by at most ε between the two, which is the second display; the advantages of Definition 12.8, being $2\Pr[\textsc{Win}] - 1$, then differ by at most 2ε, which is the first. The order matters for use rather than for proof: the probability bound is what the argument gives, and the advantage bound is it doubled. □

Two things about that proof. The reduction is the textbook one, and absorption is what makes it so: the absorbed environment $\mathcal{Z}^{\star\,\gamma\setminus\varphi}$ of Definition 10.10 hosts the game itself along with the rest of $\gamma \setminus \varphi$, so "an environment playing the game against π" becomes "an environment distinguishing π from φ" by moving the game inside it, which is what Theorem 10.15 was proved about. And the property on the right is stated against $\mathcal{S}$ in the slot rather than against the dummy, which is why Definition 12.8 keeps the occupant as an argument — but it is not an obligation anyone need meet in that form, because the slot absorbs into the environment as well.

Corollary 12.11 (Transfer, against every adversary)**.** *In the setting of Proposition 12.10, suppose in addition that the cores of γ and of $\gamma[\pi/\varphi]$ place no responsive calls (Definition F.2), that the simulator $\mathcal{S}$ that proposition supplies and the occupant $\mathcal{A}$ below are refusal-oblivious (Definition D.3), and that the environments named lie in $\mathbb{Z}^{\circ}_{\gamma}$. If σ has the property $\mathfrak{Gm}$ within ε' against $\mathbb{Z}_{\gamma,\gamma}$ in the sense of Definition 12.8, then*

$$\mathrm{ADV}^{\mathfrak{Gm}}_{\sigma[\pi/\varphi]}(\mathcal{Z},\mathcal{A}) \;\leqslant\; \varepsilon' + 2\varepsilon$$

for every occupant $\mathcal{A}$ of the adversary slot and every $\mathcal{Z}$ the proposition admits with $\mathcal{Z}^{\mathcal{A}}$ among them. If in place of the hypothesis on advantages the ideal side satisfies $\Pr[\mathrm{WIN}'] \leqslant p$ under the same quantification, then $\Pr[\mathrm{WIN}] \leqslant p + \varepsilon$, with WIN and WIN′ naming the real and ideal events as in Proposition 12.10.

Proof. Three steps, of which the outer two are absorption. By Proposition 12.9 at the real game system, $\mathrm{ADV}^{\mathfrak{Gm}}_{\sigma[\pi/\varphi]}(\mathcal{Z},\mathcal{A}) = \mathrm{ADV}^{\mathfrak{Gm}}_{\sigma[\pi/\varphi]}(\mathcal{Z}^{\mathcal{A}})$, the right-hand side being the dummy's case. Proposition 12.10 at $\mathcal{Z}^{\mathcal{A}}$ bounds it by $\mathrm{ADV}^{\mathfrak{Gm}}_{\sigma}(\mathcal{Z}^{\mathcal{A}},\mathcal{S}) + 2\varepsilon$. And Proposition 12.9 again, now at the ideal game system, rewrites the first term as $\mathrm{ADV}^{\mathfrak{Gm}}_{\sigma}\big((\mathcal{Z}^{\mathcal{A}})^{\mathcal{S}}\big)$, which the hypothesis on σ bounds by ε', the doubly absorbed machine being an admissible environment by two applications of the same proposition. The probability form is the same three steps read on the second display of Proposition 12.10 rather than the first, absorption changing no probability at all. □

Two things this buys. The hypothesis on the ideal side is now the property against the *dummy* alone — one statement about σ, quantified over environments and nothing else — where Proposition 12.10 on its own asked for the property against the particular simulator the emulation supplied. Nothing need be assumed about how that simulator corrupts or leaks — only that it does not branch on a framework-internal refusal, which Appendix D asks of every machine it moves: whatever else it does, absorbing it into the environment turns it into part of a machine the ideal-side hypothesis already covers. And the conclusion is the property of the real system against *every* adversary, not only against the dummy, by the same move applied on the other side. Read concretely, the two absorptions are the whole price: for an **a**-bounded $\mathcal{A}$ and an **s**-bounded $\mathcal{S}$ the innermost statement is reached at environment budget $\mathbf{z} \oplus \mathbf{a} \oplus \mathbf{s}$, one hosting cost per machine moved, and Appendix E charges nothing else.

Remark 12.12 (What the occupant of the slot does on the ideal side)**.** Nothing here is needed for the proof above, which absorbs whatever the slot does into the ε Theorem 10.15 supplies. What is at stake is instead the cost of the *hypothesis* on the right: the two executions

the proposition compares differ in the slot as well as in the system. Corruption divides in two, and only half of it is at issue. The half that is not is what the environment does for itself: line 3 admits $\mathrm{id}'.\mathsf{F} \in \{\mathsf{A}, \mathsf{Z}\}$, so an environment needs nobody's cooperation for the corruptions it wants, and claiming (Z, P) from its root it makes them identically on both sides. The half that is at issue is the occupant's own initiative: $\mathcal{S}$ may corrupt parties nobody asked it to, or decline what it is asked through the slot, where $\mathcal{D}$ would relay. That is why the right-hand side of Proposition 12.10 is the property against $\mathcal{S}$ and not against the dummy, and why Definition 12.8 keeps the occupant as an argument. What an ideal σ has to satisfy *in that form* is the game against every simulator the emulation may produce; Corollary 12.11 reduces it to the dummy again, and does so by moving the simulator into the environment rather than by asking anything of it.

Nothing escapes the ε, and the register is why. The environment reads $\mathbf{C}$ through FULLSTATUS, which mediates nothing and returns the whole corrupted set (Remark 3.1), so it may read $\mathbf{C}$ at any point and compare it against the parties it corrupted itself together with whatever it instructed the slot to add. Let $\mathcal{Z}$ be an environment the class admits, and let $\mathcal{Z}'$ be $\mathcal{Z}$ with that comparison added and a return of 0 where it fails. On the real side it never fails, $\mathcal{D}$ corrupting what it is told and nothing besides; on the ideal side it fails with whatever probability δ the simulator strays with, in either direction — declining what it is asked, or corrupting unbidden. So $\mathcal{Z}'$ distinguishes with advantage δ, and the emulation hypothesis gives $\delta \leqslant \varepsilon$. One may therefore reason as though $\mathcal{S}$ echoes the corruptions it is instructed to make, at a price already paid — which is Section 3's commensurateness, read for properties. It is worth having, but it is not what discharges the hypothesis on the right of Proposition 12.10; absorbing the slot does, and it needs nothing at all of the simulator's conduct.

Remark 12.13 (Leakage on the ideal side)**.** Leakage divides between the sides as corruption does, and needs one check of its own because it does not go through GUARD. Only the slot can read a LEAK: line 2 asks $\mathrm{id}'.\mathsf{F} = \mathsf{A}$, and the environment cannot claim such an identity. So $\mathcal{G}\mathfrak{m}.\mathbf{N}$ does not protect the game there, and something else must: line 4's gate together with the party, since the game's leakage is readable only at a corrupt party, the root is never corrupt, and at every other party $\mathcal{G}\mathfrak{m}$'s LEAK returns $\perp$ by Remark 12.4. The game's state therefore reaches the adversary by that route on neither side. What does differ between the sides is φ's leakage against π's, which $\mathcal{S}$ must produce; that is an ordinary part of emulation, bounded by the same ε, and a property inherits the bound with nothing further to check.

References

[1] C. Badertscher, U. Maurer, D. Tschudi, and V. Zikas. Bitcoin as a transaction ledger: A composable treatment. In *Advances in Cryptology — CRYPTO 2017*, volume 10401 of *LNCS*, pages 324–356. Springer, 2017.

[2] R. Canetti. Universally composable security. *Journal of the ACM*, 67(5):28:1–28:94, 2020.

[3] J. Katz, U. Maurer, B. Tackmann, and V. Zikas. Universally composable synchronous computation. In *Theory of Cryptography — TCC 2013*, volume 7785 of *LNCS*, pages 477–498. Springer, 2013.

A Blockers

Definition 10.6 of Section 10 silences an environment on one and the same name closure in both worlds, so the claims open to it do not depend on which world it faces — and Proposition 10.11 and Theorem 10.15 use no relation between the two identity sets at all. What blocking buys is narrower: it makes that closure attainable as an exact silenced set, and so delivers the least restricted admissible class, as the discussion of Definition 10.6 in Section 10 records. A *blocker* occupies identities and nothing else. It admits no caller, $\mathbf{N} := \varnothing$, so the conjunct on line 2 of GUARD is false for every call and the caller receives REJ. Its BLOCK core is therefore never entered and its body is a formality. Leakage does not go through GUARD, so a corrupt party's LEAK core is reachable; it returns $\perp$, all a machine with no state has to give. Both are cores, wrapped as Definition 1.2 requires of any standard functionality, and it is the wrapper that makes the first unreachable.

Blocker $\mathcal{B}[\mathcal{F}]$
pid $:= \mathcal{F}.pid$, $\mathbf{P} := \mathcal{F}.\mathbf{P}$, $\mathbf{N} := \varnothing$, $\mathbf{U} := \varnothing$, *par* $:= \perp$

id.BLOCK() from id′
1: **return** $\perp$ // *unreachable due to* GUARD

id.LEAK() from id′
2: **return** $\perp$ // *reachable, but there is no state*

The blocker $\mathcal{B}[\mathcal{F}]$ is indexed by the functionality it stands in for, from which it copies the two fields that fix an identity set and nothing else. By construction $\text{IDS}(\mathcal{B}[\mathcal{F}]) = \text{IDS}(\mathcal{F})$: it occupies precisely the identities of $\mathcal{F}$, session and instance included, and nothing besides. Being unreachable, it stands in for $\mathcal{F}$ as an occupant of those identities and in no other respect.

Write $I_\pi := \{\mathcal{F}.pid : \mathcal{F} \in \pi\}$ for the process ids a system uses. By Definition 1.2 each $q \in I_\pi$ is carried by exactly one member, which we write π_q, so $\pi = \{\pi_q : q \in I_\pi\}$ and $\text{IDS}(\pi) = \bigcup_{q \in I_\pi} \text{IDS}(\pi_q)$. Blockers are indexed by *ids* and not by members, and the distinction matters: a process id may occur in both worlds carrying different code, in which case φ_q is not a member of π although π uses q, and a blocker for such a q would put two machines at the same id.

Indexing by $I_\varphi \setminus I_\pi$ instead keeps a blocker clear of what is already there. No member of π carries such a q, hence no identity of $\text{IDS}(\pi)$ has q in its first component and $\text{IDS}(\varphi_q) \cap \text{IDS}(\pi) = \varnothing$. A blocker built from φ_q thus lands neither on an identity π already serves nor on the id of an existing member. Distinct ids give pairwise disjoint blockers, and all of them are new to π, so the enlarged system again has distinct ids throughout.

The pair of systems being compared must in addition agree wherever they overlap.

Definition A.1 (Compatibility). Systems π and φ are *compatible* if, whenever a process id occurs in both, the two functionalities sitting at it agree on their served parties and on whom they admit; the remaining fields — what they use, their parameters, and the code they run — may differ.

Compatibility is used at exactly one point: the ids common to both worlds. No blocker is added for such an id, so whatever it contributes must already agree on the two sides.

Without it a shared id could serve different parties in π and in φ, and the difference would go uncovered. Requiring **N** to agree as well is more than the argument needs, since only **P** enters an identity set once the process id is fixed; but it costs nothing, a shared id being carried by the same functionality on both sides in every case we use.

Definition A.2 (Blocker system, blocked systems)**.** For systems π and φ, the *blocker system* $\textsc{Blockers}(\varphi \setminus \pi)$ is

$$\textsc{Blockers}(\varphi \setminus \pi) \;:=\; \{\, \mathcal{B}[\varphi_q] \;:\; q \in I_\varphi \setminus I_\pi \,\},$$

and the *blocked systems* of the pair are

$$\overline{\pi} \;:=\; \pi \cup \textsc{Blockers}(\varphi \setminus \pi), \qquad \overline{\varphi} \;:=\; \varphi \cup \textsc{Blockers}(\pi \setminus \varphi).$$

The argument of $\textsc{Blockers}$ is notation rather than an input: the construction reads the ids $I_\varphi \setminus I_\pi$ and φ's occupants there, not the member set the difference denotes — at a shared id carrying different code, $\varphi_q \in \varphi \setminus \pi$ and yet no blocker is built. Each id in $I_\varphi \setminus I_\pi$ is thus covered by the blocker of the functionality living there, with that functionality's party set. Each world gains exactly the process ids the other has and it lacks, all blocking has to achieve. Ids common to both worlds get no blocker, which is where Definition A.1 does its work.

Informally, Theorem A.3 says blocking equalises the two worlds: each blocked system is again a system, and for a compatible pair the two occupy one and the same identity set, $\textsc{Ids}(\pi) \cup \textsc{Ids}(\varphi)$ — each world having gained exactly the process ids the other has and it lacks.

Theorem A.3 (Equalisation)**.** *Let π and φ be systems. Then the blocked systems $\overline{\pi}$ and $\overline{\varphi}$ of Definition A.2 are systems. If π and φ are moreover compatible, then*

$$\textsc{Ids}(\overline{\pi}) \;=\; \textsc{Ids}(\overline{\varphi}) \;=\; \textsc{Ids}(\pi) \cup \textsc{Ids}(\varphi).$$

Proof. For the first claim, being a system asks three things of $\overline{\pi}$: that it be finite, that its members be standard, and that they carry distinct process ids. Finiteness is immediate, the added members being indexed by $I_\varphi \setminus I_\pi \subseteq I_\varphi$. Standardness is inherited: a blocker carries the wrappers Definition 1.2 asks of a standard functionality, $\textsc{Block}$ as in Section 7 and $\textsc{Leak}$ as in Section 4, and its process id is copied from a member of φ, so its name is none of A, Z, C. For distinctness, the members of $\textsc{Blockers}(\varphi \setminus \pi)$ are the $\mathcal{B}[\varphi_q]$ with $q \in I_\varphi \setminus I_\pi$, and $\mathcal{B}[\varphi_q].\mathit{pid} = \varphi_q.\mathit{pid} = q$. These ids are pairwise distinct and none lies in I_π, so the members of $\overline{\pi}$ carry distinct process ids and $I_{\overline{\pi}} = I_\pi \cup I_\varphi$. The same argument applies to $\overline{\varphi}$.

For the second, $\textsc{Ids}(\mathcal{B}[\mathcal{F}]) = \textsc{Ids}(\mathcal{F})$ holds for every $\mathcal{F}$, since a blocker copies the two fields that determine an identity set. Hence

$$\begin{aligned}
\textsc{Ids}(\overline{\pi}) \;&=\; \bigcup_{q \in I_\pi} \textsc{Ids}(\pi_q) \;\cup\; \bigcup_{q \in I_\varphi \setminus I_\pi} \textsc{Ids}(\varphi_q), \\
\textsc{Ids}(\overline{\varphi}) \;&=\; \bigcup_{q \in I_\varphi} \textsc{Ids}(\varphi_q) \;\cup\; \bigcup_{q \in I_\pi \setminus I_\varphi} \textsc{Ids}(\pi_q),
\end{aligned}$$

and both unions are indexed by $I_\pi \cup I_\varphi$. Compare them at each q in that set. If $q \in I_\pi \setminus I_\varphi$ then both contribute $\mathrm{IDS}(\pi_q)$, and if $q \in I_\varphi \setminus I_\pi$ then both contribute $\mathrm{IDS}(\varphi_q)$. If q lies in both, the first contributes $\mathrm{IDS}(\pi_q)$ and the second $\mathrm{IDS}(\varphi_q)$. Both are carried at the index q, so $\pi_q.\mathit{pid} = \varphi_q.\mathit{pid} = q$ outright, and compatibility supplies the one thing that does not follow from the indexing, $\pi_q.\mathbf{P} = \varphi_q.\mathbf{P}$. An identity set is fixed by those two fields, so $\mathrm{IDS}(\pi_q) = \mathrm{IDS}(\varphi_q)$. The two unions therefore agree term by term, and each equals $\bigcup_{q \in I_\pi} \mathrm{IDS}(\pi_q) \cup \bigcup_{q \in I_\varphi} \mathrm{IDS}(\varphi_q) = \mathrm{IDS}(\pi) \cup \mathrm{IDS}(\varphi)$. □

Blocking enlarges a system, and enlarging a system is one of the two ways Definition 1.3 could come undone.

Proposition A.4 (Blocking preserves well-formedness). *Let π and φ be systems with $\mathrm{WF}(\pi)$ and $\mathrm{WF}(\varphi)$. Then $\mathrm{WF}(\overline{\pi})$ and $\mathrm{WF}(\overline{\varphi})$.*

Proof. $\overline{\pi}$ is a system by the first claim of Theorem A.3, which asks no compatibility, so $\mathrm{WF}(\overline{\pi})$ is a legal question. We argue for $\overline{\pi}$; the argument for $\overline{\varphi}$ is the same with $\mathrm{WF}(\varphi)$ in place of $\mathrm{WF}(\pi)$. Take $\mathcal{F} \in \overline{\pi}^+$ and $(\mathcal{F}', \mathsf{R}) \in \mathcal{F}.\mathbf{U}$.

If $\mathcal{F}$ is one of the added blockers there is nothing to check: a blocker has $\mathbf{U} = \varnothing$ by construction, so it contributes no entry. Otherwise $\mathcal{F}$ lies in π^+, and $\mathrm{WF}(\pi)$ supplies a witness $\mathcal{G} \in \pi^+$ for that entry. That witness is still present, since $\pi^+ \subseteq \overline{\pi}^+$, and blocking alters no member's fields, so $\mathcal{G}.\mathsf{F}$, $\mathcal{G}.\mathsf{s}$ and everything R reads are what they were. The three conditions therefore still hold of the same $\mathcal{G}$.

The argument uses nothing about blockers beyond their empty $\mathbf{U}$: enlarging a system to another system preserves well-formedness whenever the members added carry no entries of their own, because a witness once present is never removed and the fields it is judged on never change. Neither compatibility nor Theorem A.3's second claim is used. □

Remark A.5 (Tight realizations). The closure keeps the environment's own claims off the used names; hosting is the one route around it. An admissible environment may host a machine at an *unoccupied* identity of a used name — an invented second instance of a caller, say — and a hosted machine is a full interface, claiming its own identity through its own silencer rather than through *par* (Definition 10.8). Against a pair whose members admit that name, the two worlds can then answer differently at an identity only one of them occupies, and no condition on the class can refuse the intruder without refusing absorption: the absorbed environment of Definition 10.10 hosts machines of exactly the same shape — outside occupants at admitted names — and differs from the intruder only in the code it runs, which admissibility rightly does not read. Part of the repair belongs to the realization. Call a system *tight at* a subset I of its members if every member outside I admits whole process ids occupied by the system, rather than names, and serves only parties each of its admitted callers serves — with SERVES this makes a private member's party set exactly its callers', which is what a private subroutine is. Tightness closes the host route *to the members outside* I: an unoccupied process id lies outside every pinned $\mathbf{N}$ and fails line 2, and the party condition makes every identity a pinned caller could speak from occupied, where Definition 10.6 forbids hosting; so a call an intruder addresses to a member outside I is refused, on the ideal side too, by the blocker's empty $\mathbf{N}$ or by EXEC finding no occupant. What tightness does *not* close is the route to a member of I itself: an interface member that admits a bare used name — the local pattern of Section 1 — is reachable by a hosted

fake instance exactly as a private member would be, and tightness constrains no member of I. Pinning those too, or admitting only $\mathbf{Z} \cup \mathbf{A}$ where the environment reaches them directly, would close it; we do not pursue a general sufficient condition here. Tightness is a hygiene condition in the spirit of a subroutine-respecting protocol, necessary for the private members not to leak; a full characterization of when comparisons against blocked pairs are faithful we leave open.

B Relocation and Renaming

Section 1 records that a functionality's code hardly reads its own session and never its instance number: s occurs at line 3 of GUARD, in the wrappers that invoke it, and in Definition 1.3, and i occurs nowhere. This appendix turns that observation into a statement. Renaming sessions and instances is an isomorphism of executions, so an emulation proved at one process id holds at every other, within the same ε and at the same budgets. One hygiene condition is needed and no more — that a functionality reachable from outside every session be reachable from every session, which every machine the paper specifies satisfies — and isolating it is half the work. That is what lets a realization be written for a single session, as $\mathcal{F}_{\mathsf{Sig}}$ of Appendix H is, and read as covering the rest.

Definition B.1 (Relocation). A *relocation* is a permutation θ of the process ids such that

(i) $\theta(q).F = q.F$ for every q;

(ii) $q.s = q'.s \iff \theta(q).s = \theta(q').s$ for all q, q' with $q.F, q'.F \in \{0,1\}^*$;

(iii) $\theta(q) = q$ for each of the three ids $q \in \{(A,0,0),(Z,0,0),(C,0,0)\}$ the execution supplies.

Conditions (i) and (ii) say exactly that on the standard names

$$\theta(F,s,i) \;=\; \big(F,\ \hat{\theta}(s),\ \theta_{F,s}(i)\big)$$

for a permutation $\hat{\theta}$ of session ids and, for each name and session, a permutation $\theta_{F,s}$ of instance numbers — we write the two components of a relocation this way rather than with letters of their own, σ and ι being spoken for by Proposition 10.14 and Definition 10.12, whose notation the proofs below import. Names are kept, because GUARD and WF match by name and nothing may become a different functionality; sessions are permuted *as blocks*, because line 3 asks whether two ids share a session and not which; and instances are permuted freely within a block, that component being read nowhere.

Condition (iii) pins the three process ids that occur as *literals* in code: A, in the hook $(A, id.P)$ of MEDIATE; C, in the register read $(C, id.P)$ of lines 2 and 6; and Z, at line 2 and in the relay of line 5. Every identity named outright in the paper is one of these three paired with a party in scope, so pinning the three pins them all — and it pins those three and no more. It must not be read as pinning every id of a reserved name, and the difference is not cosmetic: by Section 1 the set $\mathbf{A}$ holds a triple (A,s,i) for every s and i, only $(A,0,0)$ ever occurring, so a condition fixing all of them and read together with (ii) unrestricted would give $\hat{\theta}(s) = s$ at every s and leave no relocation that moves a session at all. Restricting (ii) to the standard names is the other half of the same care: $\hat{\theta}$ is determined there, and the ids of a reserved name that never occur are left to move harmlessly inside their own class, which Proposition B.5 shows is where they stay.

Nothing here pins session 0, and $\hat{\theta}$ is free to move it. That freedom has a price, and it is worth seeing exactly where, since this is the only point at which the framework tells one session from another. Line 3 compares the *caller's* session while GLOBAL is tested of the callee, and the three machines the execution supplies sit in session 0; so a call from $\mathcal{A}$ or $\mathcal{Z}$ to a callee that is not global passes that line exactly when the callee's own session is 0. A callee of that shape would tell a relocation moving session 0 from one leaving it alone. Nothing else can: for every other callee the supplied machines are either waived by GLOBAL or refused at line 2 before the session is looked at. So one condition on the callee closes the gap, and it is a hygiene condition rather than a restriction on θ.

Definition B.2 (Session-uniform)**.** A functionality $\mathcal{F}$ is *session-uniform* if

$$\mathcal{F}.\mathbf{N} \cap (\mathbf{A} \cup \mathbf{Z}) \neq \varnothing \qquad \Longrightarrow \qquad \text{GLOBAL}(\mathcal{F}),$$

so that it admits a caller from outside every session only if it is reachable from every session. A system is session-uniform if each of its members is, and an environment if each functionality it hosts is.

The sessions a session-uniform functionality can be reached from do not depend on the session it sits in, which is what relocation needs of it. The three supplied machines meet the condition outright, all three being global — $\mathcal{A}$ and $\mathcal{C}$ by the first disjunct of GLOBAL, $\mathcal{Z}$ by the second — and so does every occupant of the adversary slot, global by that same second disjunct, and so does a blocker, whose $\mathbf{N}$ is empty and which line 2 therefore refuses whatever the session. Those are every $\mathbf{N}$ the paper declares; $\mathcal{F}_{\mathsf{Sig}}$ takes its as a parameter, where the condition is one line to check. What the condition excludes is the shape Section 1 leaves open in saying that a local $\mathcal{F}$ may itself declare whether $\mathbf{A}$ and $\mathbf{Z}$ may call it: a local functionality admitting the adversary is reachable from the slot in session 0 and in no other, and that is the one thing a relocation cannot carry. Remark B.17 records the alternative, a one-line widening of line 3 that would dispense with the condition altogether.

Definition B.3 (Action of a relocation)**.** A relocation θ acts on identities by $\theta(\mathit{pid}, \mathsf{P}) := (\theta(\mathit{pid}), \mathsf{P})$, leaving the party component alone, and on identity sets pointwise. It sends a functionality $\mathcal{F}$ to $\theta\mathcal{F}$ with

$$\theta\mathcal{F}.\mathit{pid} := \theta(\mathcal{F}.\mathit{pid}), \quad \theta\mathcal{F}.\mathbf{P} := \mathcal{F}.\mathbf{P}, \quad \theta\mathcal{F}.\mathbf{N} := \theta(\mathcal{F}.\mathbf{N}), \quad \theta\mathcal{F}.\mathbf{U} := \theta(\mathcal{F}.\mathbf{U}), \quad \theta\mathcal{F}.\mathit{par} := \theta(\mathcal{F}.\mathit{par}),$$

where θ acts on a $\mathbf{U}$ entry by $\theta(\mathcal{F}', \mathsf{R}) := (\theta\mathcal{F}', \mathsf{R})$ and on *par* through whatever identities and process ids it contains; the code of $\theta\mathcal{F}$ is the code of $\mathcal{F}$ with every process id occurring in it — in a call target, a claimed identity, a test, or a stored value — replaced by its image; and every identity set a machine *declares* rather than computes is replaced by its image too, which for a bundle (Definition 10.8) means its occupied set, its outward set and any routing it fixes by hand. It acts on a system memberwise, and on any other machine of the execution the same way, a bundle's hosted members included, so that $\theta\mathcal{Z}$ hosts the images of what $\mathcal{Z}$ hosts. We require the requirement predicates to be equivariant, $\mathsf{R}(\mathcal{F}, \mathcal{G}) \iff \mathsf{R}(\theta\mathcal{F}, \theta\mathcal{G})$, as SERVES is, it reading only $\mathbf{P}$ and a membership of $\mathbf{N}$. Every clause above relabels pointwise, so the action is one: ID

acts as the identity and $(\theta'\theta)\mathcal{F} = \theta'(\theta\mathcal{F})$. In particular θ^{-1} undoes θ on machines as it does on process ids, which is what lets the results below argue in both directions.

Relabelling the process ids a program *names* is not by itself relabelling what it does, and the gap between the two is a condition on the machine model. We state it, because Lemma B.9 is false without it.

Definition B.4 (Identity-atomic code)**.** Code is *identity-atomic* if it handles identities and process ids as opaque tokens. It may project one to its name or party component; test two of them, or two of their name, party, session or instance components, for equality; test one for membership in a set of them the code has been given, as line 2 tests $\mathcal{F}.\mathbf{N}$ and SILENCE tests $\mathcal{I}$; form a set of them by those tests, as lines 2 and 3 form $\overline{\mathcal{I}}$ and line 5 forms $\{\mathtt{id}'' \neq \mathtt{id}\}$; name one outright; and destructure a value that carries one. It may not compute with a session or an instance number: no arithmetic, no ordering, and no session or instance written down on its own, apart from inside an identity or process id named outright. Each of these counts as one step, whatever the identities involved, as Definition 11.1 counts a call as one answer.

Every machine in this paper is identity-atomic. GUARD compares sessions for equality and nothing else, no core reads $\mathtt{i}$ at all, and the only process ids named outright are the three of condition (iii). We take the model to be this throughout, and it is what makes Definition B.3 sound: on identity-atomic code the substitution there does not merely rewrite literals but computes the relabelled behaviour, each permitted operation being equivariant — projections because θ leaves names and parties alone, equality and membership because θ is injective and the sets tested and formed are relabelled with the code, a named identity because substitution replaces it by its image. It is also what makes a machine and its image take the same steps: they perform the same operations in the same order, the control flow being the same on both, and the last clause of the definition charges one step for each however the ids inside it have been renamed. So the budgets of Definition 11.1 carry over unchanged — which a cost model charging by the symbol would not grant, a relocation being free to lengthen the encoding of a session.

Without the restriction both fail. A core returning $\mathtt{id.i}$ mod 2 names no process id, so substitution leaves it untouched; carried to another instance number it answers differently, and the relabelled machine is not the original relabelled. A core calling $(\mathtt{F}, \mathtt{id.s}, \mathtt{id.i} + 1)$ misses its target in the same way. Neither is a machine anyone would write — Definition 10.2 permits an *arbitrary* core, which is why the exclusion has to be stated rather than assumed.

Proposition B.5 (Basic invariances)**.** *Let θ be a relocation, $\mathcal{F}$ a functionality and π a system. Then*

(i) $\mathrm{IDs}(\theta\mathcal{F}) = \theta\big(\mathrm{IDs}(\mathcal{F})\big)$*, and occupants carrying pairwise disjoint identity sets keep them;*

(ii) *θ carries each name class onto itself, so $\theta(\mathbf{S}) = \mathbf{S}$, $\theta(\mathbf{A}) = \mathbf{A}$ and $\theta(\mathbf{Z}) = \mathbf{Z}$, and every $\mathbf{N}$ declared by names — a local $\{pid : pid.\mathsf{F} = \mathsf{F}^{\uparrow}\}$ among them — is its own image;*

(iii) $\textsc{global}(\theta\mathcal{F}) \iff \textsc{global}(\mathcal{F})$;
(iv) $\langle\theta\pi\rangle = \theta\big(\langle\pi\rangle\big) = \langle\pi\rangle$;
(v) $\theta\pi$ *is a system, and* $\textsc{wf}(\pi) \iff \textsc{wf}(\theta\pi)$*; and*
(vi) θ^{-1} *is again a relocation.*

Proof. (i) An identity set is fixed by *pid* and **P**, and a relocation relabels the first while leaving the second alone; disjointness survives because θ is injective on identities.

(ii) By (i) of Definition B.1 each name class maps into itself. The classes partition the process ids, so for x in the class of F the point $\theta^{-1}(x)$ lies in some class, which maps into itself and therefore is the class of F; so the map is onto that class as well. The three displayed sets are unions of name classes, and a set of ids specified by a name is a union of them.

(iii) $\textsc{global}(\mathcal{F})$ reads $\mathbf{S} \subseteq \mathcal{F}.\mathbf{N}$ or $\mathcal{F}.F \in \{A, Z, C\}$. The second is untouched by (i) of Definition B.1; for the first, $\mathbf{S} \subseteq \theta(\mathcal{F}.\mathbf{N}) \iff \theta^{-1}(\mathbf{S}) \subseteq \mathcal{F}.\mathbf{N}$, and $\theta^{-1}(\mathbf{S}) = \mathbf{S}$ by item (ii) above, θ being a bijection.

(iv) A name closure is specified by the names its system uses; θ keeps those, so $\theta\pi$ uses the names π does and $\langle\theta\pi\rangle = \langle\pi\rangle$. And $\langle\pi\rangle$ is a union of name classes crossed with party ids, so item (ii) gives $\theta(\langle\pi\rangle) = \langle\pi\rangle$ as well.

(v) The members of $\theta\pi$ carry distinct process ids, θ being injective, and are standard, their names being unchanged and their wrappers untouched by relabelling; the family is finite. For $\textsc{wf}$, take $\mathcal{F} \in \pi^+$ and an entry $(\mathcal{F}', R)$ with witness $\mathcal{G}$. Names match on both sides. For the session conjunct $\mathcal{G}.s = \mathcal{F}.s$, either $\textsc{global}(\mathcal{G})$ and item (iii) above waives the conjunct on both sides, or $\mathcal{G}$ is standard — and then so is $\mathcal{F}$, the entries of the three supplied machines naming only each other and $\mathcal{C}.\mathbf{U}$ being empty — so that both ids are standard and (ii) of Definition B.1 preserves the test. Finally $R(\mathcal{F}, \mathcal{G}) \iff R(\theta\mathcal{F}, \theta\mathcal{G})$ by the equivariance Definition B.3 requires. The entries of the three supplied machines are settled identically on both sides, every field $\textsc{serves}$ reads of them being fixed by items (ii) above and (iii) of Definition B.1. For the converse, run the same argument on θ^{-1}, a relocation by item (vi), using $\theta^{-1}(\theta\pi) = \pi$ from Definition B.3.

(vi) Conditions (i) and (ii) of Definition B.1 are stated as an identity of components and as an equivalence, and so hold of the inverse; (iii) does because θ fixes those three ids. □

Two of these deserve a word. By (ii), only the pinned **N** declarations of Remark A.5 move at all, and they move with the members they pin. And by (iv) name closures are invariant, which is what makes Proposition B.10 come out against an environment class constrained exactly as before rather than one merely corresponding to it.

Definition B.6 (Pid-blind core)**.** A core is *pid-blind* if $\theta\textsc{Op} = \textsc{Op}$ for every relocation θ: it names no process id that a relocation moves. It may compare the sessions of two identities in scope, as GUARD does, and it may name one of the three process ids that (iii) of Definition B.1 fixes — but not some other id of a reserved name, which (iii) leaves free to move.

Every core in this paper is pid-blind. The wrappers of Section 2 read names and parties only; MEDIATE hooks to $(A, \mathtt{id}.P)$ and lines 2 and 6 to $(C, \mathtt{id}.P)$, both reserved; the $\mathcal{Z}$ box computes $\overline{\mathcal{I}}$ from *par* by tests on names and parties; $\mathcal{D}$ places the call its payload names;

and $\mathcal{F}_{\mathsf{Sig}}$ reads $\mathtt{id}.\mathsf{P}$ and its own **P** and nothing else of an identity.

That is a stronger property than Lemma B.9 needs, and a different one. The lemma needs Definition B.4, which every machine of an execution must meet, the environment and the occupant of the adversary slot included — and neither of those can be pid-blind, each having to name the process ids it drives. What pid-blindness buys is the reading of the conclusion: for a system of pid-blind cores, $\theta\pi$ is π with five fields relabelled and not a line of code rewritten, so a relocated protocol is the protocol, run elsewhere.

Before the lemma, the other half of the question the definition answers: which fields an execution reads at all.

Proposition B.7 (Inert fields). *Let $\mathcal{F}$ and $\mathcal{F}'$ be standard functionalities agreeing on pid,* **P**, *par and code.*

(i) If they agree on **N** *as well, so that they differ in* **U** *alone, then no execution distinguishes them: in the two systems got by putting one or the other at that process id, every activation produces the same configuration.*

(ii) If they differ in **N** *alone, then from the same state and on the same call* $\mathtt{id}.\textsc{FullOp}(\mathtt{in})$ *from* $\mathtt{id}'$*, either both wrappers refuse at line 1 with* REJ*, or both admit and enter the same core at the same point in the same state.*

Proof. By inspection of where the two fields are read during an execution. **U** is read only by the static check of Definition 1.3 and the results about it, never by EXEC or by any interface, which gives (i). The readers of **N** inside an execution are two: line 2 of GUARD, and the predicate GLOBAL and hence line 3. Both sit inside $\textsc{Guard}_{\mathcal{F}}$, which line 1 of the full interface **require**s before anything else. (The field is read outside an execution too — by Definition 1.3 through those two, by Definition A.1, and by the typing conditions of Definition 10.3 and Remark A.5 — but those are conditions on a system, not steps of one.) So the field decides that verdict and is read nowhere else in the wrapper: the lines between line 1 and the core — the register read, the corruption gate, the first MEDIATE hook, the silenced set $\{\mathtt{id}'' \neq \mathtt{id}\}$ — are functions of *pid*, **P**, *par*, the code and the call, so two wrappers that both admit reach line 5 together and enter the same core at the same point in the same state. That is (ii). □

So **N** is access control and nothing else, and **U** is inert in every execution, as Remark 10.4 already records for the adversary slot. Inertness past the guard is a statement about one step, and it cannot be strengthened to one about the value returned. The obvious reason is that a call one admits and the other refuses sends the two states apart. The less obvious one is that even a call both admit can end differently, because the core entered at line 5 may re-enter this very wrapper — Section 8 permits an instance to be re-entered, and line 5 lets a core claim its own identity — and the inner call meets $\textsc{Guard}_{\mathcal{F}}$ again, where the field differs. A core that places $\mathtt{id}.\textsc{FullOp}'()$ as $\mathtt{id}$ and returns 1 just if the answer is not REJ separates an $\mathcal{F}$ admitting its own process id from an $\mathcal{F}'$ that does not, on a call both admitted. What survives is the exact claim: the two differ in guard verdicts alone, and a single differing verdict, outer or inner, is enough to separate them.

Remark B.8 (The two fields that are read). Neither **P** nor *par* admits a statement of this kind, and for different reasons. **P** fixes $\textsc{Ids}(\mathcal{F})$, hence which calls EXEC dispatches to $\mathcal{F}$ at all and the second conjunct of line 1; and cores read it directly — INITIALIZE of $\mathcal{F}_{\mathsf{Sig}}$ types

$\mathtt{VK} : \mathcal{F}_{\mathsf{Sig}}.\mathbf{P} \to \mathcal{K} \cup \{\square\}$, and the test $\exists\, \mathsf{P}' \notin \mathbf{C}$ of VERIFY quantifies over that domain. So enlarging or shrinking a party set is not a symmetry. What *is* one is *renaming*: a permutation β of $\{0,1\}^*$ extended to fix A and Z, acting on identities, on $\mathbf{P}$, on $\mathbf{C}$ and on the party-indexed state, gives an isomorphism of executions by the argument of Lemma B.9, provided cores read party ids only through equality and indexing — as $\mathcal{F}_{\mathsf{Sig}}$ does, its $\exists \mathsf{P}' \notin \mathbf{C} : \mathit{vk} = \mathtt{VK}[\mathsf{P}']$ being equivariant. The two places a party id enters otherwise survive it as well: line 3 names the party Z, which β fixes, and FULLCORRUPT tests $\mathtt{id}.\mathsf{P} \in \{0,1\}^*$, a set β permutes. *par* is read by the code by definition of the field, and $\mathcal{Z}.\mathit{par}$ is read by Definition 10.6 besides; there is nothing to be invariant in. A relocation moves it, and moves it to itself whenever it is declared by names, which $\mathcal{Z}.\mathit{par} := \langle\overline{\pi}\rangle$ is.

Informally, Lemma B.9 says that relabelling sessions and instances throughout an execution changes nothing observable: it is Proposition 10.14's induction with θ in place of the absorption bijection, and every check is the equivariance of one line of one wrapper.

Lemma B.9 (Relocation). *Let θ be a relocation, π a session-uniform system, $\mathcal{X}$ an occupant of the adversary slot (Definition 8.1), and $\mathcal{Z}$ a session-uniform environment hosting at no identity of* $\mathrm{IDs}(\pi)$*, so that* EXEC *is defined at the four. Then*

$$\textsc{Exec}\big(\theta\pi,\, \theta\mathcal{Z},\, \theta\mathcal{X},\, \mathcal{C}\big) \qquad \textit{and} \qquad \textsc{Exec}\big(\pi,\, \mathcal{Z},\, \mathcal{X},\, \mathcal{C}\big)$$

are identically distributed.

Proof. The correspondence is that of Definition 10.12 with $\iota := \theta$ — each machine paired with its image, $\mathcal{C}$ with itself — and with (C1), (C3) and (C4) read up to θ: two states, two program points, two chains of suspended callers correspond when one is the θ-image of the other. Since θ is a bijection on identities the relation is symmetric, and as before the proof is an induction on activations. The paragraphs below discharge the step.

Machines. $\theta\pi$ is a system and occupancy is relabelled bijectively, by (i) and (v) of Proposition B.5; since $\mathcal{Z}$ hosts clear of $\mathrm{IDs}(\pi)$ and $\theta\mathcal{Z}$ hosts at the images of what $\mathcal{Z}$ hosts, the occupants are pairwise disjoint on the right exactly where they are on the left, so both executions are defined. Line 1 initialises $\mathcal{C}$ and every member on both sides, and a member's initial state is its image's relabelled, giving (C1) at the outset.

Guards. Take a call $\mathtt{id}.\textsc{FullOp}(\mathtt{in})$ from $\mathtt{id}'$ at $\mathcal{F}$ and its image at $\theta\mathcal{F}$. Line 1: $\mathtt{id}.\mathit{pid} = \mathcal{F}.\mathit{pid}$ iff $\theta(\mathtt{id}.\mathit{pid}) = \theta(\mathcal{F}.\mathit{pid})$ by injectivity, and $\mathtt{id}.\mathsf{P} \in \mathbf{P}$ is untouched. Line 2: $\mathtt{id}'.\mathit{pid} \in \mathcal{F}.\mathbf{N}$ iff $\theta(\mathtt{id}'.\mathit{pid}) \in \theta(\mathcal{F}.\mathbf{N})$, again by injectivity, and the party test reads components θ leaves alone. Line 3 is a disjunction whose second disjunct agrees on the two sides by (iii) of Proposition B.5, so suppose $\textsc{Global}(\mathcal{F})$ fails — it fails on both sides together or on neither — and consider the first. Every machine of the execution is session-uniform: π and $\mathcal{Z}$'s hosted copies by hypothesis, and $\mathcal{X}$, $\mathcal{C}$ and $\mathcal{Z}$'s own interfaces because their process ids are A, C and Z, which GLOBAL admits by its second disjunct. So $\mathcal{F}.\mathbf{N}$ meets neither $\mathbf{A}$ nor $\mathbf{Z}$, and a claim carrying the id of $\mathcal{A}$ or of $\mathcal{Z}$ is refused at line 2 on both sides — (iii) fixing those ids, and $\theta(\mathcal{F}.\mathbf{N})$ missing $\mathbf{A}$ and $\mathbf{Z}$ just as $\mathcal{F}.\mathbf{N}$ does, those two sets being θ-invariant by (ii) of Proposition B.5. The third supplied id cannot arrive at all, $\mathcal{C}$ placing no calls. The claims that reach line 3 therefore carry standard process ids, and so does $\mathtt{id}$, the name of $\mathcal{F}$ lying in $\{0,1\}^*$ once $\textsc{Global}(\mathcal{F})$ fails; so $\mathtt{id}'.\mathtt{s} = \mathtt{id}.\mathtt{s}$ holds of the images exactly when it holds of the originals, by (ii) of Definition B.1. This is the one paragraph session-uniformity is for;

without it a relocation moving session 0 would change the verdict here. So the verdict is the same on both sides, at every callee, including the leakage interface of Section 4, whose three requirements read only what these lines read.

Silencers, mediation, the register. The silenced set of line 5 is $\{\mathtt{id}'' \neq \mathtt{id}\}$, whose image is $\{\mathtt{id}'' \neq \theta\mathtt{id}\}$, so a core claims its own identity on both sides and nothing else. The set $\overline{\jmath}$ of the $\mathcal{Z}$ box is computed from $\theta(\mathit{par})$ by tests on names and parties, so it is the image of the set computed on the left, and admits exactly the image of what that admits. MEDIATE reads $\mathbf{C}$, $\mathtt{id}.\mathrm{P}$ and $\mathtt{id}'.\mathrm{F}$ and hooks to $(\mathrm{A}, \mathtt{id}.\mathrm{P})$; lines 2 and 6 address $(\mathrm{C}, \mathtt{id}.\mathrm{P})$. Every id named outright here is one of the three that (iii) of Definition B.1 fixes. And the register itself needs no image: $\mathbf{C}$ collects *party* ids, which a relocation does not touch, so $\mathcal{C}$ is literally the same machine holding the same set at corresponding configurations. Corruption being indexed by party alone (Section 3) is what buys this.

Routing, claims, token. EXEC dispatches a call to the occupant of its target identity, and targets correspond under θ while occupancy does by the first paragraph; a call addressed to an unoccupied identity is answered REJ on the left exactly when its image is on the right. Claims correspond by the previous paragraph. The code being identity-atomic (Definition B.4), a machine and its image take the same branches and place calls that are θ-images of one another, so they reach the same program point with values related by θ; and each reads the same coins in the same order, which is (C2). The token discipline is unchanged, EXEC having one operation on both sides and the same chain of suspensions arising, which is (C3) and (C4).

Corresponding halting configurations output alike, the output being $\mathcal{Z}$'s and $\theta\mathcal{Z}$'s return at (Z, Z), an identity (iii) of Definition B.1 fixes. So the two executions agree as distributions, in particular on the probability of returning 1. □

Proposition B.10 (Emulation transports)**.** *Let θ be a relocation and π, φ systems; for (ii) and (iii) let the two of them and every environment of the classes named be session-uniform, which (i) does not need. Then*

(i) $\mathbb{Z}_{\theta\pi,\theta\varphi} = \theta\big(\mathbb{Z}_{\pi,\varphi}\big)$*, and θ carries adversaries to adversaries and simulators to simulators, fixing the dummy of Appendix D;*

(ii) if π UC-emulates φ within ε against $(\mathbb{A}, \mathbb{S}, \mathbb{Z})$, then $\theta\pi$ UC-emulates $\theta\varphi$ within ε against $(\theta\mathbb{A}, \theta\mathbb{S}, \theta\mathbb{Z})$; and

(iii) for budgeted systems and every $\mathbf{a}, \mathbf{s}, \mathbf{z}$, $\mathrm{ADV}^{\mathrm{UC}}_{\theta\pi,\theta\varphi}[\mathbf{a}, \mathbf{s}, \mathbf{z}] = \mathrm{ADV}^{\mathrm{UC}}_{\pi,\varphi}[\mathbf{a}, \mathbf{s}, \mathbf{z}]$.

Proof. For (i), Definition 10.6 asks two things of $\mathcal{Z}$. The *par* clause is θ-invariant outright: $\theta\mathcal{Z}.\mathit{par} = \theta(\mathcal{Z}.\mathit{par})$ contains $\theta(\langle\pi\rangle \cup \langle\varphi\rangle) = \langle\theta\pi\rangle \cup \langle\theta\varphi\rangle$ exactly when $\mathcal{Z}.\mathit{par}$ contains $\langle\pi\rangle \cup \langle\varphi\rangle$, name closures being invariant by (iv) of Proposition B.5. The hosting clause transports by injectivity, $\theta\mathcal{Z}$ hosting at the images of what $\mathcal{Z}$ hosts and $\mathrm{IDs}(\theta\pi) \cup \mathrm{IDs}(\theta\varphi)$ being the image of $\mathrm{IDs}(\pi) \cup \mathrm{IDs}(\varphi)$ by (i) of that proposition. The containment goes both ways, θ^{-1} being a relocation by (vi). For the slot, an adversary occupies $\mathrm{IDs}(\mathcal{A})$, whose identities carry the process id (iii) of Definition B.1 fixes and party components θ leaves alone, and it carries $\mathcal{A}$'s wrapper, which is pid-blind; so θ relabels the core alone and $\theta\mathcal{A}$ is an adversary of the same shape. Definition 10.3 asks $\mathbf{N} \supseteq \mathbf{S} \cup \mathbf{Z}$, a set θ fixes by (ii), so simulators go to simulators. The dummy's core is pid-blind — line 3 places the call its payload names — so $\theta\mathcal{D} = \mathcal{D}$. Nothing of the kind is claimed of the $\mathcal{A}$ of Section 6, whose core Definition 10.2 leaves arbitrary, and nothing below needs it.

For (ii), fix $\mathcal{A} \in \mathbb{A}$ and let $\mathcal{S} \in \mathbb{S}$ be the simulator the hypothesis supplies for it. Given $\theta\mathcal{A} \in \theta\mathbb{A}$, return $\theta\mathcal{S} \in \theta\mathbb{S}$; it depends on $\theta\mathcal{A}$ alone, so the quantifier order of Definition 10.7 is met. For any $\theta\mathcal{Z} \in \theta\mathbb{Z}$, Lemma B.9 applied to each world gives

$$\mathrm{ADV}^{\mathrm{UC}}_{\theta\pi,\theta\varphi}\big(\theta\mathcal{Z}, \theta\mathcal{A}, \theta\mathcal{S}\big) \;=\; \mathrm{ADV}^{\mathrm{UC}}_{\pi,\varphi}\big(\mathcal{Z}, \mathcal{A}, \mathcal{S}\big) \;\leqslant\; \varepsilon,$$

the two executions on the left being the θ-images of the two on the right. By (i) the conclusion is a legal statement of emulation, $\theta\mathbb{Z} \subseteq \mathbb{Z}_{\theta\pi,\theta\varphi}$.

Session-uniformity is preserved throughout: θ fixes $\mathbf{A}$ and $\mathbf{Z}$ by (ii) of Proposition B.5 and GLOBAL by its (iii), so $\theta\mathcal{F}$ is session-uniform exactly when $\mathcal{F}$ is, and the same of a system and of an environment memberwise. Every class below is therefore closed under θ in this respect as well.

For (iii), a machine and its image take the same steps and answer the same calls, the code being identity-atomic and its control flow therefore the same on both (Definition B.4); a member of a budgeted system and its image carry the same budget for the same reason. So θ is a bijection of each of the three classes onto the class the conclusion quantifies over: $\mathbb{A}(\mathbf{a})$ and $\mathbb{S}(\mathbf{s})$ onto themselves, and $\mathbb{Z}_{\pi,\varphi}(\mathbf{z})$ onto $\mathbb{Z}_{\theta\pi,\theta\varphi}(\mathbf{z})$ by (i), each with inverse induced by θ^{-1}. Lemma B.9 makes the advantage of Definition 10.1 equal at corresponding triples, so the two nested extrema of Definition 11.2 are the same three operators applied to one function reindexed along bijections, and reindexing changes neither a supremum nor an infimum. No finiteness is needed here; what the finiteness of Definition 11.2 settles is that the extrema are *attained*, which is a separate matter and holds on both sides alike. □

Emulation is thus a property of a system up to relocation, and a proof may fix the session and the instance at the outset. To say so for a family of instances, one needs to know when two lists of process ids are related by a relocation at all, and the answer is three conditions.

Proposition B.11 (When one tuple relocates to another). *Let $\vec{q} = (q_1, \dots, q_m)$ and $\vec{q}\,' = (q'_1, \dots, q'_m)$ be tuples of process ids of standard name. There is a relocation θ with $\theta(q_j) = q'_j$ for every j if and only if, for all j and k,*

$$q_j.\mathsf{F} = q'_j.\mathsf{F}, \qquad \big(q_j.s = q_k.s \iff q'_j.s = q'_k.s\big), \qquad \big(q_j = q_k \iff q'_j = q'_k\big).$$

In particular any two process ids of one name are related by a relocation.

Proof. Necessity is the conditions of Definition B.1 read off in order: names by (i), sessions shared or not by (ii), and equality by θ being injective.

For sufficiency, let S and S' be the finitely many sessions occurring in $\vec{q}$ and in $\vec{q}\,'$. The second condition makes $q_j.s \mapsto q'_j.s$ a well-defined bijection $S \to S'$ — onto, every $q'_j.s$ being an image — so what is left over on the two sides is of one size, finite or infinite alike, and it extends to a permutation $\hat{\theta}$ of session ids. Now fix a name F and a session $s \in S$ and consider the indices j with $q_j.\mathsf{F} = \mathsf{F}$ and $q_j.s = s$. Their instance assignment $q_j.i \mapsto q'_j.i$ is well defined and injective: if $q_j.i = q_k.i$ then $q_j = q_k$, so $q'_j = q'_k$ by the fourth condition and the instances agree; and if $q_j.i \neq q_k.i$ then $q_j \neq q_k$, so $q'_j \neq q'_k$ while their names and sessions agree, and the instances differ. Each is again a bijection between the instance numbers occurring on the two sides, so it extends to a permutation $\theta_{\mathsf{F},s}$ as $\hat{\theta}$ did; take the identity at every other name and session, and let θ act as $(\mathsf{F}, s, i) \mapsto (\mathsf{F}, \hat{\theta}(s), \theta_{\mathsf{F},s}(i))$ on the

standard names and as the identity on the reserved ones. Then θ meets Definition B.1, (iii) because the reserved ids are fixed outright, and it carries $\vec{q}$ to $\vec{q}\,'$. For the last claim, a single pair q, q' of one name meets the three conditions vacuously. □

Corollary B.12 (One instance suffices). *Call a pair of maps $\vec{q} \mapsto \pi(\vec{q})$ and $\vec{q} \mapsto \varphi(\vec{q})$ an* instance family *if $\pi(\theta\vec{q}) = \theta\big(\pi(\vec{q})\big)$ and $\varphi(\theta\vec{q}) = \theta\big(\varphi(\vec{q})\big)$ for every relocation θ, where $\vec{q}$ lists the process ids occupied by $\pi(\vec{q})$ and $\varphi(\vec{q})$ together — a protocol, its private subroutines and the specification it is measured against indexed by one tuple and moving with it. Suppose $\pi(\vec{q})$ and $\varphi(\vec{q})$ are session-uniform and that $\pi(\vec{q})$ UC-emulates $\varphi(\vec{q})$ within ε against $\big(\mathbb{A}(\mathbf{a}), \mathbb{S}(\mathbf{s}), \mathbb{Z}_{\pi(\vec{q}),\varphi(\vec{q})}(\mathbf{z})\big)$ cut down to the session-uniform environments, for a single $\vec{q}$. Then at every $\vec{q}\,'$ meeting the three conditions of Proposition B.11, $\pi(\vec{q}\,')$ UC-emulates $\varphi(\vec{q}\,')$ within the same ε against $\mathbb{A}(\mathbf{a})$, $\mathbb{S}(\mathbf{s})$ and the same cut-down class for that pair. In particular, for a family occupying one process id, emulation at one id gives emulation at every id of that name.*

Proof. Proposition B.11 supplies a relocation carrying $\vec{q}$ to $\vec{q}\,'$, equivariance turns it into one carrying the pair of systems to the pair at $\vec{q}\,'$, and Proposition B.10 transports the statement — session-uniformity of the relocated pair coming from that of the original, as the proof of that proposition records. The last claim is the final clause of Proposition B.11: a one-id tuple meets the three conditions against any other id of its name. □

Indexing by the whole tuple is what makes the condition satisfiable, and the point is worth recording, since indexing by the principal process id alone — the first thing one writes — does not. A relocation may fix a given q and still move a private member sitting at another id, so equivariance in q alone would force every member of $\pi(q)$ to have an id fixed by the whole stabiliser of q; and the only such ids are q itself and the reserved three, every other being movable by some relocation that fixes q. Equivariance in q alone therefore admits only systems with no private members at all, which is not the case anyone wants.

Written for a pair of systems instead, and asking what else has to move, the statement splits in two. A new *instance* of a name can be taken up by itself; a new *session* cannot, and the reason is condition (ii) of Definition B.1: a relocation permutes sessions as blocks, so nothing can carry one member out of session s while leaving another behind in it.

Corollary B.13 (Moving one instance, moving one session). *Let π and φ be session-uniform systems each carrying a member at one and the same process id q_0, and let q be a process id with $q.F = q_0.F$ occupied by neither system. Suppose π UC-emulates φ within ε against $\big(\mathbb{A}(\mathbf{a}), \mathbb{S}(\mathbf{s}), \mathbb{Z}_{\pi,\varphi}(\mathbf{z})\big)$. In each case below the conclusion is that the moved pair UC-emulates within the same ε against $\mathbb{A}(\mathbf{a})$, $\mathbb{S}(\mathbf{s})$ and the largest admissible class of that pair at $\mathbf{z}$.*

(i) If $q.s = q_0.s$, the transposition $\theta := (q_0\ q)$ of instance numbers at the name $q_0.F$ in that session is a relocation, and it fixes the process id of every other member of either system. So $\theta\pi$ and $\theta\varphi$ are π and φ with the instance at q_0 carried to q and nothing else disturbed — save that a member naming the moved instance in a field, by pinning it among its callers in the manner of Remark A.5 or otherwise, has that mention carried along with it.

(ii) If $q.s \neq q_0.s$*, no relocation carries* q_0 *to* q *while fixing a process id of standard name in session* $q_0.s$ *— so no member of a system lying there is fixed, and the block moves entire. One carrying* q_0 *to* q *exists; and if in addition neither system has a member in session* $q.s$*, one may be chosen that carries each member of* π *and of* φ *lying in session* $q_0.s$ *into session* $q.s$ *and fixes the process id of every member lying elsewhere. Without that further hypothesis the members already in session* $q.s$ *move too — by the same argument applied at* q*, no relocation carrying* q_0 *to* q *can fix one of them — so the conclusion is then about a pair with two blocks moved rather than one.*

Proof. Both parts are Proposition B.10 once the relocation is exhibited, its part (iii) supplying the classes. For (i), the transposition has $\hat{\theta} = \text{ID}$, so conditions (i) and (ii) of Definition B.1 hold, and (iii) does because $q_0.F$ is the name of a member of a system and so none of A, Z, C. It moves only the two ids q_0 and q, and q is occupied by neither system, so every other occupied id is fixed; what is not fixed is a field naming q_0, which Definition B.3 relabels.

For (ii), suppose $\theta(q_0) = q$ and let p be any process id of standard name with $p.s = q_0.s$; by condition (ii) of Definition B.1, $\theta(p).s = \theta(q_0).s = q.s \neq p.s$, so $\theta(p) \neq p$. For existence, take $\hat{\theta}$ the transposition of the two sessions and instance permutations carrying $q_0.i$ to $q.i$ at the name $q_0.F$ in session $q_0.s$, the identity elsewhere. That θ carries session $q_0.s$ into session $q.s$ and fixes every id of a third session; if no member lies in session $q.s$, the members it moves are exactly those of session $q_0.s$. For the final sentence, let θ carry q_0 to q and let m be a member lying in session $q.s$. Then θ^{-1} is a relocation by (vi) of Proposition B.5 and carries q to q_0 with $q_0.s \neq q.s$, so the negative claim applied to it moves every standard id of session $q.s$, m among them; and $\theta(m) = m$ would give $\theta^{-1}(m) = m$, so θ moves m too. □

Sessions and instances are not the only components a permutation can move, and the third is what settles a question Section 12 raises: a game exercises $\mathcal{F}$ under a claim carrying the game's own name (line 5), so $\mathcal{F}.\mathbf{N}$ must name the game, while the $\mathcal{F}$ that ships inside a protocol admits that protocol instead. The two differ in one field, and it is the field GUARD reads.

Definition B.14 (Renaming). A *renaming* is a permutation ν of the process ids of the form

$$\nu(F, s, i) \;=\; \big(\bar{\nu}(F),\; s,\; i\big)$$

for a permutation $\bar{\nu}$ of the names fixing A, Z and C. It acts on identities, fields, code and declared sets as Definition B.3 prescribes, and in addition replaces every name occurring outright in code by its image.

The extra clause is needed and was not before. Definition B.3 substitutes process ids, which for a relocation is enough, names being what a relocation keeps; a renaming moves them, so a test against a bare name — $id'.F \neq A$ in MEDIATE, or the $id'.F = Z$ the dummy of Appendix D dispatches on — has to travel too. For the machines of this paper it travels nowhere: the only names they mention outright are the three $\bar{\nu}$ fixes, so a renaming changes no line of code at all, only fields.

Proposition B.15 (Renaming). *Let ν be a renaming. Then*

(i) $\textsc{Ids}(\nu\mathcal{F}) = \nu(\textsc{Ids}(\mathcal{F}))$; $\nu(\mathbf{S}) = \mathbf{S}$, $\nu(\mathbf{A}) = \mathbf{A}$ *and* $\nu(\mathbf{Z}) = \mathbf{Z}$; $\textsc{global}(\nu\mathcal{F}) \iff \textsc{global}(\mathcal{F})$; $\nu\mathcal{F}$ *is session-uniform exactly when* $\mathcal{F}$ *is;* $\langle\nu\pi\rangle = \nu(\langle\pi\rangle)$; $\nu\pi$ *is a system with* $\textsc{wf}(\pi) \iff \textsc{wf}(\nu\pi)$; *and* ν^{-1} *is a renaming;*

(ii) for every system π*, every occupant* $\mathcal{X}$ *of the adversary slot and every environment* $\mathcal{Z}$ *hosting at no identity of* $\textsc{Ids}(\pi)$ *— with no session-uniformity asked of any of them —*

$$\textsc{Exec}(\nu\pi, \nu\mathcal{Z}, \nu\mathcal{X}, \mathcal{C}) \quad \textit{and} \quad \textsc{Exec}(\pi, \mathcal{Z}, \mathcal{X}, \mathcal{C})$$

are identically distributed; and

(iii) $\mathbb{Z}_{\nu\pi,\nu\varphi} = \nu(\mathbb{Z}_{\pi,\varphi})$; *if* π *UC-emulates* φ *within* ε *against* $(\mathbb{A}, \mathbb{S}, \mathbb{Z})$ *then* $\nu\pi$ *UC-emulates* $\nu\varphi$ *within* ε *against* $(\nu\mathbb{A}, \nu\mathbb{S}, \nu\mathbb{Z})$; *and the concrete advantages of Definition 11.2 agree at every budget.*

Proof. (i) is the proof of Proposition B.5 with two changes of bookkeeping. Name classes are now permuted rather than carried onto themselves, so an **N** declared by names becomes the one declared by the images — which is the whole point of the definition — while **S**, being the union of every standard class, and **A** and **Z**, being single reserved classes, are fixed, and with them GLOBAL, Definition B.2 and the reserved ids of (iii) of Definition B.1. And a name closure is carried rather than fixed, $\langle\rangle$ being specified by the names a system uses. For WF the session conjunct is untouched, ν moving no session, and the name match holds on one side exactly when on the other.

(ii) is the proof of Lemma B.9 verbatim but for its one hard paragraph, which becomes trivial: line 3's first disjunct is literally unchanged, ν leaving every session alone, and its second is preserved by (i). Nothing needs excluding, which is why no session-uniformity appears here. Every other check reads a name for equality or a membership in a set that travels with the code, and Definition B.4 together with the extra clause of Definition B.14 makes the substitution compute the relabelled behaviour as before.

(iii) is the proof of Proposition B.10, which used of the closures only that $\nu(\langle\pi\rangle \cup \langle\varphi\rangle) = \langle\nu\pi\rangle \cup \langle\nu\varphi\rangle$ and never that they stay put. What is lost is the reading recorded after Proposition B.5: the environment class of the conclusion corresponds to the one of the hypothesis rather than being constrained identically, since *par* must now name the images. □

Corollary B.16 (The caller's name is immaterial). *Let ν be a renaming and let $\gamma = \{\mathfrak{Gm}\} \cup \sigma$ be a game system for $\mathcal{F}$ (Definition 12.1). Then $\nu\gamma = \{\nu\mathfrak{Gm}\} \cup \nu\sigma$ is a game system, $\nu\mathfrak{Gm}$ is a property functionality for $\nu\mathcal{F}$, and*

$$\textsc{Adv}^{\nu\mathfrak{Gm}}_{\nu\sigma}(\nu\mathcal{Z}, \nu\mathcal{X}) = \textsc{Adv}^{\mathfrak{Gm}}_{\sigma}(\mathcal{Z}, \mathcal{X})$$

for every $\mathcal{Z} \in \mathbb{Z}_{\gamma,\gamma}$ and every occupant $\mathcal{X}$ of the adversary slot. Since $\nu\mathcal{D} = \mathcal{D}$, the abbreviated form reads $\textsc{Adv}^{\nu\mathfrak{Gm}}_{\nu\sigma}(\nu\mathcal{Z}) = \textsc{Adv}^{\mathfrak{Gm}}_{\sigma}(\mathcal{Z})$. So σ has the property $\mathfrak{Gm}$ within ε against a class of environments exactly when $\nu\sigma$ has $\nu\mathfrak{Gm}$ within ε against the image of that class.

Proof. That $\nu\gamma$ is a system with $\textsc{wf}(\nu\gamma)$ is (i). For $\nu\mathfrak{Gm}$ being a property functionality, check the four fields of Definition 12.1: its process id is $(\bar{\nu}(\mathfrak{Gm}.\mathsf{F}), 0, 0)$, ν moving neither session

nor instance; its $\mathbf{P}$ is untouched and $\nu\mathcal{F}.\mathbf{P} = \mathcal{F}.\mathbf{P}$, so it is $\nu\mathcal{F}.\mathbf{P} \cup \{Z\}$ as required; its $\mathbf{N}$ is $\nu(\mathbf{Z}) = \mathbf{Z}$ by (i); its entry is $(\nu\mathcal{F}, \text{SERVES}_{\mathcal{G}\mathfrak{m}})$, and $\text{SERVES}_{\mathcal{G}\mathfrak{m}}$ is equivariant, reading $\mathbf{P}$ and a membership of $\mathbf{N}$. Line 7 still serves FINALIZE at the root, party ids being no part of a process id. For the advantage, WIN is the event that the interface at $(\mathcal{G}\mathfrak{m}.\mathit{pid}, Z)$ returns 1, and ν carries that identity to $(\nu\mathcal{G}\mathfrak{m}.\mathit{pid}, Z)$; by (ii) the two executions are identically distributed under the correspondence, so the event has the same probability on both sides and Definition 12.8 gives the same number. That $\nu\mathcal{Z}$ is admissible for $\nu\gamma$ is (iii). For the abbreviated form, $\nu\mathcal{D} = \mathcal{D}$: the dummy's fields are $\mathit{pid} := A$ and sets of ids that (i) fixes, and its core dispatches on $\mathtt{id}'.\mathsf{F} = Z$ and places the call its payload names, so no name it mentions moves. □

Read at the transposition of $\mathcal{G}\mathfrak{m}.\mathsf{F}$ with a name γ does not use, and with $\mathcal{G}\mathfrak{m}.\mathsf{F}$ carried by $\mathcal{G}\mathfrak{m}$ alone, this says what one wants of Definition 12.1. The game may be given any name and $\mathcal{F}.\mathbf{N}$ renamed to match; no other field moves, and by Definition B.14 no line of code does either. A property is therefore not tied to what its game is called, and the $\mathcal{F}$ whose property one proves may admit whatever caller name the deployment will use.

What no renaming gives is a different caller *programme*. It carries a game to a game and a protocol to a protocol, never a game to the protocol that will use $\mathcal{F}$ in earnest — and a protocol that hands $\mathcal{F}$'s answers to the adversary satisfies no property of $\mathcal{F}$, so nothing could. Proposition B.7 narrows the gap from the other side, $\mathbf{N}$ being read nowhere but inside $\text{GUARD}_{\mathcal{F}}$: a property is a statement about $\mathcal{F}$'s code and the access control around it, and about nothing else that field could carry. Crossing the rest is what emulation is for, and Proposition 12.10 is the crossing. Relocations and renamings compose, each being an isomorphism of executions, so a system may be moved in name, session and instance at once.

Remark B.17 (Session 0 is distinguished)**.** Session-uniformity is not an artefact of the proofs, and the functionality it excludes is a real one. Line 3 compares the *caller's* session, and $\mathcal{A}$ and $\mathcal{Z}$ sit in session 0, so a functionality that is *local* in the sense of Section 1 yet admits $\mathbf{A}$ is addressable from the adversary slot exactly when its own session is 0: with $\mathcal{F}.\mathtt{s} \neq 0$ the conjunct $\mathtt{id}'.\mathtt{s} = \mathtt{id}.\mathtt{s}$ fails and $\text{GLOBAL}(\mathcal{F})$ does not save it. The difference is observable — it is how $\mathcal{D}$ carries out an instruction of Appendix D at a corrupt party — so for such a pair no relocation moving session 0 is an isomorphism, and Definition B.2 is necessary rather than convenient. It is also the least one can exclude: for every other callee the supplied machines are waived by GLOBAL or refused at line 2, so no smaller condition would do and no larger one is needed.

The shape it excludes is not exotic, and one place the paper itself reaches for it should be said plainly. Remark A.5 offers, as a way of closing the hosting route to an *interface* member, that such a member admit only $\mathbf{Z} \cup \mathbf{A}$ — the environment and the adversary and no one else. A member of that shape is not global, $\mathbf{S}$ lying outside its $\mathbf{N}$, so it is not session-uniform. That is no defect of Definition B.2 but the artefact showing through at the place it costs most: line 3 lets the environment reach such a member in session 0 and in no other, so a protocol whose interface members admit exactly $\mathbf{Z} \cup \mathbf{A}$ is drivable only in session 0, whatever relocation has to say about it. The two routes an admissible environment has into a protocol divide accordingly. It may claim a fresh name, which line 2 admits only at a member admitting all of $\mathbf{S}$ — global, hence shared across sessions, hence not one run of one protocol. Or it may claim its own Z-identity, which line 2 admits at a member admitting

Z — and then line 3 pins that member to session 0. Neither route reaches a session-local protocol interface in a session of its own choosing.

That the environment can claim its own Z-identity at all is worth recalling, since it is what makes the second route available: the name Z is used by no system, so Z-identities lie outside $\langle\pi\rangle \cup \langle\varphi\rangle$ and Definition 10.6 leaves them claimable. A member whose **N** is only its parent's name class is reachable by neither route, and there the artefact shows through the adversary's slot alone.

Reading line 3 as

$$\big(\mathrm{id}'.s = \mathrm{id}.s \;\vee\; \textsc{global}(\mathcal{F}) \;\vee\; \mathrm{id}'.F \in \{A, Z\}\big)$$

would dispense with the condition. Session 0 would then confer nothing, a local functionality admitting the adversary would be reachable from the slot in every session alike, and Definition B.2 could be dropped from every statement above: the results would hold of all systems rather than the session-uniform ones. Nothing else in the appendix would change — condition (iii) stays, the reserved ids being named outright in code whatever the session rule is, and the tuple conditions of Proposition B.11 are already free of session 0. The widening is small and in the spirit of the rest: the two machines it names are global by GLOBAL's second disjunct wherever GLOBAL is tested of them, the corruption gate of line 3 still confines $\mathcal{A}$ to corrupt parties, and $\mathcal{Z}$ stays out by **N** and by its own *par*. What it would buy is more than relocation: it is what would let a protocol be local to its session and driven by the environment at the same time, which is the pairing the two routes above deny and which Remark A.5's suggestion silently assumes. We keep the narrower rule, which is Canetti's, and record both the option and its price here.

Remark B.18 (Relocation is not a many-instance theorem)**.** Proposition B.10 moves one copy and charges nothing. It does not follow, and is not true, that n copies at n indices cost ε rather than $n\varepsilon$: the executions of $\pi(\vec{q}_1), \ldots, \pi(\vec{q}_n)$ and of $\varphi(\vec{q}_1), \ldots, \varphi(\vec{q}_n)$ are not each other's relabellings — no relocation carries a system to n copies of itself — and the only route between them is the hybrid chain of Theorem C.3, which adds. What the two results give together is the statement one wants from a single proof: prove $\pi(\vec{q})$ emulates $\varphi(\vec{q})$ at one $\vec{q}$; Corollary B.12 gives it at every other within ε; and Theorem C.3, whose disjointness hypotheses come to the $\vec{q}_k$ being pairwise disjoint and clear of the surrounding shell, replaces all n at once within $n\varepsilon$, at the budgets Appendix E charges. Relocation is also functorial in the constructions of Section 9 and Appendix A, each of which reads only names, ids and identity sets: $\theta\big(\rho[\pi/\varphi]\big) = (\theta\rho)[\theta\pi/\theta\varphi]$ since an injective action commutes with the set difference, $\theta\overline{\pi} = \overline{\theta\pi}$ — the right-hand side read, as Definition A.2 requires, as the blocked system of the pair $(\theta\pi, \theta\varphi)$ — since a blocker copies the two fields θ relabels and $I_{\theta\varphi} \setminus I_{\theta\pi} = \theta(I_\varphi \setminus I_\pi)$, and $\theta\big(\mathcal{Z}^{\rho\setminus\varphi}\big) = (\theta\mathcal{Z})^{\theta\rho\setminus\theta\varphi}$ since Definition B.3 relabels a bundle's declared outward set, whose reserved half $\{\mathrm{id} : \mathrm{id}.F \in \{A, C\}\}$ is fixed by (ii) of Proposition B.5. So a relocated proof of composition is the composition of the relocated proof.

C Parallel Composition

Theorem 10.15 replaces one subsystem. Replacing several at once is not a new argument but n applications of that one along a chain of hybrids; what needs care is the bookkeeping of the three classes down the chain.

The disjointness hypotheses below are meant literally, and are satisfiable only because of the convention of Section 1: the register is a machine of the execution and a member of no system, and there is only ever one of it. Counted a member, it would lie in every φ_i and every π_k at once, so no two subsystems of ρ would be disjoint and no two π_k would carry disjoint process ids; the hypotheses would be unsatisfiable for $n \geqslant 2$ and would have to be read modulo $\mathcal{C}$. Keeping the register out of the systems is what lets them be read as written.

Definition C.1 (Simultaneous replacement). Let ρ be a system and let $\varphi_1, \dots, \varphi_n \subseteq \rho$ be pairwise disjoint subsystems. For systems $\pi_1, \dots, \pi_n$,

$$\rho[\pi_1/\varphi_1, \dots, \pi_n/\varphi_n] \;:=\; \Big(\rho \setminus \bigcup_{i=1}^{n} \varphi_i\Big) \cup \bigcup_{i=1}^{n} \pi_i.$$

Write $\rho_k := \rho[\pi_1/\varphi_1, \dots, \pi_k/\varphi_k]$ for $0 \leqslant k \leqslant n$, so that $\rho_0 = \rho$ and $\rho_n = \rho'$. Since the φ_i are pairwise disjoint, φ_k is untouched by the first $k-1$ replacements, so $\varphi_k \subseteq \rho_{k-1}$ and

$$\rho_k \;=\; \rho_{k-1}[\pi_k/\varphi_k].$$

Consecutive hybrids therefore differ by a single replacement, which lets Theorem 10.15 act on each step; the proof below walks the chain as n game hops, one per replacement. Fitting the steps together also needs the classes to be adjustable, and they are.

Lemma C.2 (Monotonicity). *Suppose π UC-emulates φ within ε against $(\mathbb{A}, \mathbb{S}, \mathbb{Z})$. Then it does so against $(\mathbb{A}^-, \mathbb{S}^+, \mathbb{Z}^-)$ for any $\mathbb{A}^- \subseteq \mathbb{A}$, any class of simulators $\mathbb{S}^+ \supseteq \mathbb{S}$ and any $\mathbb{Z}^- \subseteq \mathbb{Z}$.*

Proof. Definition 10.7 is a $\forall\exists\forall$ statement over the three classes. Narrowing the range of either universal quantifier and widening that of the existential one each preserve it, and $\mathbb{Z}^- \subseteq \mathbb{Z} \subseteq \mathbb{Z}_{\pi,\varphi}$ keeps the result a legal statement of emulation. □

Informally, Theorem C.3 says that replacing n pairwise disjoint subsystems at once costs the sum of the individual errors: it is Theorem 10.15 applied n times along a chain of hybrids, the adversaries being those of the last replacement, the simulators those of the first, and the environments admissible for every step at once.

Theorem C.3 (Parallel composition). *Let ρ be a system, let $\varphi_1, \dots, \varphi_n \subseteq \rho$ be pairwise disjoint, and let $\pi_1, \dots, \pi_n$ be systems such that the π_k carry pairwise disjoint process ids and, for each k, π_k shares no process id with $\rho \setminus \varphi_k$. Suppose that for each k*

$$\pi_k \text{ UC-emulates } \varphi_k \text{ within } \varepsilon_k \text{ against } (\mathbb{A}_k, \mathbb{S}_k, \mathbb{Z}_k),$$

and write $\mathbb{Z}'_k$ for the class Theorem 10.15 attaches to the k-th step,

$$\mathbb{Z}'_k \;=\; \big\{\, \mathcal{Z} \in \mathbb{Z}_{\rho_k,\,\rho_{k-1}} \;:\; \mathcal{Z}^{\rho_{k-1}\setminus\varphi_k} \in \mathbb{Z}_k \,\big\}.$$

Assume finally that

$$\mathbb{S}_{k+1} \subseteq \mathbb{A}_k \quad (1 \leqslant k < n), \qquad \mathbb{Z}^\star \;\subseteq\; \bigcap_{k=1}^{n} \mathbb{Z}'_k.$$

Then $\mathbb{Z}^\star \subseteq \mathbb{Z}_{\rho',\rho}$, *and*

$$\rho' := \rho[\pi_1/\varphi_1, \ldots, \pi_n/\varphi_n] \text{ UC-emulates } \rho \text{ within } \sum_{k=1}^{n} \varepsilon_k \text{ against } (\mathbb{A}_n, \mathbb{S}_1, \mathbb{Z}^\star).$$

Proof. First, each step is well-formed. The members of $\rho_{k-1} \setminus \varphi_k$ are those of $\rho \setminus \varphi_k$ the first $k-1$ replacements leave in place, together with $\pi_1, \ldots, \pi_{k-1}$; by hypothesis π_k shares a process id with neither group. So the replacement of φ_k by π_k in ρ_{k-1} is well-formed in the sense of Definition 9.2. Each ρ_k is a system, by induction on k: $\rho_0 = \rho$ is one, and $\rho_k = \rho_{k-1}[\pi_k/\varphi_k]$ is one by Proposition 9.3 from that well-formed replacement. So Theorem 10.15 and Proposition 10.11, which ask ρ_{k-1} and π_k to be systems, apply to the k-th step.

Next, admissibility. Let $\mathcal{Z} \in \mathbb{Z}^\star$ and fix k. From $\mathbb{Z}^\star \subseteq \mathbb{Z}'_k \subseteq \mathbb{Z}_{\rho_k,\rho_{k-1}}$ we get

$$\mathcal{Z}.par \supseteq \langle \rho_k \rangle \cup \langle \rho_{k-1} \rangle,$$

so $\mathcal{Z}$ is admissible for the k-th pair of hybrids and the advantage $\mathrm{ADV}^{\mathrm{UC}}_{\rho_k,\rho_{k-1}}$ below is defined. Proposition 10.11, applied to the replacement of φ_k by π_k in ρ_{k-1}, then places the absorbed environment in $\mathbb{Z}_{\pi_k,\varphi_k}$: its parameter is $\mathcal{Z}$'s, which already contains the identities of π_k and of φ_k, so the inner advantage $\mathrm{ADV}^{\mathrm{UC}}_{\pi_k,\varphi_k}$ is defined too. Taking the union of the displayed containments over all k,

$$\mathcal{Z}.par \supseteq \bigcup_{k=0}^{n} \langle \rho_k \rangle \supseteq \langle \rho' \rangle \cup \langle \rho \rangle,$$

and likewise $\mathcal{Z}$ hosts at no identity of $\bigcup_k \mathrm{IDS}(\rho_k) \supseteq \mathrm{IDS}(\rho') \cup \mathrm{IDS}(\rho)$ — the hosting clause of each $\mathbb{Z}_{\rho_k,\rho_{k-1}}$, at $k = n$ and $k = 1$ — so both clauses of Definition 10.6 give $\mathbb{Z}^\star \subseteq \mathbb{Z}_{\rho',\rho}$ and the conclusion is itself a legal statement of emulation.

Fix $\mathcal{A} \in \mathbb{A}_n$ and put $\mathcal{S}_{n+1} := \mathcal{A}$. For $k = n, n-1, \ldots, 1$ let $\mathcal{S}_k \in \mathbb{S}_k$ be the simulator the k-th step supplies for the adversary $\mathcal{S}_{k+1}$. This is legitimate at every index: at $k = n$ because $\mathcal{S}_{n+1} = \mathcal{A} \in \mathbb{A}_n$, and at $k < n$ because $\mathcal{S}_{k+1} \in \mathbb{S}_{k+1} \subseteq \mathbb{A}_k$. The chain of simulators is built downwards, each from the one before it, and $\mathcal{S}_1$ depends on $\mathcal{A}$ alone.

Now fix $\mathcal{Z} \in \mathbb{Z}^\star$ and consider the $n+1$ games

$$G_k := \textsc{Exec}(\rho_k, \mathcal{Z}, \mathcal{S}_{k+1}, \mathcal{C}), \qquad p_k := \Pr[G_k = 1], \qquad 0 \leqslant k \leqslant n.$$

Here G_0 is the execution of ρ under $\mathcal{S}_1$ and G_n that of ρ' under $\mathcal{A}$, so the sequence starts at ρ_0 and ends at ρ_n. Consecutive games differ in one replaced subsystem and in the machine playing the adversary, and that difference is exactly one application of Theorem 10.15: for $1 \leqslant k \leqslant n$,

$$|p_k - p_{k-1}| = \mathrm{ADV}^{\mathrm{UC}}_{\rho_k,\rho_{k-1}}(\mathcal{Z}, \mathcal{S}_{k+1}, \mathcal{S}_k) \leqslant \mathrm{ADV}^{\mathrm{UC}}_{\pi_k,\varphi_k}(\mathcal{Z}^{\rho_{k-1}\setminus\varphi_k}, \mathcal{S}_{k+1}, \mathcal{S}_k) \leqslant \varepsilon_k.$$

The equality is Definition 10.1 read backwards, since G_k and G_{k-1} are the two executions that the advantage $\mathrm{ADV}^{\mathrm{UC}}_{\rho_k,\rho_{k-1}}$ compares. The first inequality is the reduction of Theorem 10.15, available because $\mathcal{Z} \in \mathbb{Z}^\star \subseteq \mathbb{Z}'_k$ by hypothesis, so Lemma C.2 makes the k-th step a statement against $\mathbb{Z}^\star$. The second is the emulation of φ_k by π_k, applied to the adversary $\mathcal{S}_{k+1}$ and the absorbed environment.

Summing the n hops and telescoping,

$$\mathrm{ADV}^{\mathrm{UC}}_{\rho',\rho}(\mathcal{Z},\mathcal{A},\mathcal{S}_1) \;=\; |\,p_n - p_0\,| \;\leqslant\; \sum_{k=1}^{n} |\,p_k - p_{k-1}\,| \;\leqslant\; \sum_{k=1}^{n} \varepsilon_k.$$

As $\mathcal{S}_1 \in \mathbb{S}_1$ was chosen from $\mathcal{A}$ alone and $\mathcal{Z} \in \mathbb{Z}^\star$ was arbitrary, this is the statement of the theorem. □

Corruption along the chain. Corruption behaves along the chain as it does at a single step. Within a game the register is one machine underneath $\rho \setminus \varphi_k$, π_k and φ_k alike, keyed by party, so a hop cannot corrupt a party halfway; across games the registers are separate instances and carry nothing between hops. The environment corrupts the same parties at every hop, being the same machine placing the same calls under line 3; the slot does not have to, so the two machines that change per hop — the replaced subsystem and the slot occupant $\mathcal{S}_k \to \mathcal{S}_{k+1}$ — need not corrupt the same parties; but the environment reads **C** unmediated (Remark 3.1), so a $\mathcal{S}_k$ corrupting differently is caught by the same ε_k, and commensurateness is again a consequence rather than an assumption. The parties $\rho_{k-1} \setminus \varphi_k$ serves are handled alike: their MEDIATE hooks leave the absorbed environment for the slot, which is why the outward set of Definition 10.10 carries the ids A and C. So the theorem claims only that no environment in $\mathbb{Z}^\star$ tells the corruption patterns apart by more than $\sum_k \varepsilon_k$.

Remark C.4 (The largest classes)**.** The union of the hybrid closures has a closed form. Each ρ_k consists of the untouched part of ρ together with $\pi_1, \ldots, \pi_k$, so

$$\bigcup_{k=0}^{n} \langle \rho_k \rangle \;=\; \langle \rho \rangle \cup \bigcup_{i=1}^{n} \langle \pi_i \rangle \;=\; \langle \rho' \rangle \cup \langle \rho \rangle.$$

Suppose now that each $\mathbb{Z}_k$ is the largest admissible class $\mathbb{Z}_{\pi_k,\varphi_k}$. Proposition 10.11 then makes the second condition defining $\mathbb{Z}'_k$ redundant, so $\mathbb{Z}'_k = \mathbb{Z}_{\rho_k,\rho_{k-1}}$, and the largest permissible choice of $\mathbb{Z}^\star$ is

$$\bigcap_{k=1}^{n} \mathbb{Z}_{\rho_k,\rho_{k-1}} \;=\; \{\, \mathcal{Z} : \mathcal{Z}.\mathit{par} \supseteq \textstyle\bigcup_{k=0}^{n} \langle \rho_k \rangle \,\} \;=\; \mathbb{Z}_{\rho',\rho}.$$

Parallel composition against the largest classes therefore delivers the largest class again, as the single-step theorem does.

The three classes behave as the chain suggests. The adversaries are those of the *last* replacement, the argument starting from an adversary against $\rho_n = \rho'$; the simulators are those of the *first*, the chain ending by producing a simulator against $\rho_0 = \rho$. Each intermediate class serves twice, as the simulators of one step and the adversaries of the next, which is what $\mathbb{S}_{k+1} \subseteq \mathbb{A}_k$ arranges. The environments must be admissible for every step at once, hence the intersection. The advantage is additive, as a hybrid argument over n steps always makes it.

D Completeness of the Dummy Adversary

The dummy adversary carries out whatever the environment tells it and reports back whatever it is told; it has no strategy of its own. Anything a real adversary achieves, an

environment can achieve by driving the dummy, which is why quantifying over adversaries can be replaced by quantifying over environments. This appendix defines it and proves that, for the systems and classes the theorem makes precise — Remark F.4 records the one regime, cores placing responsive calls, that the claim does not reach.

What the framework already fixes. The dummy is not a new kind of machine. It is the adversary of Section 6 with a particular core, so it carries $\mathcal{A}$'s fields and its calls pass through $\mathcal{A}$'s wrapper. Three consequences settle the shape of the code before a line of it is written.

First, it may claim one identity only. Line 2 of the $\mathcal{A}$ box silences a core on $\{\mathtt{id}'' \neq \mathtt{id}\}$, so every call the dummy places claims $\mathtt{id}$, the identity it runs at. An instruction of the form "call $\mathtt{id}''$.OP *as* $\mathtt{id}'''$" therefore cannot be carried out for an arbitrary $\mathtt{id}'''$: the dummy speaks as $(A, \mathtt{id}.P)$ or not at all. What the environment chooses is not the claimed id but the party, by instructing the dummy at (A, P) rather than at (A, Q); the root of $\mathcal{Z}$ may act for any party and so may pick either.

Second, the party of the target is forced. GUARD asks $\mathtt{id}'.P = \mathtt{id}.P$ of the callee, and the claim is $(A, \mathtt{id}.P)$, so the dummy at (A, P) reaches interfaces of party P and no other. A target that does not admit **A**, or an honest party, refuses the call by line 2 or line 3; the dummy passes the refusal back, and the rule of Section 8 delivers it to the dummy's own caller as $\perp$.

Third, its route to the environment is fixed too: $(A, P) \to (Z, P)$, which GUARD$_{\mathcal{Z}}$ admits because $\mathbf{A} \subseteq \mathcal{Z}.\mathbf{N}$ and the parties agree.

The code. The core dispatches on who called it. A call from the environment is an instruction to carry out; a call from anything else — a mediation hook, or an $\mathcal{A}(\cdot)$ placed inside some core — is a message to hand on. In both directions the payload has the shape MEDIATE already uses, an operation together with a value.

Dummy adversary $\mathcal{D}$
pid $:= A$, $\mathbf{P} := \{0,1\}^* \cup \{A, Z\}$, $\mathbf{N} := \mathbf{S} \cup \mathbf{A} \cup \mathbf{Z}$, $\mathbf{U} := \{(\mathcal{Z}, \text{SERVES})\}$, *par* $:= \perp$

id.A(in) from $\mathtt{id}'$

1: **if** $\mathtt{id}'.F = Z$ **then**
2: parse in as $(\mathtt{id}''.\text{OP}, x)$ *// unparseable: return $\perp$*
3: **return** $\mathtt{id}''$.FULLOP(x) as id *// carry out the instruction*
4: **else**
5: **return** $(Z, \mathtt{id}.P)$.FULLZ$(\mathtt{id}', \text{in})$ as id *// hand it to $\mathcal{Z}$, caller and all*

Line 3 is the outward direction: the environment names a target interface and an input, and the dummy places exactly that call and returns exactly its answer. Nothing is inspected and nothing remembered — the dummy keeps no state, so it has no INITIALIZE. Line 5 is the inward one: whatever reaches the adversary is handed to the environment together with the identity it came from, and whatever the environment answers is returned to that caller unchanged. The two lines are the same relay read in opposite directions, which is what makes the dummy transparent.

An instruction line 2 cannot parse is answered with $\perp$: the guards answer anything the dummy cannot *place* with REJ, but a payload that names no call at all needs a value of its own, and $\perp$ is what a machine with nothing to say returns.

One detail is worth pausing on. The caller $\mathtt{id}'$ is passed to $\mathcal{Z}$ on line 5 and not merely

the payload: without it the environment could not tell which interface is asking, and a relay that loses that cannot be transparent.

Interposing an adversary. Completeness is a statement about moving an adversary from the execution into the machines around it, and needs a name for each move. Both are the same operation seen from two sides: $\mathcal{A}$ is placed between some machine and the dummy interface, so what used to reach $\mathcal{A}$ now reaches it inside, and what $\mathcal{A}$ used to place now leaves as an instruction to $\mathcal{D}$.

Definition D.1 (Adversary absorbed into the environment)**.** Let $\mathcal{Z}$ be an environment and $\mathcal{A}$ an adversary. The environment $\mathcal{Z}^{\mathcal{A}}$ is the bundle (Definition 10.8) of $\mathcal{Z}$ with one copy of $\mathcal{A}$, the copy *not* keeping its identities; it occupies $\mathrm{IDs}(\mathcal{Z})$ alone and takes its output and *par* from $\mathcal{Z}$. A call $\mathcal{Z}$ places on an A-identity is served internally by $\mathcal{A}$'s interface there, and one $\mathcal{A}$ places on a Z-identity by $\mathcal{Z}$'s. Any other call the hosted $\mathcal{A}$ places, on $\mathtt{id}''.\textsc{FullOp}(x)$ while running at (A, P), is rewritten to the instruction

$$(A, P).\textsc{FullA}\big(\mathtt{id}''.\mathrm{Op},\ x\big) \ \text{ as } (Z, P),$$

keyed to the party $\mathcal{A}$ runs at and never its target's, so a call the unabsorbed $\mathcal{A}$ could place only as (A, P) is relayed at that same party and admitted or refused exactly where it was. A call arriving from an A-identity with payload $(\mathtt{id}', \mathtt{in})$ is handed to $\mathcal{A}$'s interface at (A, P), guard included, as a call from $\mathtt{id}'$.

Definition D.2 (Adversary interposed before a simulator)**.** Let $\mathcal{S}$ be a simulator and $\mathcal{A}$ an adversary. The simulator $\mathcal{S}[\mathcal{A}]$ is the bundle (Definition 10.8) of $\mathcal{S}$ with one copy of $\mathcal{A}$; it occupies $\mathrm{IDs}(\mathcal{A})$ and carries $\mathcal{S}$'s remaining fields — $\mathcal{S}$, not $\mathcal{A}$, being the machine it stands in for, so $\mathcal{S}[\mathcal{A}]$ is a simulator whenever $\mathcal{S}$ is. A call from a Z-identity is served by the hosted $\mathcal{A}$; one from any other process id, a mediation hook in particular, by $\mathcal{S}$. The hosted $\mathcal{A}$'s calls on a Z-identity — its route back to the environment — leave the bundle under (A, P) to the external $\mathcal{Z}$; its other outward calls become the instruction of Definition D.1, served by $\mathcal{S}$ at (A, P) as a call from (Z, P); and a call $\mathcal{S}$ places on a Z-identity is handed, past $\textsc{Guard}_{\mathcal{Z}}$, to $\mathcal{A}$ at (A, P).

Neither construction is a new kind of machine. $\mathcal{Z}^{\mathcal{A}}$ occupies what an environment occupies and claims what $\mathcal{Z}$ claimed, so it is admissible exactly when $\mathcal{Z}$ is. It occupies no A-identity, those being the relay's in the right-hand execution, so by Definition 10.8 the hosted $\mathcal{A}$ has no identity of its own to place a call under: hence the rewriting above, and hence the obligation — discharged by Siting below — to check the rewritten claim against $\mathcal{Z}$'s silencer. Absorption in Section 10 is the other case of Definition 10.8, which is why a hosted functionality there keeps its claim instead. Meanwhile $\mathcal{S}[\mathcal{A}]$ occupies $\mathrm{IDs}(\mathcal{A})$, which is what standing in the adversary slot means.

The theorem. Emulation *with respect to the dummy alone* is Definition 10.7 with the adversary class cut down to the single machine $\mathcal{D}$. The claim is that nothing is lost.

One mild condition on the adversary is needed, and it is the price of the dummy being a relay.

Definition D.3 (Refusal-oblivious)**.** An adversary is *refusal-oblivious* if its behaviour is unchanged when any REJ it receives, as the answer to a call it placed, is replaced by $\bot$.

The dummy relays such an answer, and by Section 8 its own **return** of a REJ reaches the caller as $\bot$; so where an unabsorbed adversary placing a refused call reads REJ directly, the same adversary driving the dummy reads $\bot$. A refusal-oblivious adversary cannot tell the two apart, which is what the regrouping below needs. The dummy is itself refusal-oblivious — it relays its answer without branching on it — so the hypothesis costs the dummy class nothing, and a real adversary that reads a framework-internal refusal signal is in any case a modelling artefact rather than an attack.

Informally, Theorem D.4 says that emulation with respect to the dummy alone loses nothing: anything a real adversary achieves, an environment can achieve by driving the dummy, and the simulator for $\mathcal{A}$ is $\mathcal{S}_{\mathcal{D}}[\mathcal{A}]$ — the one simulator the hypothesis supplies, with $\mathcal{A}$ interposed before it — which depends on $\mathcal{A}$ alone and not on $\mathcal{Z}$.

Theorem D.4 (Completeness of the dummy adversary)**.** *Let π and φ be systems whose cores place no responsive calls (Definition F.2), let $\varepsilon \geqslant 0$, let $\mathbb{A}$ be a class of refusal-oblivious adversaries (Definition D.3), let $\mathbb{S}$ be a class of simulators, and let $\mathbb{Z} \subseteq \mathbb{Z}_{\pi,\varphi}$. Suppose*

$$\pi \text{ UC-emulates } \varphi \text{ within } \varepsilon \text{ against } (\{\mathcal{D}\}, \mathbb{S}, \mathbb{Z}).$$

Then

$$\pi \text{ UC-emulates } \varphi \text{ within } \varepsilon \text{ against } (\mathbb{A}, \mathbb{S}', \mathbb{Z}'),$$

where

$$\mathbb{S}' = \{\, \mathcal{S}[\mathcal{A}] \,:\, \mathcal{S} \in \mathbb{S},\ \mathcal{A} \in \mathbb{A} \,\},$$
$$\mathbb{Z}' = \{\, \mathcal{Z} \in \mathbb{Z}_{\pi,\varphi} \,:\, \mathcal{Z}.par \cap \{\, \mathrm{id} : \mathrm{id}.\mathsf{F} = \mathsf{Z} \,\} = \varnothing \ \wedge\ \mathcal{Z}^{\mathcal{A}} \in \mathbb{Z} \text{ for every } \mathcal{A} \in \mathbb{A} \,\}.$$

Explicitly, let $\mathcal{S}_{\mathcal{D}} \in \mathbb{S}$ be the simulator the hypothesis supplies for $\mathcal{D}$. Then

$$\forall \mathcal{A} \in \mathbb{A}\ \forall \mathcal{Z} \in \mathbb{Z}' :\qquad \mathrm{ADV}^{\mathrm{UC}}_{\pi,\varphi}\big(\mathcal{Z}, \mathcal{A}, \mathcal{S}_{\mathcal{D}}[\mathcal{A}]\big) \leqslant \mathrm{ADV}^{\mathrm{UC}}_{\pi,\varphi}\big(\mathcal{Z}^{\mathcal{A}}, \mathcal{D}, \mathcal{S}_{\mathcal{D}}\big) \leqslant \varepsilon,$$

so the simulator Definition 10.7 asks for is $\mathcal{S}_{\mathcal{D}}[\mathcal{A}]$, which depends on $\mathcal{A}$ alone and not on $\mathcal{Z}$.

Proof. Fix $\mathcal{A} \in \mathbb{A}$ and $\mathcal{Z} \in \mathbb{Z}'$, and let $\mathcal{S}_{\mathcal{D}} \in \mathbb{S}$ be the simulator the hypothesis supplies for $\mathcal{D}$. Membership in $\mathbb{Z}'$ gives $\mathcal{Z}^{\mathcal{A}} \in \mathbb{Z}$, so the hypothesis applies to it, and $\mathcal{Z}^{\mathcal{A}}.par = \mathcal{Z}.par \supseteq \langle\pi\rangle \cup \langle\varphi\rangle$, so the advantage on the right of the display is defined. The conventions of Proposition 10.14 — independent coins, unmerged tapes, call-free INITIALIZE — are in force. The work is two regroupings, one per world:

$$\textsc{Exec}\big(\pi, \mathcal{Z}, \mathcal{A}, \mathcal{C}\big) \equiv \textsc{Exec}\big(\pi, \mathcal{Z}^{\mathcal{A}}, \mathcal{D}, \mathcal{C}\big), \tag{R1}$$
$$\textsc{Exec}\big(\varphi, \mathcal{Z}, \mathcal{S}_{\mathcal{D}}[\mathcal{A}], \mathcal{C}\big) \equiv \textsc{Exec}\big(\varphi, \mathcal{Z}^{\mathcal{A}}, \mathcal{S}_{\mathcal{D}}, \mathcal{C}\big), \tag{R2}$$

each an equality of distributions; granting them, the two advantages agree term by term and the hypothesis at $\mathcal{Z}^{\mathcal{A}}$ bounds the second by ε, which is the display. Neither is a pure rebracketing — the right-hand executions carry a machine the left-hand ones do not, the relay in the adversary slot — so both are the ceding half of Lemma 10.13 rather than

Proposition 10.14's.

Proof of (R1). Lemma 10.13 at $\mathsf{H} = \{\mathcal{A}\}$ with $\mathsf{H}_{\text{KEEP}} = \varnothing$, so that $\mathcal{Z}^{\mathsf{H}}$ is the $\mathcal{Z}^{\mathcal{A}}$ of Definition D.1 and the relay at the ceded $\text{IDs}(\mathcal{A})$ is $\mathcal{D}$. Its three hypotheses are the theorem's: $\mathcal{A}$ is refusal-oblivious because $\mathbb{A}$ is; no core of π places a responsive call, which is the hypothesis on the systems; and $\mathcal{Z} \in \mathbb{Z}'$ silences no Z-identity. The occupant families are legal, $\mathcal{Z}^{\mathcal{A}}$ occupying $\text{IDs}(\mathcal{Z})$ alone and $\mathcal{D}$ the identities $\mathcal{A}$ let go. *Proof of* (R2). Here $\mathcal{A}$ moves from one bundle to another rather than out of the execution, so Lemma 10.13 does not apply on its face and the argument is its ceding half run again with φ for π and $\mathcal{S}_{\mathcal{D}}$ for $\mathcal{D}$. That it transfers is a matter of what the lemma's proof used of the machine in the slot, which is no more than that its guard returns $\text{GUARD}_{\mathcal{A}}$'s verdicts, that it dispatches on whether the claim has $\mathsf{id}'.\mathsf{F} = \mathsf{Z}$, and — where the hosted $\mathcal{A}$ reads a refused instruction, REJ from $\mathcal{S}_{\mathcal{D}}$ inside the bundle on the left against the deemed $\perp$ across the slot on the right — that $\mathbb{A}$ is refusal-oblivious, exactly as in (R1). What differs is which machine holds $\mathcal{A}$ — $\mathcal{S}_{\mathcal{D}}[\mathcal{A}]$ on the left, $\mathcal{Z}^{\mathcal{A}}$ on the right — and Definitions D.2 and D.1 make the two perform the same hand-offs, each routed through the receiving machine's guarded interface, the instruction travelling inside a machine on one side and across the execution on the other. The chain $\mathcal{Z} \to \mathcal{A} \to \mathcal{S}_{\mathcal{D}} \to \varphi$ is the same in both, cut at a different point: $\mathcal{S}_{\mathcal{D}}$ meets the same instructions in the same order under the same claims, Siting holds verbatim, and the occupant is judged by the same fields either way — *pid* and **P** by occupancy, and **N** by Definition 10.3, which $\mathcal{S}_{\mathcal{D}}$ meets and $\mathcal{S}_{\mathcal{D}}[\mathcal{A}]$ inherits, carrying $\mathcal{S}_{\mathcal{D}}$'s fields by Definition D.2; no guard reads the rest. Definition D.2 routes the left-hand hand-off through the same guarded interface that EXEC serves on the right, and the hosted $\mathcal{A}$'s own calls back to $\mathcal{Z}$ leave the bundle under (A, P) just as they cross EXEC on the right.

Conclusion. By (R1) and (R2) the two advantages of the display are equal, and the hypothesis at $(\mathcal{D}, \mathcal{Z}^{\mathcal{A}})$ bounds the second by ε. $\mathcal{S}_{\mathcal{D}}[\mathcal{A}]$ lies in $\mathbb{S}'$ and depends on $\mathcal{A}$ alone, the hypothesis knowing nothing of $\mathcal{Z}$, so the quantifier order of Definition 10.7 is met; $\mathcal{Z} \in \mathbb{Z}'$ was arbitrary. □

The three classes move as in Theorem 10.15, and for the same reasons. The adversaries are those the conclusion quantifies over, and the hypothesis had only $\mathcal{D}$; the simulators are computed, one per adversary, by interposing it before the simulator the hypothesis supplies; and the environments are a preimage, an outer $\mathcal{Z}$ counting exactly when what it becomes on absorbing each $\mathcal{A}$ is one the hypothesis covers. Taking $\mathbb{Z}$ to be the largest admissible class makes $\mathbb{Z}'$ the largest one again — less only the environments that silence their own Z-identities, a condition no admissibility imposes and no application wants, excluded because the instruction of Definition D.1 leaves under a Z-claim, which such an environment would refuse on one side alone — since absorption leaves *par* alone.

Read concretely in the sense of Section 11, the two constructions cost what the adversary costs: if $\mathcal{A}$ is $\mathbf{a}$-bounded then $\mathcal{Z}^{\mathcal{A}}$ needs $\mathbf{z} \oplus \mathbf{a}$ where $\mathcal{Z}$ needed $\mathbf{z}$, and $\mathcal{S}_{\mathcal{D}}[\mathcal{A}]$ needs $\mathbf{s} \oplus \mathbf{a}$ where $\mathcal{S}_{\mathcal{D}}$ needed $\mathbf{s}$. The invocation counts add and the per-invocation times take a maximum, both independent of π and φ.

E Concrete Costs

Two threads of concrete cost sit outside the main line and are collected here: the cost of a system in itself, read off its members, and the cost the composition results of Section 11 charge, arithmetic on the budgets of Definition 11.2.

Budgeted member by member, a system also has a cost as a whole, and it is a functionality's cost read one level up.

Definition E.1 (Cost of a system). An *invocation* of a system π is a call reaching one of its members from outside π; π services it by a cascade of internal activations before control returns outside. The *cost* of π is the pair (T_π, R_π): the *per-invocation time* T_π, the largest total number of steps in one such cascade — summed over the activations it triggers, one machine running at a time — and the *invocation count* R_π, the number of invocations π answers. So π runs in at most $T_\pi R_\pi$ steps.

Proposition E.2 (System cost from member costs). *Let $\pi = \{\mathcal{F}_1, \dots, \mathcal{F}_n\}$ with each $\mathcal{F}_i$ of cost (t_i, r_i), and let k_i bound the activations of $\mathcal{F}_i$ within one invocation of π. Then*

$$T_\pi \leqslant \sum_i k_i\, t_i, \qquad R_\pi \leqslant \sum_i r_i,$$

and π runs in at most $\sum_i t_i r_i$ steps. If every invocation activates each member at most once — π subroutine-respecting, $k_i \leqslant 1$ — then $T_\pi \leqslant \sum_i t_i$; if moreover each invocation drives all members, $R_\pi \leqslant \min_i r_i$.

Proof. An invocation activates $\mathcal{F}_i$ at most k_i times, each for at most t_i steps, so it spends at most $\sum_i k_i t_i$. The activations of $\mathcal{F}_i$ over the whole execution number at most r_i, and every invocation is at least one activation, so the invocations number at most $\sum_i r_i$; the total steps are $\sum_i (\text{activations of } \mathcal{F}_i)\, t_i \leqslant \sum_i r_i t_i$. When each invocation drives all members, R_π invocations activate each $\mathcal{F}_i$ exactly R_π times, so $R_\pi \leqslant r_i$ for every i. □

Corollary E.3 (Cost of a union). *Let π_1 and π_2 be systems with disjoint process ids and costs (T_1, R_1) and (T_2, R_2). If no member of one invokes a member of the other, then*

$$\big(T_{\pi_1 \cup \pi_2},\ R_{\pi_1 \cup \pi_2}\big) \;=\; (T_1, R_1) \oplus (T_2, R_2) \;=\; \big(\max(T_1, T_2),\ R_1 + R_2\big).$$

If instead each invocation of π_1 makes at most k calls into π_2 and the cross-call graph is acyclic, then $T_{\pi_1 \cup \pi_2} \leqslant T_1 + k\, T_2$ from a π_1 entry, and symmetrically, with $R_{\pi_1 \cup \pi_2} \leqslant R_1 + R_2$; with the cross-calls unbounded the summary costs no longer determine the union's.

Proof. With no cross-calls a call from outside $\pi_1 \cup \pi_2$ enters π_1 or π_2 and its cascade stays there, costing at most T_1 or T_2 and belonging to that subsystem's invocation count; so the per-invocation time is $\max(T_1, T_2)$ and the counts add. A cross-call from π_1 into π_2 is internal to the union and extends the cascade by one invocation of π_2, at most T_2 steps; at most k of them, acyclically, give $T_1 + kT_2$, while a call once external to π_2 but placed from π_1 is no longer counted, so the invocations still number at most $R_1 + R_2$. □

The subroutine-respecting proviso is not idle: without it one call can be driven around the system without end, and T_π degenerates towards the whole of $\sum_i t_i r_i$ — the same discipline that keeps a private member's callers pinned in Remark A.5. This system cost is dual to the $\oplus$ of Definition 11.1: hosting a system as a bundle, one member per activation,

combines its members by $\oplus$ into $(\max_i t_i, \sum_i r_i)$, the cost $\mathbf{c}_{\rho\setminus\varphi}$ pays in the composition costs below; calling it as a subsystem sums the times over a cascade and counts external calls, $(\sum_i t_i, \min_i r_i)$ in the synchronous case. Both bound the same total $\sum_i t_i r_i$, the second more tightly when every invocation is a full pass. And $\oplus$ recurs at the system level itself: by Corollary E.3 non-interacting subsystems compose by it, one operator at both scales.

When the members of π call one another the per-invocation time turns on the shape of that calling, and one case is clean.

Proposition E.4 (DAG systems). *Suppose the call graph of $\pi = \{\mathcal{F}_1, \dots, \mathcal{F}_n\}$ is acyclic and every invocation activates each internal member $\mathcal{F}_i$ at most $\max_{j:\,\mathcal{F}_j \to \mathcal{F}_i} t_j$ times — the largest per-invocation time among the members that call it. Then*

$$T_\pi \;\leqslant\; \sum_i \Big(\max_{j:\,\mathcal{F}_j \to \mathcal{F}_i} t_j \Big) t_i \;\leqslant\; \Big(\max_i t_i \Big) \sum_i t_i,$$

a per-invocation time independent of the depth of the graph.

Proof. An invocation entering a source activates each $\mathcal{F}_i$ some a_i times and spends $\sum_i a_i t_i$ steps. A source runs once and each internal $\mathcal{F}_i$ at most $\max_{j\to i} t_j$ times by hypothesis, so $a_i \leqslant \max_i t_i$ throughout, whence $T_\pi \leqslant \sum_i (\max_{j\to i} t_j)\, t_i \leqslant (\max_i t_i) \sum_i t_i$. □

The hypothesis earns its place, for acyclicity alone does not bound T_π. Writing n_{ji} for the calls $\mathcal{F}_j$ makes to $\mathcal{F}_i$ in one activation, the per-invocation activation counts solve $a_i = \sum_{j\to i} n_{ji}\, a_j$, so a_i counts the weighted source-to-$\mathcal{F}_i$ paths, and a reconvergent graph multiplies them: a chain of N diamonds — $s \to \{A_k, B_k\} \to C_k \to \{A_{k+1}, B_{k+1}\} \to \cdots$ — doubles the count at each level, so a at the foot is 2^N and the per-invocation time is exponential in the depth. Capping each node by the *maximum* over its callers rather than by their sum is exactly what stops the paths accumulating — the $\oplus$-over-summation theme once more, and the DAG counterpart of the subroutine-respecting case $k_i \leqslant 1$ of Proposition E.2, one activation relaxed to $\max_{j\to i} t_j$. The invocation caps bound $a_i \leqslant r_i$ in any case; the point here is a bound in the times alone, independent of the caps and of the depth.

That last remark is the whole of the general statement: the caps alone already close the cost class, with no condition on the call structure.

Corollary E.5 (Cost composition). *A system $\pi = \{\mathcal{F}_1, \dots, \mathcal{F}_n\}$ whose members have costs (t_i, r_i) runs in at most $\sum_i t_i r_i$ steps over at most $\sum_i r_i$ invocations, so $T_\pi \leqslant \sum_i t_i r_i$ and $R_\pi \leqslant \sum_i r_i$ whatever its call structure. Hence if each t_i and r_i is polynomial in a parameter λ and n is polynomially bounded, (T_π, R_π) is polynomial: the polynomially bounded systems are closed under formation.*

Proof. The step and invocation bounds are Proposition E.2, and one invocation's cascade is part of the whole execution, so $T_\pi \leqslant \sum_i t_i r_i$ too. A sum of polynomially many polynomials is a polynomial. □

No condition on the call graph is needed here, unlike the depth-free *time* bound of Propo-

sition E.4, because the caps carry it. A reconvergent cascade that would activate a member 2^N times can do so only if that member answers 2^N calls, so $r_i \geqslant 2^N$ — a member already outside the polynomial class; within the class the cap refuses the excess and truncates the cascade at $\sum_i r_i$ activations. This is the efficiency counterpart of Theorem 10.15: where composition preserves *security* — $\rho[\pi/\varphi]$ emulates ρ once π emulates φ — cost composition preserves *efficiency*, so a protocol assembled from polynomial-time functionalities is itself polynomial-time. Together the two are what the concrete apparatus is for: polynomial-time UC-secure protocols compose into polynomial-time UC-secure protocols, which is what lets the asymptotic reading below be a corollary rather than a theory of its own.

The same closure survives the operation the composition theorem runs.

Proposition E.6 (Replacement preserves polynomial boundedness). *Let ρ and π be systems whose members are polynomial and polynomially many, and let the replacement of φ by π in ρ be well-formed. Then $\rho[\pi/\varphi]$ is polynomially bounded.*

Proof. The members of $\rho[\pi/\varphi] = (\rho \setminus \varphi) \cup \pi$ (Definition 9.1) are those of ρ outside φ, each polynomial as a member of ρ, together with those of π, again polynomial; they number at most $|\rho| + |\pi|$, polynomially many. Corollary E.5 applies. □

So the system that runs after a replacement is efficient whenever the specification ρ and the realization π are — Theorem 10.15 read for cost rather than security, and on the same operation.

Composition, numerically. In Theorem 10.15 the adversaries and the simulators carry over unchanged, so their budgets do. The environments are the one place something is computed, and what absorption costs is the cost of hosting. Systems are budgeted member by member, so the cost can be looked up: write $\mathbf{c}_{\rho\setminus\varphi}$ for the $\oplus$-combination of the budgets of the members of $\rho \setminus \varphi$ — the largest of their per-invocation times, the sum of their invocation counts, the hosted-bundle cost dual to the called-subsystem cost of Proposition E.2 — which bounds them in any execution, whoever is driving them. The absorbed environment of Definition 10.10 runs $\mathcal{Z}$ and those functionalities and nothing else, so

$$\mathcal{Z} \in \mathbb{Z}_{\rho[\pi/\varphi],\,\rho}(\mathbf{z}) \qquad \Longrightarrow \qquad \mathcal{Z}^{\rho\setminus\varphi} \in \mathbb{Z}_{\pi,\varphi}\big(\mathbf{z} \oplus \mathbf{c}_{\rho\setminus\varphi}\big),$$

the containment in $\mathbb{Z}_{\pi,\varphi}$ being Proposition 10.11. Feeding this into the theorem, the conclusion is

$$\mathrm{ADV}^{\mathrm{UC}}_{\rho[\pi/\varphi],\,\rho}[\mathbf{a},\mathbf{s},\mathbf{z}] \;\leqslant\; \mathrm{ADV}^{\mathrm{UC}}_{\pi,\varphi}\big[\mathbf{a},\ \mathbf{s},\ \mathbf{z} \oplus \mathbf{c}_{\rho\setminus\varphi}\big].$$

Composition is free in the adversary and in the simulator, and costs the surrounding system once, on the environment side.

Transitivity, numerically. Proposition 10.16 asks that the first step's simulator class be the second step's adversary class, which becomes an inequality between budgets: $\mathbb{S}(\mathbf{s}) \subseteq \mathbb{A}(\mathbf{a}')$ holds as soon as $\mathbf{s} \leqslant \mathbf{a}'$. A simulator one is willing to build must be an adversary the next step is willing to face. The proposition also asks for one class admissible for both steps, and that hypothesis must be carried here: for $\mathbb{Z} \subseteq \mathbb{Z}_{\pi,\varphi} \cap \mathbb{Z}_{\varphi,\psi}$, which by the proof of Proposition 10.16 lies in $\mathbb{Z}_{\pi,\psi}$ as well,

$$\mathrm{ADV}^{\mathrm{UC}}_{\pi,\psi}[\mathbf{a},\mathbf{s}',\mathbb{Z}] \;\leqslant\; \mathrm{ADV}^{\mathrm{UC}}_{\pi,\varphi}[\mathbf{a},\mathbf{s},\mathbb{Z}] \;+\; \mathrm{ADV}^{\mathrm{UC}}_{\varphi,\psi}[\mathbf{s},\mathbf{s}',\mathbb{Z}].$$

Budgets alone will not do here. Written with $\mathbf{z}$ in all three places the brackets would range over $\mathbb{Z}_{\pi,\psi}(\mathbf{z})$, $\mathbb{Z}_{\pi,\varphi}(\mathbf{z})$ and $\mathbb{Z}_{\varphi,\psi}(\mathbf{z})$, and the first is contained in neither of the others — an environment admissible for the outer pair is told nothing about $\mathrm{IDs}(\varphi)$ — so no hypothesis would cover the middle probability that the triangle inequality cancels. The composition display above has no such trouble: absorption maps its one class into the other. Corollary 10.17 is the two displays put together. Along the chain of Theorem C.3 the same two effects accumulate: the errors add, each hop widens the environment budget by the cost of hosting what it absorbs at that step, and the class relations of Remark C.4 become the inequalities $\mathbf{s}_{k+1} \leqslant \mathbf{a}_k$.

Nesting, and whether the simulator blows up. Say that an emulation statement has *overhead* f if it holds against $\big(\mathbb{A}(\mathbf{a}), \mathbb{S}(f(\mathbf{a})), \cdot\big)$ for every adversary budget $\mathbf{a}$: whatever the adversary is given, the simulator one builds costs f of it. In the per-invocation budget the two components behave differently,

$$f(t, r) \;=\; \big(\max(t, \tau),\;\; \gamma r + \delta\big),$$

the query count affine — γ the calls the simulator places for each call of the adversary it runs, δ its own — and the per-invocation time a maximum, τ the simulator's own per-activation work, since one machine runs per activation and the simulator adds only bounded work to each.

Theorem 10.15 does not touch f at all. Its adversary and simulator classes are those of the hypothesis, so a single replacement returns the very machine the hypothesis supplied and the whole price is paid on the environment side. Overheads compose in one place only: where a simulator is re-read as an *adversary*, which is Proposition 10.16. There $\mathcal{S}'$ is built from $\mathcal{S}$, itself built from $\mathcal{A}$, so the composite has overhead $f_2 \circ f_1$. Applying the results k times, the query budget obeys $r_j = \gamma\, r_{j-1} + \delta$, that is

$$r_k \;=\; \gamma^k r_0 \;+\; \delta\,\frac{\gamma^k - 1}{\gamma - 1} \quad (\gamma \neq 1), \qquad r_k \;=\; r_0 + k\delta \quad (\gamma = 1),$$

while the per-invocation time saturates, $t_k = \max(t_0, \tau_1, \ldots, \tau_k)$, bounded by the largest level's own work and constant in depth.

The blow-up therefore lives entirely in the query count, a dichotomy turning on γ alone. A simulator that must place more than one call for some call of the adversary it runs has $\gamma > 1$, and k applications cost γ^k: genuinely exponential, and at $\gamma = 2$ a depth of 20 already costs a factor of 2^{20}, about a million, on the query count. A simulator that relays each call once and adds bounded traffic of its own has $\gamma = 1$, and k applications cost $r_0 + k\delta$: linear, δ paid once per level. This is the property to check when a simulator is built; nothing proved here implies it. It is also why straight-line simulators matter beyond the usual reasons, rewinding being the standard way to acquire $\gamma > 1$: one rewinding simulator anywhere in the tower puts a factor γ on every level below it.

The same formula governs breadth as well as depth. Theorem C.3 builds $\mathcal{S}_n, \ldots, \mathcal{S}_1$ by handing each simulator to the next step as its adversary, so n parallel instances compose n overheads exactly as an n-deep tower does, and Remark C.4 read with budgets asks $\mathbf{a}_k \geqslant f_{k+1}(\mathbf{a}_{k+1})$ at every link. Composing n instances of a $\gamma = 1$ protocol costs $n\delta$ extra queries; composing n instances of a $\gamma = 2$ one is hopeless for even moderate n.

Nothing comparable happens on the environment side, or in the per-invocation time. Each level adds the cost of hosting the shell it absorbs, and a tower's shells are disjoint, so a

depth-k nesting reaches the innermost statement at budget $\mathbf{z} \oplus \bigoplus_{i \leqslant k} \mathbf{c}_i$ — its invocation count that of running the ambient system once, its per-invocation time the largest shell's, however deep the tower. The errors behave likewise, k applications of Proposition 10.16 giving $\sum_i \varepsilon_i$. Where a security claim degrades exponentially under nesting, the simulator's query overhead $\gamma > 1$ is the only possible source: one machine runs per activation, so neither the per-invocation time nor the environment budget can multiply.

Recovering the asymptotic reading. Nothing above mentions a security parameter. To get the familiar statement, index everything by λ, let the budgets be polynomials in λ, and ask that the advantage be negligible in it; the classes then become the polynomially bounded machines and Definition 10.7 the usual definition of UC emulation. The concrete form is the one the composition results are proved in, and the asymptotic form is a corollary of it, not the other way round.

F Responsive Calls

Appendix H needs a promise the framework has so far had no way to write down. When SIGN asks the adversary for a signature it must get one back, and get it back *now*: were the adversary free to go off and drive the rest of the execution before answering, the party would be left suspended mid-signature and locality lost. The text there settles for restricting the simulator class. This appendix says the thing in code instead.

What has to be forbidden. Not that the adversary answer helpfully — it may return anything, and Appendix G deals with a bad answer. What has to be forbidden is the *token* leaving: between receiving the call and returning, the responder must place no call at all. That is a restriction on the calls a machine makes, and the paper already has a wrapper for exactly that.

Definition F.1 (Responsive answering). RESPOND is SILENCE at its maximum: with $\mathcal{I}_{\text{ALL}}$ the set of all identities,

$$\text{RESPOND}\big(\mathtt{id}.\text{OP}(\mathtt{in}) \text{ from } \mathtt{id}'\big) \;:=\; \text{SILENCE}\big(\mathcal{I}_{\text{ALL}},\ \mathtt{id}.\text{OP}(\mathtt{in}) \text{ from } \mathtt{id}'\big).$$

Every claim lies in $\mathcal{I}_{\text{ALL}}$, so by Section 2 every call the wrapped programme issues is refused with REJ *without ever being placed*. The programme may compute as long as it likes and must still return a value; what it cannot do is hand the token on.

Wrapper RESPOND
programme $\mathtt{id}.\text{OP}(\mathtt{in})$ from $\mathtt{id}'$

RESPOND$\big(\mathtt{id}.\text{OP}(\mathtt{in})$ from $\mathtt{id}'\big)$:

```
1: handle id.OP(in) from id' with
2:     return out                          ↦ out
3:     id''.FULLOP(in'') as id''' ; k      ↦ k(REJ)
```

Read as a handler. Written out, RESPOND is SILENCE with its last clause deleted. SILENCE forwards a call whose claim it permits, so its operation clause is partly a re-raise; RESPOND has no re-raise, only a constant. In the vocabulary of Section 2 it *discharges* the ambient call

effect rather than forwarding it: inside its scope the effect is handled locally and completely, so the wrapped programme is pure with respect to it. It is also the definition of *locality*, the property Appendix H wanted: a call on the slot is served within the responder's own activation, the token never leaving it. That is the crisp statement of responsiveness — *a responsive core is one whose call effect does not escape to* EXEC — and the operational reading follows, since EXEC dispatches exactly the calls that do escape. A responsive call is one activation of the responder and no more.

Definition F.2 (Responsive interface, responsive call)**.** The adversary carries, beside FULLA, a second interface FULLA$^!$ with the same guard and the same core but RESPOND in place of SILENCE:

$$\mathtt{id}.\text{FULLA}^!(\mathtt{in}) \text{ from } \mathtt{id}' : \quad \textbf{require } \text{GUARD}_{\mathcal{A}}(\mathtt{id}, \mathtt{id}'); \\ \textbf{return } \text{RESPOND}\big(\mathtt{id}.\mathrm{A}(\mathtt{in}) \text{ from } \mathtt{id}'\big).$$

A *responsive call* is a call on FULLA$^!$. Inside an interface with identity $\mathtt{id}$ we write $\mathcal{A}^!(\mathtt{in})$ for $(\mathrm{A}, \mathtt{id}.\mathrm{P}).\text{FULLA}^!(\mathtt{in})$ as $\mathtt{id}$, as $\mathcal{A}(\mathtt{in})$ abbreviates the ordinary one.

Three things follow, and they are the price.

The responder cannot read the register. **C** is reached by calling FULLSTATUS, and that call is refused, so a responsive answer is computed from what the responder already knows. It cannot delegate: no subroutine, no other party, no environment. And it cannot corrupt, CORRUPT being a call like any other. The last deserves stating on its own, because Section 7 reads the register twice and the gap between the two reads is exactly where a corruption can slip in.

Proposition F.3 (A responsive call cannot corrupt)**.** *In any execution, the register's state* **C** *when a responsive call returns is its state when the call was placed.*

Proof. By Section 3, **C** changes only on line 5 of FULLCORRUPT, and only when that interface is reached by a call. Between the placing and the return of a responsive call the responder holds the token throughout, and by Definition F.1 every call it issues is refused before being placed, so FULLCORRUPT is not reached. No other machine holds the token in that interval, so none can reach it either. □

So if a core reaches the adversary only through $\mathcal{A}^!$, the party it serves is corrupt at line 7 exactly when it was corrupt at line 3, and the second mediation hook cannot fire on account of that call. Remark 7.1 describes what happens when it does; responsiveness is the condition under which it does not.

This is what Appendix H wanted. Writing $\mathcal{A}^!$ in place of $\mathcal{A}$ in GEN, SIGN and VERIFY makes locality a property of the code rather than a condition on the simulator class, and the wrapper enforces it, so it holds of *every* occupant of the adversary slot — the simulator included, Definition 10.3 giving a simulator the adversary's interfaces, FULLA$^!$ among them. What it does not do is make the answer good: a responsive adversary may still return $\perp$, and SAN repairs that. Responsiveness buys the token back; sanitization buys the value.

Remark F.4 (Responsiveness and the dummy)**.** The two appendices pull against each other, and it is worth being explicit about where. The dummy of Appendix D answers a call from a functionality by relaying it to the environment on line 5 — and that is a call, which RESPOND refuses. So $\mathcal{D}$ cannot serve a responsive call: the relay is refused, and the rule of Section 8 delivers its answer as $\perp$ where a real adversary would answer with substance. The regrouping (R1) would therefore fail for a system whose cores use $\mathcal{A}^!$, at the first responsive call — which is why Theorem D.4 hypothesises systems whose cores place none. Recovering completeness for responsive systems needs a matching notion on the environment's side, so that the relayed question can be answered without the token moving on; we leave that open.

G Sanitization

We will often need to *sanitize* simulator outputs, so attacks do not affect ideal functionalities too adversely. In honest-majority settings, for instance, we typically sanitize a $\perp$ output to a non-$\perp$ one to preserve liveness. We write such a procedure SAN[CLEAN]$(v; c)$, where CLEAN is a predicate on a value together with whatever context it consults, and replace a simulator output v by $v' \leftarrow$ SAN[CLEAN]$(v; c)$, which operates as below unless stated otherwise. The context c after the semicolon lists the functionality state the predicate reads; it is read-only, and different predicates take different contexts.

Sanitizer SAN[CLEAN]
predicate CLEAN, value v, context c

SAN[CLEAN]$(v; c)$:
1: **if** CLEAN$(v; c)$ **then**
2: **return** v
3: pick $v' \in \{ v' : \text{CLEAN}(v'; c) \}$
4: **return** v'

So sanitization acts only when CLEAN$(v; c)$ fails, and what it returns then satisfies CLEAN.[4] Our sanitizers need no more than *read-only* access to the functionality state. How line 3 picks is part of the specification — the picked value reaches the caller, so different rules are observably different functionalities — and we fix the lexicographically first good value throughout. The choice matters less than it looks: the pick surfaces only when the slot's own answer fails CLEAN, and an adversary preferring some particular good value may simply submit it.

H Digital Signatures

H.1 Functionality

$\mathcal{F}_{\mathsf{Sig}}$ below idealizes digital signatures. Its process id is a parameter and not a constant. The name component is fixed — that is what the subscript records, by the naming convention

[4] A good value exists whenever the space CLEAN draws from is infinite and everything it excludes beyond non-membership is a value named in the context c. Both halves are needed for the predicates of Appendix H: membership in an infinite $\mathcal{K}$ or Σ is the first conjunct of each, and everything else they exclude is an entry of a table passed in c. Under Definition 11.1 the context is finitely supported in any execution — a bounded run records finitely many entries — so the excluded set is finite inside an infinite space.

of Section 1 — but the session and instance are not, so one system may hold a copy per session and several within a session, the situation composition exists to handle, and one Definition 1.2 would forbid outright if the whole process id were pinned. The code below reads neither component, so Appendix B lets it be read at any other — for an $\mathbf{N}$ meeting the condition of Definition B.2, which any global or purely local one does. Recall from Section 6 that $\mathcal{A}(x)$ abbreviates a call to the adversary on behalf of the party being served; in the ideal execution the simulator answers those calls, and every one of them here is sanitized in the sense of Appendix G before use.

How we write ideal functionalities. The style is the same throughout. Wherever a value has to be produced and the specification does not pin it down, we let the simulator choose it and sanitize the choice on the fly. The simulator is then meant to have as much freedom as the security goal permits and no more — a design intention, not a theorem — and the goal stays visible in the predicates rather than buried in the code.

In $\mathcal{F}_{\mathsf{Sig}}$ this happens three times, and the first two protect correctness. GEN takes the verification key from the adversary, and $\textsc{Clean}_{vk}$ makes it an actual key, distinct from every key already issued and free of any signature already recorded valid under it, so generation always succeeds and keys neither collide nor arrive pre-forged. SIGN takes the signature, and $\textsc{Clean}_{\sigma}$ makes it an actual signature not already recorded invalid, so signing succeeds whenever a key has been generated, and what it produces verifies. Locality asks for that, and asks in addition that the simulator answer without passing the token elsewhere, a condition on the simulator class rather than a property of the code. VERIFY is sanitized for unforgeability instead: a verdict outside $\{0, 1\}$ becomes 0, and so does any verdict of 1 on a fresh triple under a key an honest party generated, which makes signatures *strongly* unforgeable. There the good value is unique, so the sanitization is written inline rather than through SAN. Verification keys are not assumed authenticated, so VERIFY takes a vk of its own.

LEAK, which Section 4 requires of an ideal functionality, hands the adversary the party's verification key together with the signatures recorded valid under it, and nothing else of the state. INITIALIZE fixes the two tables and their types: `VK` holds one key per served party and `Ver` the verdict on each triple of key, message and signature, both starting everywhere at $\square$, the value that separates "not yet decided" from a recorded 0 or 1. Both $\mathcal{K}$ and Σ are infinite and contain neither $\perp$ nor $\square$: infinite so that the sanitizers never run out of good values, and clean of the two markers so that membership already refuses them.

Where locality is wanted, the simulator class must be restricted to the *responsive* ones: those answering each simulator call without placing any call of their own, so the token returns at once and simulator calls are subroutine rather than coroutine calls — the discipline Appendix F enforces in code. For non-interactive, stateless signatures this costs nothing, the usual simulators being responsive already.

One further line in GEN and VERIFY guards against re-entrance. Section 8 lets a suspended instance be re-entered, and both operations suspend at an adversary call between testing a table entry and writing it; each therefore re-tests the entry on resumption, and a value recorded meanwhile wins. Without the re-test two nested calls could issue two keys to one party, or return different verdicts on one triple — the very consistency `Ver` exists to record.

Functionality $\mathcal{F}_{\mathsf{Sig}}$
pid, **P**, **N**, **U** $:= \{(\mathcal{A}, \textsc{serves})\}$, *par* $:= \bot$

INITIALIZE():
1: $\mathtt{VK} : \mathcal{F}_{\mathsf{Sig}}.\mathbf{P} \to \mathcal{K} \cup \{\square\}$
2: $\mathtt{VK}[*] \leftarrow \square$
3: $\mathtt{Ver} : \mathcal{K} \times \mathcal{M} \times \Sigma \to \{0,1\} \cup \{\square\}$
4: $\mathtt{Ver}[*,*,*] \leftarrow \square$

id.GEN() from id′
5: **if** $\mathtt{VK}[\mathrm{id.P}] \neq \square$ **then**
6: **return** $\mathtt{VK}[\mathrm{id.P}]$
7: $vk \leftarrow \mathcal{A}(\mathrm{id.GEN})$
8: **if** $\mathtt{VK}[\mathrm{id.P}] \neq \square$ **then**
9: **return** $\mathtt{VK}[\mathrm{id.P}]$ *// recorded while suspended*
10: $vk \leftarrow \textsc{San}[\textsc{Clean}_{vk}](vk; \mathtt{VK}, \mathtt{Ver})$
11: $\mathtt{VK}[\mathrm{id.P}] \leftarrow vk$
12: **return** vk

id.SIGN(*msg*) from id′
13: $vk \leftarrow \mathtt{VK}[\mathrm{id.P}]$
14: **if** $vk = \square$ **then**
15: **return** $\bot$
16: $\sigma \leftarrow \mathcal{A}(\mathrm{id.SIGN}, msg)$
17: $\sigma \leftarrow \textsc{San}[\textsc{Clean}_{\sigma}](\sigma; msg, vk, \mathtt{Ver})$
18: $\mathtt{Ver}[vk, msg, \sigma] \leftarrow 1$
19: **return** σ

id.VERIFY(vk, msg, σ) from id′
20: **if** $\mathtt{Ver}[vk, msg, \sigma] \neq \square$ **then**
21: **return** $\mathtt{Ver}[vk, msg, \sigma]$
22: $\mathrm{b} \leftarrow \mathcal{A}(\mathrm{id.VERIFY}, vk, msg, \sigma)$
23: **if** $\mathtt{Ver}[vk, msg, \sigma] \neq \square$ **then**
24: **return** $\mathtt{Ver}[vk, msg, \sigma]$ *// recorded while suspended*
25: **if** $\mathrm{b} \notin \{0,1\}$ **then**
26: $\mathrm{b} \leftarrow 0$
27: **if** $\exists\, \mathrm{P}' \notin \mathbf{C} : vk = \mathtt{VK}[\mathrm{P}']$ **then**
28: $\mathrm{b} \leftarrow 0$ *// no forgery under an honest key*
29: $\mathtt{Ver}[vk, msg, \sigma] \leftarrow \mathrm{b}$
30: **return** b

id.LEAK() from id′
31: **if** $\mathtt{VK}[\mathrm{id.P}] = \square$ **then**
32: **return** $(\bot, \varnothing)$
33: **return** $\big(\mathtt{VK}[\mathrm{id.P}], \{\,(msg, \sigma) : \mathtt{Ver}[\mathtt{VK}[\mathrm{id.P}], msg, \sigma] = 1\,\}\big)$

$\textsc{Clean}_{vk}(vk; \mathtt{VK}, \mathtt{Ver})$:
34: **return** $vk \in \mathcal{K} \;\wedge\; \neg\exists\, \mathrm{P} : \mathtt{VK}[\mathrm{P}] = vk$
35: $\qquad\qquad \wedge\; \neg\exists\, msg, \sigma : \mathtt{Ver}[vk, msg, \sigma] = 1$

$\textsc{Clean}_{\sigma}(\sigma; msg, vk, \mathtt{Ver})$:
36: **return** $\sigma \in \Sigma \wedge \mathtt{Ver}[vk, msg, \sigma] \neq 0$

H.2 Properties

Definition 12.1 is a shape, and a shape is worth an instance. We give two games over $\mathcal{F}_{\mathsf{Sig}}$: $\mathcal{G}\mathrm{m}_{\mathrm{COR}}$, saying that what the signing interface produces verifies, and $\mathcal{G}\mathrm{m}_{\mathrm{UF}}$, saying that nothing else does. They differ in FINALIZE alone, so we write the shell once and let $\mathcal{G}\mathrm{m}$ stand for either where the shell is all that matters.

Both are played over the game system $\gamma := \{\mathcal{G}\mathrm{m}\} \cup \{\mathcal{F}_{\mathsf{Sig}}\}$, one for each game, with $\mathcal{F}_{\mathsf{Sig}}.\mathbf{N} := \{\mathcal{G}\mathrm{m}.pid\}$ — the game's process id, pinned, not its name — and $\mathcal{F}_{\mathsf{Sig}}.\mathbf{P} \subseteq \mathcal{G}\mathrm{m}.\mathbf{P}$, so the relays exist at every party $\mathcal{F}_{\mathsf{Sig}}$ serves and, at the root, address a party it does not serve and are answered REJ at line 1 — the root being where the game finalizes, not where it plays. Inside an interface at id we write $\mathrm{id}_{\mathcal{F}_{\mathsf{Sig}}}$ for $(\mathcal{F}_{\mathsf{Sig}}.pid, \mathrm{id.P})$, the interface of $\mathcal{F}_{\mathsf{Sig}}$ at the party the game was called at. That is tightness at $\{\mathcal{G}\mathrm{m}\}$, which Definition 12.1 requires, and here it earns its keep twice over. It makes γ a legal game system; and it makes the trace *complete*. No caller but $\mathcal{G}\mathrm{m}$ passes line 2 at $\mathcal{F}_{\mathsf{Sig}}$: not the environment, whose Z-identities and fresh names lie outside a pinned **N**; not the adversary, whose claim is an A-identity; and not a machine the environment hosts at an invented instance of the game's name, which is what pinning the id rather than the name refuses. So every signature $\mathcal{F}_{\mathsf{Sig}}$ ever issues passes through line 8 below and is recorded. A game whose verdict is "this was not obtained from the signing interface" has no other way to know.

Game shell over $\mathcal{F}_{\mathsf{Sig}}$, and the FINALIZE of $\mathcal{G}\mathrm{m}_{\mathrm{COR}}$
pid $:= (\mathcal{G}\mathrm{m}.\mathsf{F}, 0, 0)$, $\mathbf{P} := \mathcal{F}_{\mathsf{Sig}}.\mathbf{P} \cup \{\mathsf{Z}\}$, $\mathbf{N} := \mathbf{Z}$, $\mathbf{U} := \{(\mathcal{F}_{\mathsf{Sig}}, \mathrm{SERVES}_{\mathcal{G}\mathrm{m}})\}$, *par* $:= \bot$

INITIALIZE():
DONE $\leftarrow$ FALSE
$\mathtt{tr} \leftarrow (\,)$ // *the trace, a sequence*

id.GEN() from id′
require ¬DONE
$vk \leftarrow \mathrm{id}_{\mathcal{F}_{\mathsf{Sig}}}$.FULLGEN() as id
$\mathtt{tr} \leftarrow \mathtt{tr} \cdot (\mathrm{GEN}, \mathrm{id}.\mathsf{P}, vk)$
return vk

id.SIGN(msg) from id′
require ¬DONE
$\sigma \leftarrow \mathrm{id}_{\mathcal{F}_{\mathsf{Sig}}}$.FULLSIGN($msg$) as id
$\mathtt{tr} \leftarrow \mathtt{tr} \cdot (\mathrm{SIGN}, \mathrm{id}.\mathsf{P}, msg, \sigma)$
return σ

id.VERIFY(vk, msg, σ) from id′
require ¬DONE
$\mathsf{b} \leftarrow \mathrm{id}_{\mathcal{F}_{\mathsf{Sig}}}$.FULLVERIFY($vk, msg, \sigma$) as id
$\mathtt{tr} \leftarrow \mathtt{tr} \cdot (\mathrm{VER}, \mathrm{id}.\mathsf{P}, vk, msg, \sigma, \mathsf{b})$
return b

id.LEAK() from id′
return $\bot$

id.FINALIZE() from id′ (correctness)
require ¬DONE
require $\mathrm{id}.\mathsf{P} = \mathsf{Z}$
DONE $\leftarrow$ TRUE
$\mathbf{C} \leftarrow (\mathsf{C}, \mathsf{Z})$.FULLSTATUS() as id
$w \leftarrow 1$ if $\exists\, \mathsf{P}, \mathsf{Q}, vk, msg, \sigma$:
  $\mathsf{P} \notin \mathbf{C} \wedge \mathsf{Q} \notin \mathbf{C} \wedge \sigma \in \Sigma$
  $\wedge\ (\mathrm{GEN}, \mathsf{P}, vk) \in \mathtt{tr}$
  $\wedge\ (\mathrm{SIGN}, \mathsf{P}, msg, \sigma) \in \mathtt{tr}$
  $\wedge\ (\mathrm{VER}, \mathsf{Q}, vk, msg, \sigma, 0) \in \mathtt{tr}$;
else $w \leftarrow 0$
return w

The FINALIZE of $\mathcal{G}\mathrm{m}_{\mathrm{UF}}$; the shell above is unchanged

id.FULLFINALIZE() from id′
require ¬DONE
require $\mathrm{id}.\mathsf{P} = \mathsf{Z}$
DONE $\leftarrow$ TRUE
$\mathbf{C} \leftarrow (\mathsf{C}, \mathsf{Z})$.FULLSTATUS() as id
$w \leftarrow 1$ if $\exists\, \mathsf{P}, \mathsf{Q}, vk, msg, \sigma$: $\mathsf{P} \notin \mathbf{C} \wedge \mathsf{Q} \notin \mathbf{C}$
  $\wedge\ (\mathrm{GEN}, \mathsf{P}, vk) \in \mathtt{tr} \wedge (\mathrm{VER}, \mathsf{Q}, vk, msg, \sigma, 1) \in \mathtt{tr} \wedge (\mathrm{SIGN}, \mathsf{P}, msg, \sigma) \notin \mathtt{tr}$;
else $w \leftarrow 0$
return w

Four things about the code. The trace is a sequence, $\in$ means occurrence in it, and no verdict reads the order. The verdicts read tr and the register and call nothing, which is why the root exemption in $\mathrm{SERVES}_{\mathcal{G}\mathrm{m}}$ suffices and $\mathcal{F}_{\mathsf{Sig}}$ need not serve Z. Both ask $\mathsf{P} \notin \mathbf{C}$ at the end rather than at the time of signing, and monotonicity (Section 7) makes that the stronger reading: a party honest when the game closes was honest throughout. Correctness asks $\sigma \in \Sigma$ because SIGN answers $\bot$ where no key has been generated, and $\bot$ is no signature to hold against the functionality. And unforgeability is *strong*: line 5 excludes the triple, not the message, so a second signature on a signed message counts as a forgery — which is the notion Appendix H's sanitizer was written to give.

Proposition H.1 ($\mathcal{F}_{\mathsf{Sig}}$ has both properties). *For $\mathcal{G}\mathrm{m}$ either of $\mathcal{G}\mathrm{m}_{\mathrm{COR}}$ and $\mathcal{G}\mathrm{m}_{\mathrm{UF}}$, for every $\mathcal{Z} \in \mathbb{Z}_{\gamma,\gamma}$ and every occupant $\mathcal{X}$ of the adversary slot, $\Pr[\mathrm{WIN}] = 0$. So $\{\mathcal{F}_{\mathsf{Sig}}\}$ has each property within 0 against $(\mathbb{Z}_{\gamma,\gamma}, \mathbb{X})$ in the sense of Definition 12.8, for $\mathbb{X}$ the class of all occupants.*

Proof. Take parties honest at the close. Monotonicity makes them honest throughout, so no call the game places at them is mediated, line 4 firing only at a corrupt party and the game's own name lying in $\{0,1\}^*$. Both verdicts ask this of the signer P *and* of the verifier Q, and both need it of both: at a corrupt party the core does not run and the answer is the adversary's, so a mediated VERIFY reports whatever it likes — a 0 on a good signature, or a 1 on a forgery — and either game would be won with $\mathcal{F}_{\mathsf{Sig}}$ having done nothing wrong.

For correctness, suppose $(\mathrm{SIGN}, \mathsf{P}, msg, \sigma) \in \mathtt{tr}$ with $\sigma \in \Sigma$. The core of SIGN ran, so it sanitized σ against CLEAN_σ and then set $\mathtt{Ver}[vk, msg, \sigma] \leftarrow 1$ for $vk = \mathtt{VK}[\mathsf{P}]$, which GEN fixed once and never changes. Any later $\mathrm{VERIFY}(vk, msg, \sigma)$ finds that entry set and returns it at its first line, so it returns 1; thus $(\mathrm{VER}, \mathsf{Q}, vk, msg, \sigma, 0) \notin \mathtt{tr}$ for every Q, and line 20 gives 0.

For unforgeability, suppose $(\mathrm{VER}, \mathsf{Q}, vk, msg, \sigma, 1) \in \mathtt{tr}$ with $vk = \mathtt{VK}[\mathsf{P}]$ for an honest P. Either VERIFY read a recorded entry or it computed one. It cannot have computed a 1: the last test before writing $\mathtt{Ver}$ sets $\mathsf{b} \leftarrow 0$ whenever $vk = \mathtt{VK}[\mathsf{P}']$ for some $\mathsf{P}' \notin \mathbf{C}$, which P witnesses. So the entry was recorded, and only SIGN records a 1 — at the party whose key is vk, that party being P since CLEAN_{vk} issues no key twice. Hence $(\mathrm{SIGN}, \mathsf{P}, msg, \sigma) \in \mathtt{tr}$ and line 5 gives 0. □

The bound is not the interesting part; that it is 0 merely says the ideal functionality was written to have these properties, as an ideal functionality should be. The content is what Corollary 12.11 then does with it, and the useful form is the one on probabilities: a system π put in place of $\{\mathcal{F}_{\mathsf{Sig}}\}$ wins either game with probability at most ε, against every adversary and not only the dummy. Read as advantages the same conclusion says only 2ε, which for a game meant to be unwinnable is no constraint at all — a forger succeeding half the time meets it. That is the calibration warning of Section 12 arriving where it bites, and why Proposition 12.10 carries both displays.

Four conditions come with the corollary and none is discharged by emulation alone. The replacement of $\{\mathcal{F}_{\mathsf{Sig}}\}$ by π must be well-formed; $\{\mathcal{G}\mathrm{m}\} \cup \pi$ must again be a game system, which asks π's members to pin $\mathcal{G}\mathrm{m}.pid$ where $\mathcal{F}_{\mathsf{Sig}}$ did, since tightness is part of Definition 12.1; the simulator and the adversary must be refusal-oblivious; and the environments must silence no Z-identity. A realization is thus never separately argued to be correct or unforgeable — it inherits both from the one emulation proof, together with those four checks, and the games above are what the inheritance is *of*.

One caveat travels with it, and this is where it bites. Corollary 12.11 asks that the cores of the game system place no responsive call, and $\mathcal{F}_{\mathsf{Sig}}$ as written reaches the slot through $\mathcal{A}(\cdot)$, so the corollary applies. Should one adopt Appendix F's advice and write $\mathcal{A}^!$ in GEN, SIGN and VERIFY — which buys locality — the transfer must be argued directly instead, for the reason Remark F.4 gives. Locality and inheritance are, as things stand, priced against each other.

I Public-Key Infrastructure

I.1 Functionality

$\mathcal{F}_{\mathsf{Sig}}$ said plainly that it does not authenticate the key it hands out — VERIFY takes a vk of its own, unconnected to any caller, because verification keys are not assumed authenticated (Appendix H.1). $\mathcal{G}_{\mathsf{PKI}}$ below is the setup that closes the gap: a single, shared directory binding a key to the party that claims it, so that a vk handed to VERIFY can be trusted because it came from here rather than because it was simply believed.

What it does is quickly said. Each party may register one key, and only one: REGISTER takes the key as *input* — unlike GEN, which conjures one from the slot, this is a directory and not a factory — and keeps the first key it is given, sanitized against every key already on file so that no two parties ever hold the same one. RETRIEVE answers anyone with whatever is on file for the party they name, or nothing if no one has registered.

The fields. They are $\mathcal{G}_{\mathsf{Clock}}$'s, and for the same reason. The process id is a constant, $(\mathrm{GPKI}, 0, 0)$, so Definition 1.2 caps the system at one copy: a second directory would be a second notion of who owns which key, exactly what a PKI exists to rule out. $\mathbf{N} := \mathbf{S} \cup \mathbf{A} \cup \mathbf{Z}$ makes $\mathrm{GLOBAL}(\mathcal{G}_{\mathsf{PKI}})$ hold by the same disjunct as the clock's, and $\mathbf{P}$ is again the universe the directory covers rather than who has used it — registration is state, in $\mathtt{VK}$, not membership. One field is simpler than the clock's: $\mathbf{U} := \varnothing$. REGISTER and RETRIEVE place no call at all, the key arriving with the request rather than from the slot, so there is nothing $\mathcal{G}_{\mathsf{PKI}}$ depends on beyond the ambient machinery every functionality already has.

Two borrowed guards. REGISTER takes one line from each of the two examples before it. The first is $\mathcal{G}_{\mathsf{Clock}}$'s: a caller named A or Z acting for an *honest* party is answered that party's own key and changes nothing, exactly as it is answered the time rather than allowed to tick on an honest party's behalf — an outsider may ask what is on file, not decide what goes there. At a corrupt party the check does not fire, so the call proceeds — the adversary registers on a corrupt party's behalf as freely as it ticks one, and whatever key it submits is what the directory will show, mediation letting exactly this call through unmediated as it does for $\mathcal{F}_{\mathsf{AC}}$'s corrupt sends (Appendix L.1). The second is $\mathcal{F}_{\mathsf{Sig}}$'s: the first call to succeed wins, later ones returning the key already on file rather than overwriting it, exactly as GEN never re-issues a key once one exists. Sanitizing here differs from both signature predicates in what it repairs: CLEAN_{vk} and CLEAN_{σ} each fix a value the *slot* proposed; $\mathrm{CLEAN}_{\mathrm{REG}}$ fixes one the *caller* supplied, which is the more exposed position — an honest party's own key is presumably already good, but a corrupt one's is whatever the adversary wrote in, and the one thing a directory cannot let through is a second party claiming a key the first already holds. A good value always exists here for the same reason it does for CLEAN_{vk}: $\mathcal{K}$ is infinite and the keys excluded are exactly those a bounded run can have recorded, finitely many (footnote of Appendix G).

Functionality $\mathcal{G}_{\mathsf{PKI}}$
pid $:= (\mathrm{GPKI}, 0, 0)$, $\mathbf{P}$, $\mathbf{N} := \mathbf{S} \cup \mathbf{A} \cup \mathbf{Z}$, $\mathbf{U} := \varnothing$, *par* $:= \bot$

INITIALIZE():
1: $\mathtt{VK} : \mathcal{G}_{\mathsf{PKI}}.\mathbf{P} \to \mathcal{K} \cup \{\square\}$
2: $\mathtt{VK}[*] \leftarrow \square$

id.REGISTER(*vk*) from id′
3: **if** $\mathtt{id}'.\mathrm{F} \in \{\mathrm{A}, \mathrm{Z}\} \wedge \mathtt{id}.\mathrm{P} \notin \mathbf{C}$ **then**
4: **return** $\mathtt{VK}[\mathtt{id}.\mathrm{P}]$ *// a peek, not a write*
5: **if** $\mathtt{VK}[\mathtt{id}.\mathrm{P}] \neq \square$ **then**
6: **return** $\mathtt{VK}[\mathtt{id}.\mathrm{P}]$
7: $vk \leftarrow \textsc{San}[\textsc{Clean}_{\mathrm{REG}}](vk; \mathtt{VK})$
8: $\mathtt{VK}[\mathtt{id}.\mathrm{P}] \leftarrow vk$
9: **return** *vk*

id.RETRIEVE(P) from id′
10: **return** $\mathtt{VK}[\mathrm{P}]$

id.LEAK() from id′
11: **return** VK

$\textsc{Clean}_{\mathrm{REG}}(vk; \mathtt{VK})$:
12: **return** $vk \in \mathcal{K} \wedge \neg\exists\, \mathrm{P} : \mathtt{VK}[\mathrm{P}] = vk$

Three remarks on the code. Registration is recorded against the party, not the caller, for the same reason ticks are: GUARD has already tied the call to id.P by line 2, so whichever machine holds the token when P registers, the key lands on P's own entry. There is no DEREGISTER, and none is added: a real PKI can support revocation, but a key once bound is the simplest directory there is, and revocation is a genuine addition left for elsewhere rather than a gap papered over. And LEAK answers with the whole table, which is no more than RETRIEVE already gives out one entry at a time — the directory is public at every one of its interfaces, not only this one, so LEAK tells the adversary nothing RETRIEVE, asked enough times, would not.

Set against both examples before it, $\mathcal{G}_{\mathsf{PKI}}$ needs less than either. No value is the simulator's to choose out of nothing, as in GEN — the key arrives with the call — so sanitizing here repairs a caller's input rather than a slot's output, the one place this differs from every SAN before it. And no call leaves the core at all, so there is no notification to be responsive about and no price for Appendix F to settle: the two properties below transfer as cheaply as a property can.

I.2 Properties

$\mathcal{G}_{\mathsf{Clock}}$ earned two games from having two kinds of promise — a local one, that the counter moves as coded, and a global one, that it moves only when everyone honest has agreed. $\mathcal{G}_{\mathsf{PKI}}$ splits the same way. $\mathcal{G}\mathfrak{m}_{\mathrm{FIX}}$, *fixity*, says a registered key never changes — the local promise, about one party's own entry. $\mathcal{G}\mathfrak{m}_{\mathrm{BIND}}$, *binding*, says no two parties ever hold the same key — the global one, quantified over all of $\mathcal{G}_{\mathsf{PKI}}.\mathbf{P}$ at once. They differ in FINALIZE alone, so the shell is written once.

Both are played over the game system $\gamma := \{\mathcal{G}\mathfrak{m}\} \cup \{\mathcal{G}_{\mathsf{PKI}}\}$, with $\mathcal{G}_{\mathsf{PKI}}$ taken as in Appendix I.1 save one field: $\mathbf{N} := \{\mathcal{G}\mathfrak{m}.pid\}$, pinned — Remark A.5's forcing, exactly as for the clock (Appendix J.2), and for the identical reason: a property of a global $\mathcal{F}$ is a property against an adversary that shares it, and fixity and binding are both claims about *every* key there is, certifiable only where the game's trace is all there is.

Pinned, the directory is no longer global, so line 3 asks for one session and the experiment's own session 0 meets it. No caller but $\mathcal{G}\mathfrak{m}$ passes line 2: not the environment, not a machine it hosts, and not the adversary's A-identities, refused before line 4 could even apply. So every key $\mathcal{G}_{\mathsf{PKI}}$ ever holds enters through the relay below and is recorded — a

game whose verdict is "two parties share a key" has no other way to know it. Inside an interface at id we write $\mathtt{id}_{\mathcal{G}_{\mathsf{PKI}}}$ for $(\mathcal{G}_{\mathsf{PKI}}.pid, \mathtt{id}.\mathrm{P})$.

Game shell over $\mathcal{G}_{\mathsf{PKI}}$, and the FINALIZE of $\mathcal{G}\mathrm{m}_{\mathbf{FIX}}$
$pid := (\mathcal{G}\mathrm{m}.\mathrm{F}, 0, 0)$, $\mathbf{P} := \mathcal{G}_{\mathsf{PKI}}.\mathbf{P} \cup \{\mathrm{Z}\}$, $\mathbf{N} := \mathbf{Z}$, $\mathbf{U} := \{(\mathcal{G}_{\mathsf{PKI}}, \textsc{serves}_{\mathcal{G}\mathrm{m}})\}$, $par := \bot$

INITIALIZE():
1: DONE $\leftarrow$ FALSE
2: $\mathtt{tr} \leftarrow (\,)$ *// the trace, a sequence*

id.REGISTER(vk) from $\mathtt{id}'$
3: **require** $\neg$DONE
4: **require** $\mathtt{id}.\mathrm{P} \neq \mathrm{Z}$
5: $vk' \leftarrow \mathtt{id}_{\mathcal{G}_{\mathsf{PKI}}}$.FULLREGISTER($vk$) as id
6: $\mathtt{tr} \leftarrow \mathtt{tr} \cdot (\mathrm{REG}, \mathtt{id}.\mathrm{P}, vk')$
7: **return** vk'

id.LEAK() from $\mathtt{id}'$
8: **return** $\bot$

id.RETRIEVE(P) from $\mathtt{id}'$
9: **require** $\neg$DONE
10: **require** $\mathtt{id}.\mathrm{P} \neq \mathrm{Z}$
11: $vk' \leftarrow \mathtt{id}_{\mathcal{G}_{\mathsf{PKI}}}$.FULLRETRIEVE(P) as id
12: **return** vk'

id.FINALIZE() from $\mathtt{id}'$ (fixity)
13: **require** $\neg$DONE
14: **require** $\mathtt{id}.\mathrm{P} = \mathrm{Z}$
15: DONE $\leftarrow$ TRUE
16: $\mathbf{C} \leftarrow (\mathrm{C}, \mathrm{Z})$.FULLSTATUS() as id
17: $w \leftarrow 1$ if $\exists\, \mathrm{P}, vk_1, vk_2$:
18: $\quad \mathrm{P} \notin \mathbf{C} \wedge vk_1 \neq vk_2$
19: $\quad \wedge\ (\mathrm{REG}, \mathrm{P}, vk_1) \in \mathtt{tr} \wedge (\mathrm{REG}, \mathrm{P}, vk_2) \in \mathtt{tr}$;
20: else $w \leftarrow 0$
21: **return** w

The FINALIZE of $\mathcal{G}\mathrm{m}_{\mathbf{BIND}}$; the shell above is unchanged

id.FULLFINALIZE() from $\mathtt{id}'$
1: **require** $\neg$DONE
2: **require** $\mathtt{id}.\mathrm{P} = \mathrm{Z}$
3: DONE $\leftarrow$ TRUE
4: $\mathbf{C} \leftarrow (\mathrm{C}, \mathrm{Z})$.FULLSTATUS() as id
5: $w \leftarrow 1$ if $\exists\, \mathrm{P}, \mathrm{Q}, vk$:
6: $\quad \mathrm{P} \notin \mathbf{C} \wedge \mathrm{Q} \notin \mathbf{C} \wedge \mathrm{P} \neq \mathrm{Q}$
7: $\quad \wedge\ (\mathrm{REG}, \mathrm{P}, vk) \in \mathtt{tr} \wedge (\mathrm{REG}, \mathrm{Q}, vk) \in \mathtt{tr}$;
8: else $w \leftarrow 0$
9: **return** w

Two things about the code. RETRIEVE is relayed and logged though neither verdict reads its trace: a game that could not look anyone's key up would not be exercising $\mathcal{G}_{\mathsf{PKI}}$ at all, and leaving the interface whole costs nothing, exactly as $\mathcal{G}_{\mathsf{Clock}}$'s shell relayed READ for games that read only the mutators. And fixity's verdict does not need vk_1 and vk_2 ordered, unlike steadiness's reading of its trace: idempotence means at most one of the two calls that produced them can be the write, the other only a later read of it, and either order loses the game identically, a changed key being a changed key whichever one came first.

Proposition I.1 ($\mathcal{G}_{\mathsf{PKI}}$ has fixity). *For every $\mathcal{Z} \in \mathbb{Z}_{\gamma,\gamma}$ and every occupant $\mathcal{X}$ of the adversary slot,* $\Pr[\textsc{Win}] = 0$. *So $\{\mathcal{G}_{\mathsf{PKI}}\}$ has fixity within 0 against $(\mathbb{Z}_{\gamma,\gamma}, \mathbb{X})$ in the sense of Definition 12.8, for $\mathbb{X}$ the class of all occupants.*

Proof. Suppose $(\text{REG}, \mathsf{P}, vk_1), (\text{REG}, \mathsf{P}, vk_2) \in \mathtt{tr}$ with $\mathsf{P} \notin \mathbf{C}$. By tightness both were logged at line 5, each running $\mathcal{G}_{\mathsf{PKI}}$.REGISTER's own core at $\mathtt{id}.\mathsf{P} = \mathsf{P}$: the relay's own name is $\mathcal{G}\mathfrak{m}$'s, not A or Z, so line 4 never intercepts it. The core's only write is line 8, reached only while $\mathtt{VK}[\mathsf{P}] = \square$, and once reached it fixes $\mathtt{VK}[\mathsf{P}]$ to some sanitized value for good — every later call at P finds $\mathtt{VK}[\mathsf{P}] \neq \square$ and returns line 6's value, the same one, unchanged. So the two calls producing vk_1 and vk_2 agree: whichever ran first wrote the value, and the other only read it back. Line 17 gives 0. □

Proposition I.2 ($\mathcal{G}_{\mathsf{PKI}}$ has binding). *For every $\mathcal{Z} \in \mathbb{Z}_{\gamma,\gamma}$ and every occupant $\mathcal{X}$ of the adversary slot,* $\Pr[\text{WIN}] = 0$. *So $\{\mathcal{G}_{\mathsf{PKI}}\}$ has binding within 0 against $(\mathbb{Z}_{\gamma,\gamma}, \mathbb{X})$ in the sense of Definition 12.8, for $\mathbb{X}$ the class of all occupants.*

Proof. Suppose $(\text{REG}, \mathsf{P}, vk), (\text{REG}, \mathsf{Q}, vk) \in \mathtt{tr}$ with $\mathsf{P}, \mathsf{Q} \notin \mathbf{C}$ and $\mathsf{P} \neq \mathsf{Q}$. By Proposition I.1's argument $vk = \mathtt{VK}[\mathsf{P}]$ and $vk = \mathtt{VK}[\mathsf{Q}]$ from the moment each was first written onward. Say Q's write came second, the case P's symmetric; at that moment line 8 ran $vk \leftarrow \text{SAN}[\text{CLEAN}_{\text{REG}}](\cdot; \mathtt{VK})$ with $\mathtt{VK}[\mathsf{P}]$ already set, and SAN's guarantee (Appendix G) is that its output satisfies $\text{CLEAN}_{\text{REG}}$ whatever the input — membership in $\mathcal{K}$ and freshness against every key already on file, P's among them. So Q's write cannot equal $\mathtt{VK}[\mathsf{P}]$, contradicting $vk = \mathtt{VK}[\mathsf{P}] = \mathtt{VK}[\mathsf{Q}]$. Line 5 gives 0. □

Both transfer by Corollary 12.11 with nothing extra to argue. The hypothesis it asks — that the game system's cores place no responsive call — is met the cheap way: $\mathcal{G}_{\mathsf{PKI}}$'s cores place no call of any kind, responsive or not, so Remark F.4's caveat never arises and dummy completeness (Appendix D) is untouched. Unlike $\mathcal{G}_{\mathsf{Clock}}$, whose notification bought locality at inheritance's expense, and unlike $\mathcal{F}_{\mathsf{Net}}$ and $\mathcal{F}_{\mathsf{AC}}$, whose slot calls priced their own transfer, $\mathcal{G}_{\mathsf{PKI}}$ pays nothing: a system π put in place of $\{\mathcal{G}_{\mathsf{PKI}}\}$ inherits fixity and binding directly, the four conditions of Appendix H.2's closing paragraph being the only ones left to discharge. A directory that asks the adversary nothing has nothing to spend.

J Global Clock

J.1 Functionality

$\mathcal{F}_{\mathsf{Sig}}$ is local and randomized, and most of what was said about it concerned the simulator. The second example is its opposite on both counts. A clock functionality was introduced by Katz, Maurer, Tackmann and Zikas to model synchrony in UC [3]; recast by Badertscher, Maurer, Tschudi and Zikas as a *shared* setup around a monotone counter go [1], it has become the canonical global functionality, the one the composable analyses of Bitcoin and its successors keep time by. $\mathcal{G}_{\mathsf{Clock}}$ below is that functionality in the language of this paper.

What it does is quickly said. The counter `now` starts at 0 and only ever steps forward. Anyone the clock serves may read it. Parties enter and leave the round structure through REGISTER and DEREGISTER, and the counter steps when every honest registered party has asked it to: UPDATE records a tick against the calling party, and once the honest registered ticks are all in, the test on line 17 advances the counter and clears them. A round ends exactly when the last honest party in it is done, and the adversary can neither stall the round nor close it early — how the code refuses each is below. Each recorded tick is reported to the adversary before UPDATE returns, on line 20, *responsively* (Appendix F) and with

the answer discarded; what the notification buys and what it costs is settled at the end of Appendix J.2. Every operation answers the counter as it stands on return, so the tick that closes a round learns the new time and the ticks before it the old; a party that must know where it stands reads again.

The fields. The process id is a constant, $(\text{GCLOCK}, 0, 0)$, where $\mathcal{F}_{\text{Sig}}$'s was a parameter. Definition 1.2 then puts at most one copy in any system, which is the point: global means one instance, shared, and a second clock would be a second time. $\mathbf{N} := \mathbf{S} \cup \mathbf{A} \cup \mathbf{Z}$ makes $\text{GLOBAL}(\mathcal{G}_{\text{Clock}})$ hold by its first disjunct, so line 3 of GUARD waives the session check and every session reaches the one instance — the very separation the session component exists to make, deliberately not made here. A protocol that keeps time declares $(\mathcal{G}_{\text{Clock}}, \text{SERVES})$ in its $\mathbf{U}$, and the global disjunct of Definition 1.3 is what lets members of every session meet the entry at the shared copy. $\mathbf{U} := \{(\mathcal{A}, \text{SERVES})\}$ for the one call the code places, the notification of line 20. And $\mathbf{P}$ stays a parameter but as the universe rather than the membership: it fixes the parties an interface exists for, while who is *in* the round structure at a given moment is state, entered and left through REGISTER and DEREGISTER as in [1] — dynamic membership is one more table, not a new kind of field.

Reading and ticking. $\mathbf{N}$ is a field of the functionality, not of an operation, so one set governs every core, and the set above is the right one for READ: the adversary and the environment may ask the time — [1] lets both, and the environment reading the shared clock is much of what makes a global setup global. It is too wide for the three mutators, where an adversary free to act for a party could close a round the party had not finished, or drop it from the round structure altogether. The refinement sits as the first line of each core: a caller named A or Z acting for an *honest* party is answered the time and changes nothing, while at a corrupt party it passes — the adversary registers, deregisters and ticks for the parties it owns, as in [1], and those ticks are inert where it matters, line 17 never reading a corrupt party's flag. One consequence deserves naming: while no honest party is registered the test of line 17 is vacuous, so the adversary may register a corrupt party and tick it freely — until the first honest registration, time is the adversary's to spend, there too as in [1].

The quantifier of line 17 reads "every honest registered party", and it is worth seeing how little of it $\mathcal{G}_{\text{Clock}}$ enforces itself. At a corrupt party a standard caller's UPDATE is handed to the adversary by line 4 of the full interface before the core runs, and the adversary's own call at the same party is admitted by line 13; either way a corrupt party's ticks are the adversary's to give or withhold, and the $\forall$ — which reads the register by the convention of Section 3 — ignores them regardless. Corrupt parties therefore neither hold a round open nor help close one. At an honest party both routes shut: MEDIATE does not fire, and line 13 turns the observers' ticks into reads. Monotonicity adds the last piece: a party corrupted after ticking leaves a stale flag behind, but it is outside the quantifier from then on, and the next advance clears it.

Functionality $\mathcal{G}_{\mathsf{Clock}}$
pid $:= (\text{GCLOCK}, 0, 0)$, $\mathbf{P}$, $\mathbf{N} := \mathbf{S} \cup \mathbf{A} \cup \mathbf{Z}$, $\mathbf{U} := \{(\mathcal{A}, \text{SERVES})\}$, *par* $:= \bot$

INITIALIZE():
1: now $\leftarrow 0$
2: reg, tick : $\mathcal{G}_{\mathsf{Clock}}.\mathbf{P} \to \{0, 1\}$
3: reg$[*] \leftarrow 0$; tick$[*] \leftarrow 0$

id.REGISTER() from id′
4: **if** id′.F $\in \{A, Z\} \wedge$ id.P $\notin \mathbf{C}$ **then**
5: **return** now
6: reg[id.P] $\leftarrow 1$; tick[id.P] $\leftarrow 0$ *// a fresh member owes a tick*
7: **return** now

id.DEREGISTER() from id′
8: **if** id′.F $\in \{A, Z\} \wedge$ id.P $\notin \mathbf{C}$ **then**
9: **return** now
10: reg[id.P] $\leftarrow 0$
11: **return** now

id.UPDATE() from id′
12: **if** id′.F $\in \{A, Z\} \wedge$ id.P $\notin \mathbf{C}$ **then**
13: **return** now *// reading, not ticking*
14: **if** reg[id.P] $= 0$ **then**
15: **return** now *// not in the round structure*
16: tick[id.P] $\leftarrow 1$
17: **if** $\forall$ P $\in \mathcal{G}_{\mathsf{Clock}}.\mathbf{P} \setminus \mathbf{C}$: reg[P] $= 1 \Rightarrow$ tick[P] $= 1$ **then**
18: now $\leftarrow$ now $+ 1$
19: tick$[*] \leftarrow 0$ *// the round is over*
20: $\mathcal{A}^{!}$(id.UPDATE, now) *// notify; the answer is discarded*
21: **return** now

id.READ() from id′
22: **return** now

id.LEAK() from id′
23: **return** (now, reg, tick)

Three remarks on the code. Registrations and ticks are recorded against the *party*, not the caller: any machine acting for P may register or tick P, GUARD having already tied the call to the party on line 2. [1] instead has every registered party and functionality tick separately before the round closes; that discipline, if wanted, is tables indexed by identity rather than party, and nothing else changes. The notification is UPDATE's alone; [1] tells the adversary about registrations and departures too, and extending line 20's pattern to the other two mutators is mechanical. And LEAK returns the whole state, a clock keeping no secrets: the counter is anyone's to read anyway, and the tables say only who is in the round and who is done with it.

Set against $\mathcal{F}_{\mathsf{Sig}}$, most of what the signature example needed is still absent, each absence legible in the code. No value is the simulator's to choose — the notification's answer is discarded — so there is nothing to sanitize. And where $\mathcal{F}_{\mathsf{Sig}}$ reached the slot through $\mathcal{A}(\cdot)$ and re-tested its tables against what could happen while it hung, the clock's one call is responsive by construction: every mutation of UPDATE precedes line 20, and RESPOND keeps the token from leaving, so no instance is ever re-entered mid-operation and there is nothing to re-test — enforced against every occupant of the slot, where Appendix H could only advise a simulator class. What the notification costs is recorded where it bites: Appendix J.2 plays two games over the clock and prices it there.

J.2 Properties

Appendix H.2 instanced Definition 12.1 once; the clock is worth a second, not least because time is the one thing in this paper whose *order* is the content. We give two games over $\mathcal{G}_{\mathsf{Clock}}$: $\mathcal{G}\mathrm{m}_{\text{STEAD}}$, *steadiness*, saying that the counter moves as the code promises — forward, by one, and only when pushed — and $\mathcal{G}\mathrm{m}_{\text{UNA}}$, *unanimity*, saying that it moves only when every honest registered party has pushed. They differ in FINALIZE alone, so the shell is written once, as before.

Both are played over the game system $\gamma := \{\mathcal{G}\mathfrak{m}\} \cup \{\mathcal{G}_{\mathsf{Clock}}\}$, with the clock taken as in Appendix J save one field: $\mathbf{N} := \{\mathcal{G}\mathfrak{m}.\mathit{pid}\}$, pinned. The change is forced. Tightness at $\{\mathcal{G}\mathfrak{m}\}$, which Definition 12.1 requires, asks the members under test to admit whole process ids, and a bare $\mathbf{S}$ is exactly what Remark A.5 refuses; Definition 12.7 says why in operational terms — an environment shares a global functionality, touching it as anyone may, and a property of a global $\mathcal{F}$ is a property against an adversary that shares it. For these two games sharing is not a nuisance but the negation of the claim: steadiness and unanimity are assertions about *all* the ticks there are, and a game can certify them only where its trace is all there is. So the clock is played privately, and the properties certified are of its *code*, which sharing does not change — the situation of Appendix H.2 exactly, where the $\mathcal{F}_{\mathsf{Sig}}$ that ships names its protocol and the $\mathcal{F}_{\mathsf{Sig}}$ under test names the game.

Pinned, the clock is no longer global, so line 3 of GUARD asks for one session; the experiment sits in session 0 and the clock's process id carries $s = 0$, so the check passes. And no caller but $\mathcal{G}\mathfrak{m}$ now passes line 2 at the clock: not the environment or a machine it hosts, whose claims lie outside a pinned $\mathbf{N}$, and not the adversary, whose A-identities are refused before line 13 could even answer them the time. So the counter changes only inside line 15 below and the membership only inside the REGISTER and DEREGISTER relays beside it, and every change is recorded — a game whose verdict is "the time moved when no one pushed" has no other way to know. Inside an interface at $\mathtt{id}$ we write $\mathtt{id}_{\mathcal{G}_{\mathsf{Clock}}}$ for $(\mathcal{G}_{\mathsf{Clock}}.\mathit{pid}, \mathtt{id}.\mathsf{P})$, and every trace entry carries the answer τ the clock gave.

One line has no analogue in Appendix H.2: the relays refuse the root. There the relays at the root addressed a party $\mathcal{F}_{\mathsf{Sig}}$ does not serve and were answered REJ at line 1, which the verdicts, existential over well-shaped tuples, never read. Steadiness reads *every* entry, so a REJ of the game's own making cannot be allowed into the trace; refused at the relay, nothing is appended, and every entry in $\mathtt{tr}$ is an answer the clock actually gave. Against a replacement this bites, and rightly: a π answering an honest read anything but a number loses steadiness on the spot, a clock that cannot say the time being broken however it moves.

Game shell over $\mathcal{G}_{\text{Clock}}$, and the FINALIZE of $\mathcal{G}\text{m}_{\text{STEAD}}$
pid $:= (\mathcal{G}\text{m}.\text{F}, 0, 0)$, $\mathbf{P} := \mathcal{G}_{\text{Clock}}.\mathbf{P} \cup \{\text{Z}\}$, $\mathbf{N} := \mathbf{Z}$, $\mathbf{U} := \{(\mathcal{G}_{\text{Clock}}, \text{SERVES}_{\mathcal{G}\text{m}})\}$, *par* $:= \bot$

INITIALIZE():
1: DONE $\leftarrow$ FALSE
2: tr $\leftarrow$ ( ) *// the trace, a sequence*

id.REGISTER() from id′
3: **require** ¬DONE
4: **require** id.P $\neq$ Z *// the root only finalizes*
5: $\tau \leftarrow \text{id}_{\mathcal{G}_{\text{Clock}}}$.FULLREGISTER() as id
6: tr $\leftarrow$ tr $\cdot$ (REG, id.P, τ)
7: **return** τ

id.DEREGISTER() from id′
8: **require** ¬DONE
9: **require** id.P $\neq$ Z
10: $\tau \leftarrow \text{id}_{\mathcal{G}_{\text{Clock}}}$.FULLDEREGISTER() as id
11: tr $\leftarrow$ tr $\cdot$ (DEREG, id.P, τ)
12: **return** τ

id.TICK() from id′
13: **require** ¬DONE
14: **require** id.P $\neq$ Z
15: $\tau \leftarrow \text{id}_{\mathcal{G}_{\text{Clock}}}$.FULLUPDATE() as id
16: tr $\leftarrow$ tr $\cdot$ (TICK, id.P, τ)
17: **return** τ

id.READ() from id′
18: **require** ¬DONE
19: **require** id.P $\neq$ Z
20: $\tau \leftarrow \text{id}_{\mathcal{G}_{\text{Clock}}}$.FULLREAD() as id
21: tr $\leftarrow$ tr $\cdot$ (READ, id.P, τ)
22: **return** τ

id.LEAK() from id′
23: **return** $\bot$

id.FINALIZE() from id′ (steadiness)
24: **require** ¬DONE
25: **require** id.P $=$ Z
26: DONE $\leftarrow$ TRUE
27: parse tr as $\big((op_j, \text{P}_j, \tau_j)\big)_{j=1}^{n}$; $\tau_0 := 0$
28: $w \leftarrow 1$ if $\exists\, j \in \{1, \ldots, n\}$:
29: $\quad \tau_j \notin \{\tau_{j-1},\ \tau_{j-1}+1\}$
30: $\quad \vee\ \big(\tau_j = \tau_{j-1}+1 \wedge op_j \neq \text{TICK}\big)$;
31: else $w \leftarrow 0$
32: **return** w

The FINALIZE of $\mathcal{G}\text{m}_{\text{UNA}}$; the shell above is unchanged

id.FULLFINALIZE() from id′
1: **require** ¬DONE
2: **require** id.P $=$ Z
3: DONE $\leftarrow$ TRUE
4: $\mathbf{C} \leftarrow (\text{C}, \text{Z})$.FULLSTATUS() as id
5: $w \leftarrow 1$ if $\exists\, t \geqslant 0\ \ \exists\, \text{P} \in \mathcal{G}_{\text{Clock}}.\mathbf{P} \setminus \mathbf{C}$:
6: $\quad \big(\exists\, \tau_r \leqslant t : (\text{REG}, \text{P}, \tau_r) \in \text{tr}\big) \wedge (\text{DEREG}, \text{P}, \cdot) \notin \text{tr}$
7: $\quad \wedge\ \big(\exists\, \tau > t : (\cdot, \cdot, \tau) \in \text{tr}\big) \wedge \big(\forall\, \tau' \in \{t,\ t+1\} : (\text{TICK}, \text{P}, \tau') \notin \text{tr}\big)$;
8: else $w \leftarrow 0$
9: **return** w

The verdicts, in words. $\mathcal{G}\text{m}_{\text{STEAD}}$ wins on an answer below its predecessor, an answer more than one above it, or an increase anywhere but at a tick: reads, registrations and departures must find the time where the last answer left it. A tick answered $\tau_{j-1}+1$ is allowed — it is the tick that closed the round — and $\tau_0 := 0$ is INITIALIZE's word taken as the first observation, so a first read answered 1 is already a counter that moved before anyone pushed. $\mathcal{G}\text{m}_{\text{UNA}}$ wins if the counter passed some t although some party — honest at the close, registered by round t and never released — has no tick answered t or $t+1$: no tick placed during round t, that is, the answer $t+1$ covering the one that closed it. The registration conjuncts read the trace, not the clock's tables, so they too name only what the

game itself did.

Two contrasts with Appendix H.2, both instructive. There the trace was a sequence whose order no verdict read; here $\mathcal{G}\mathfrak{m}_{\text{STEAD}}$ reads little else, time being an order and not much besides. And there both verdicts had to ask $\mathtt{P} \notin \mathbf{C}$ of every party they named; $\mathcal{G}\mathfrak{m}_{\text{STEAD}}$ asks nothing of anyone, because at a corrupt party the game's own wrapper intercepts before the core runs and records (Remark 12.4), so every entry in $\mathtt{tr}$ is an honest core's answer already. The one verdict that must name honesty is $\mathcal{G}\mathfrak{m}_{\text{UNA}}$'s, whose claim concerns parties that did *not* call, and it names it at the close — monotonicity (Section 7) again making that the stronger reading.

Remark J.1 (Agreement is steadiness read across parties)**.** The property a shared clock is usually wanted for is *agreement*: honest parties hold a common view of the time. Here it is not a third game but a corollary of the verdict above, and by design. Line 28 quantifies over the trace *interleaved*, not party by party: between two consecutive TICK entries every answer must equal its predecessor, whoever asked, so two honest parties reading between the same two ticks are answered the same value or the game is won. Stated per party — each party's own subsequence steady — the verdict would be strictly weaker, and agreement a separate property: a replacement could then answer P with 5 and Q with 3 in adjacent activations and lose neither game. What lets one verdict carry both claims is the execution model: a single token totally orders the trace, so "simultaneous reads" do not exist and a common view reduces to the consistency of adjacent entries. Two limits are worth naming. Agreement is among honest parties only, a corrupt party's answers being the adversary's (Remark 12.4); and it says nothing of a party that does not ask, whose view may grow stale while the clock runs ahead — staleness of the party, not disagreement of the clock, and nothing a functionality could promise a caller that never calls. A *drift* variant — views agreeing within some Δ, the loosely synchronized clocks of [3] — would be a genuine weakening of line 28, worth its own game where loose synchrony rather than a shared counter is the goal.

Proposition J.2 ($\mathcal{G}_{\mathsf{Clock}}$ has both properties)**.** *For $\mathcal{G}\mathfrak{m}$ either of $\mathcal{G}\mathfrak{m}_{\text{STEAD}}$ and $\mathcal{G}\mathfrak{m}_{\text{UNA}}$, for every $\mathcal{Z} \in \mathbb{Z}_{\gamma,\gamma}$ and every occupant $\mathcal{X}$ of the adversary slot,* $\Pr[\text{WIN}] = 0$*. So $\{\mathcal{G}_{\mathsf{Clock}}\}$ has each property within 0 against $(\mathbb{Z}_{\gamma,\gamma}, \mathbb{X})$ in the sense of Definition 12.8, for $\mathbb{X}$ the class of all occupants.*

Proof. Everything rests on one invariant: the counter changes only inside a tick the trace records, and by exactly one; the membership only inside recorded registrations and departures. Both tables are written only in the cores of Appendix J — the counter at one line of UPDATE, stepping by one when the test of line 17 passes, $\mathtt{reg}$ in REGISTER and DEREGISTER — and those cores run only on calls passing GUARD: with $\mathbf{N}$ pinned no caller but $\mathcal{G}\mathfrak{m}$ passes line 2, and $\mathcal{G}\mathfrak{m}$ calls only through the relays, at honest parties — at a corrupt one its own wrapper intercepted a step earlier and no core of γ ran — so no call is mediated and each core's answer is the answer recorded. Moreover no core of γ hands the token to anything that could run one: the one call the clock's cores place is the notification of line 20, which is responsive — RESPOND refuses every call its answerer would place, whatever occupies the slot, and every mutation of UPDATE precedes it in any case — and the relays' one call is on the clock, that chain returning with no other machine of γ having run in between. Entries

therefore append in execution order, each carrying the counter as its call returned.

For steadiness: before the first entry the counter is INITIALIZE's $0 = \tau_0$. Between consecutive entries no core of γ runs, so the counter at entry j differs from that at entry $j-1$ by the steps taken inside entry j's own call: none for a read, a registration or a departure, none or one for a tick. Hence $\tau_j \in \{\tau_{j-1}, \tau_{j-1}+1\}$ throughout, with the increment only at a tick, and line 28 gives 0.

For unanimity: suppose the four conjuncts of line 5 hold of some t and some $P \in \mathcal{G}_{\mathsf{Clock}}.\mathbf{P} \setminus \mathbf{C}$ at the close — honest throughout, by monotonicity. A recorded registration answered $\tau_r \leqslant t$ set $\mathtt{reg}[P] \leftarrow 1$; only the cores of REGISTER and DEREGISTER write reg, the game placed no DEREGISTER at P — there is no DEREG entry — and the observers cannot reach an honest party, so $\mathtt{reg}[P] = 1$ from that moment on. Some entry answers $\tau > t$, and by the invariant the answers are values the counter took, so it stepped from t to $t+1$ inside some recorded tick; its line 17 read the flags of every party honest at that moment and registered then — P among them, the register only ever growing and $\tau_r \leqslant t$ — so $\mathtt{tick}[P] = 1$. That flag is written only by the core of UPDATE at P and cleared at every step of the counter and at every re-registration, so it was set while the counter stood at t, by a tick of the game at P; that tick answered t, or $t+1$ if it was itself the one that closed the round, and it was recorded. So $(\text{TICK}, P, \tau') \in \mathtt{tr}$ for some $\tau' \in \{t, t+1\}$, contradicting the fourth conjunct, and line 5 gives 0. □

As with $\mathcal{F}_{\mathsf{Sig}}$, the bound being 0 says only that the specification was written to have its properties; the content is the transfer, and here the notification of line 20 collects its price. Corollary 12.11 asks that the cores of the game system place no responsive call, and that line is one, so the transfer to a system π put in place of $\{\mathcal{G}_{\mathsf{Clock}}\}$ must be argued directly, for the reason Remark F.4 gives — and the same hypothesis guards Theorem D.4, so a system built over the notifying clock forgoes dummy completeness with it. A clock without the notification — delete line 20 and the **U** entry — has both properties by the same proof and inherits them through the corollary with nothing to discharge.

The choice, note, is only *which* price, not whether. A notification through the ordinary $\mathcal{A}(\cdot)$, the route $\mathcal{F}_{\mathsf{Sig}}$ took, would hand the token to an adversary free to drive nested ticks before answering; answers would then append out of trace order, and steadiness would fail of the ideal clock itself — the games above are properties of $\mathcal{G}_{\mathsf{Clock}}$ only because line 20 cannot be acted on before UPDATE returns. So where $\mathcal{F}_{\mathsf{Sig}}$ chose inheritance and paid with locality, a notifying clock must choose locality and pay with inheritance; the clock that pays neither is the one that tells the adversary nothing.

K Δ-Delayed Network

K.1 Functionality

The clock exists to be read, and the first reader is a network. Communication with a bounded delay is [3]'s; the diffusion form, delays measured on a shared clock, is the network the composable blockchain analyses run over [1]. $\mathcal{F}_{\mathsf{Net}}$ below is that functionality in the language of this paper — and the first example to *use* the clock rather than be it. Section 1 named a diffusion channel $\mathcal{F}_{\mathsf{Diffuse}}$ and owed no code; $\mathcal{F}_{\mathsf{Net}}$ is what such a channel looks like with time attached.

What it does is quickly said. SEND timestamps the message by the clock — through $\mathtt{id}_{\mathcal{G}_{\mathsf{Clock}}} := (\mathcal{G}_{\mathsf{Clock}}.\mathit{pid}, \mathtt{id}.P)$, the clock's interface at the party being served, the notation

of Appendix J.2 — stores it, tells the adversary — the network is public — and answers a message id. FETCH asks the adversary what to reveal early, then answers everything released to the fetching party together with everything Δ old. The division of power is the point: the adversary chooses *when* within Δ, and to whom first, and may add traffic of its own; what it cannot do is withhold an honest message past its time, and that bound sits in FETCH's last line, where no answer of the slot reaches it. A message sent at time τ is in every fetch from time $\tau + \Delta$ on, whoever the adversary favoured first.

The fields. They follow $\mathcal{F}_{\mathsf{Sig}}$'s pattern, not the clock's. *pid*, **P** and **N** are parameters: a network is local, one instance per session, its **N** naming the protocol it serves in the local pattern of Section 1 — and every instance reads the one shared clock, which is what makes "time τ" comparable across sessions. $\mathbf{U} := \{(\mathcal{G}_{\mathsf{Clock}}, \text{SERVES}), (\mathcal{A}, \text{SERVES})\}$: the first entry requires the clock to serve every party the network does and is met, whatever session the network sits in, at the shared copy by the global disjunct of Definition 1.3 — a system containing $\mathcal{F}_{\mathsf{Net}}$ is well-formed only with the clock alongside — and the second covers the two slot calls below. *par* $:= \Delta$, the first parameter in this paper that a line of code actually reads.

Scheduling is answered, not called. [1] lets the adversary deliver a message early by acting on the network of its own accord. Transcribed literally that is an interface the adversary *calls*, so a local **N** containing **A** — reachable from the slot in session $\emptyset$ and in no other, the one shape Definition B.2 excludes and relocation cannot carry. So the slot is asked instead: FETCH hands the adversary the time and takes back a pair (S, I) — releases among the stored ids, injections of fresh traffic — sanitized inline, the good repair being canonical, as in Appendix H. Both slot calls are responsive, the clock's discipline. SEND mutates before it notifies; FETCH cannot, its mutations reading the answer, and responsiveness is what makes the difference immaterial: the token comes straight back, nothing else runs inside either call, and $\mathcal{F}_{\mathsf{Sig}}$'s re-entrance re-tests stay unnecessary here too.

Corruption, without a line of code. A corrupt party's SEND and FETCH are intercepted by line 4 of the full interface before the cores run, so the buffer holds honest sends only, and the cores' clock reads are never mediated — a timestamp is always the clock's own word. What a corrupt party "sends", the adversary supplies from the receiving side instead: the injections of line 16 enter the buffer with sender $\perp$ and timestamp ∞, released to the fetching party alone — no age, and so no ripening into line 20's guarantee, which is for the honest and never conscripts the adversary's own traffic.

Functionality $\mathcal{F}_{\mathsf{Net}}$
pid, **P**, **N**, **U** $:= \{(\mathcal{G}_{\mathsf{Clock}}, \textsc{serves}), (\mathcal{A}, \textsc{serves})\}$, *par* $:= \Delta$

INITIALIZE():
1: `ctr` $\leftarrow 0$
2: `buf` $\leftarrow \varnothing$ *// entries* $(\mathfrak{m}, \mathsf{P}, msg, \tau)$
3: `rel` $: \mathbb{N} \times \mathcal{F}_{\mathsf{Net}}.\mathbf{P} \to \{0,1\}$
4: `rel`$[*,*] \leftarrow 0$ *// what is released to whom*

id.SEND(*msg*) from id′
5: $\tau \leftarrow \mathrm{id}_{\mathcal{G}_{\mathsf{Clock}}}.\textsc{FullRead}()$ as id
6: $\mathfrak{m} \leftarrow$ `ctr`; `ctr` $\leftarrow$ `ctr` $+ 1$
7: `buf` $\leftarrow$ `buf` $\cup \{(\mathfrak{m}, \mathrm{id}.\mathsf{P}, msg, \tau)\}$
8: $\mathcal{A}^{!}(\mathrm{id}.\textsc{Send}, \mathfrak{m}, msg)$ *// the network is public*
9: **return** $\mathfrak{m}$

id.LEAK() from id′
10: **return** (`buf`, `rel`)

id.FETCH() from id′
11: $\tau \leftarrow \mathrm{id}_{\mathcal{G}_{\mathsf{Clock}}}.\textsc{FullRead}()$ as id
12: $(S, I) \leftarrow \mathcal{A}^{!}(\mathrm{id}.\textsc{Fetch}, \tau)$ *// a non-pair reads as* $(\varnothing, \varnothing)$
13: $S \leftarrow S \cap \{\mathfrak{m} : (\mathfrak{m}, \cdot, \cdot, \cdot) \in$ `buf`$\}$;
14: $I \leftarrow I \cap \mathcal{M}$ *// sanitized inline*
15: `rel`$[\mathfrak{m}, \mathrm{id}.\mathsf{P}] \leftarrow 1$ for each $\mathfrak{m} \in S$
16: for each $msg \in I$:
17: `buf` $\leftarrow$ `buf` $\cup \{($`ctr`$, \bot, msg, \infty)\}$;
18: `rel`$[$`ctr`$, \mathrm{id}.\mathsf{P}] \leftarrow 1$; `ctr` $\leftarrow$ `ctr` $+ 1$
19: **return** $\{(\mathfrak{m}, msg) : (\mathfrak{m}, \mathsf{P}', msg, \tau_s) \in$ `buf`
20: $\wedge\ (\texttt{rel}[\mathfrak{m}, \mathrm{id}.\mathsf{P}] = 1 \vee \tau_s + \Delta \leqslant \tau)\}$

Three remarks on the code. Fetched pairs carry no sender: the network is an unauthenticated diffusion, [1]'s, and the injections are what unauthenticated *means* — a fetch may contain messages no one sent, the wire being the adversary's to write. An authenticated *diffusion* is one field and one line away — return the sender the buffer already keeps, drop the injections. Authentication with addressing is a further step and a different object; Appendix L gives it. `rel` is sticky: what one fetch reveals no later fetch may hide, so a party's view of the network only grows — and line 20 reads `rel` *or* the deadline, so the adversary's silence past Δ is simply ignored rather than punished. And LEAK returns the whole state, the network keeping no secrets: its contents were told to the adversary at line 8 as they arrived, and the release table is the slot's own doing.

K.2 Properties

Appendix K.1 left three properties unformulated: delivery by Δ, monotone views, and no delivery before sending. The last needs no game at all: an id not yet in `buf` is nothing S can name, line 20 intersecting with `buf` before anything else runs, so no occupant of the slot can ever release what has not yet been sent. Monotone views we still leave unformulated, being a property of `rel` alone and orthogonal to timing. What we formalize here is the first, under the name $\mathcal{G}\mathfrak{m}_{\textsc{Live}}$, *liveness*: a message ripened by Δ is missing from no later honest fetch. And we formalize a property $\mathcal{F}_{\mathsf{Net}}$ was never claimed to have, $\mathcal{G}\mathfrak{m}_{\textsc{Auth}}$, *authentication*: a message an honest party fetches was sent by someone. $\mathcal{F}_{\mathsf{Net}}$ has the first outright and, being an unauthenticated diffusion by design — Appendix K.1's own remark on the injections of line 16 — provably lacks the second. They differ in FINALIZE alone, so the shell is written once.

Both are played over the game system $\gamma := \{\mathcal{G}\mathfrak{m}\} \cup \{\mathcal{F}_{\mathsf{Net}}, \mathcal{G}_{\mathsf{Clock}}\}$, with $\mathcal{F}_{\mathsf{Net}}.\mathbf{N} := \{\mathcal{G}\mathfrak{m}.pid\}$ pinned and $\mathcal{F}_{\mathsf{Net}}.\mathbf{P} \subseteq \mathcal{G}\mathfrak{m}.\mathbf{P}$ — Appendix H.2's tightness argument transplanted whole, since $\mathcal{F}_{\mathsf{Net}}$'s own fields already follow $\mathcal{F}_{\mathsf{Sig}}$'s pattern rather than the clock's (Appendix K.1). The interface box below states $\mathcal{G}\mathfrak{m}.\mathbf{U}$ in full. No caller but $\mathcal{G}\mathfrak{m}$ passes GUARD at $\mathcal{F}_{\mathsf{Net}}$: not the environment, not the adversary, not a machine hosted at an invented instance of the game's name. So every message $\mathcal{F}_{\mathsf{Net}}$'s core ever stores, and every answer it ever returns to

a FETCH, passes through the relays below and is recorded — a game whose verdict names "sent by someone" has no other way to know it. $\mathcal{F}_{\text{Net}}$'s own dependence on the clock and the slot is unaffected by any of this and is met exactly as it was for $\mathcal{F}_{\text{Net}}$ itself, the clock at the shared copy by the global disjunct of Definition 1.3. Inside an interface at id we write $\text{id}_{\mathcal{F}_{\text{Net}}}$ for $(\mathcal{F}_{\text{Net}}.pid, \text{id.P})$ and $\text{id}_{\mathcal{G}_{\text{Clock}}}$ for $(\mathcal{G}_{\text{Clock}}.pid, \text{id.P})$, the second the notation of Appendix J.2.

Game shell over $\mathcal{F}_{\text{Net}}$ and $\mathcal{G}_{\text{Clock}}$, and the FINALIZE of $\mathcal{G}\text{m}_{\text{LIVE}}$
$pid := (\mathcal{G}\text{m.F}, 0, 0)$, $\mathbf{P} := \mathcal{F}_{\text{Net}}.\mathbf{P} \cup \{Z\}$, $\mathbf{N} := \mathbf{Z}$, $\mathbf{U} := \{(\mathcal{F}_{\text{Net}}, \text{SERVES}_{\mathcal{G}\text{m}}), (\mathcal{G}_{\text{Clock}}, \text{SERVES})\}$, $par := \bot$

INITIALIZE():
1: DONE $\leftarrow$ FALSE
2: tr $\leftarrow$ ( ) // the trace, a sequence

id.SEND(msg) from id′
3: **require** ¬DONE
4: $\tau \leftarrow \text{id}_{\mathcal{G}_{\text{Clock}}}$.FULLREAD() as id
5: $\text{m} \leftarrow \text{id}_{\mathcal{F}_{\text{Net}}}$.FULLSEND($msg$) as id
6: tr $\leftarrow$ tr $\cdot$ (SEND, id.P, m, τ, msg)
7: **return** m

id.FETCH() from id′
8: **require** ¬DONE
9: $\tau \leftarrow \text{id}_{\mathcal{G}_{\text{Clock}}}$.FULLREAD() as id
10: $\text{M} \leftarrow \text{id}_{\mathcal{F}_{\text{Net}}}$.FULLFETCH() as id
11: tr $\leftarrow$ tr $\cdot$ (FETCH, id.P, τ, M)
12: **return** M

id.LEAK() from id′
13: **return** $\bot$

id.FINALIZE() from id′ (liveness)
14: **require** ¬DONE
15: **require** id.P $= Z$
16: DONE $\leftarrow$ TRUE
17: $\mathbf{C} \leftarrow (C, Z)$.FULLSTATUS() as id
18: $w \leftarrow 1$ if $\exists$ P, Q, m, τ_s, msg, τ, M :
19: P $\notin \mathbf{C} \wedge$ Q $\notin \mathbf{C}$
20: $\wedge$ (SEND, P, m, τ_s, msg) $\in$ tr
21: $\wedge$ (FETCH, Q, τ, M) $\in$ tr $\wedge$ $\tau \geqslant \tau_s + \Delta$
22: $\wedge$ (m, msg) $\notin$ M;
23: else $w \leftarrow 0$
24: **return** w

The FINALIZE of $\mathcal{G}\text{m}_{\text{AUTH}}$; the shell above is unchanged

id.FULLFINALIZE() from id′
1: **require** ¬DONE
2: **require** id.P $= Z$
3: DONE $\leftarrow$ TRUE
4: $\mathbf{C} \leftarrow (C, Z)$.FULLSTATUS() as id
5: $w \leftarrow 1$ if $\exists$ Q, τ, M, m, msg :
6: Q $\notin \mathbf{C} \wedge$ (FETCH, Q, τ, M) $\in$ tr $\wedge$ (m, msg) $\in$ M
7: $\wedge \neg\exists$ P, τ_s : (SEND, P, m, τ_s, msg) $\in$ tr;
8: else $w \leftarrow 0$
9: **return** w

Three things about the code. Neither relay reads SEND's or FETCH's own return value for a timestamp — there is none to read — so both read the clock themselves, one line above the call to $\mathcal{F}_{\text{Net}}$; the two readings coincide because no core of γ runs between them, not $\mathcal{F}_{\text{Net}}$'s own guard and corruption check, which only reads $\mathbf{C}$ and finds id.P honest, the

relay having run at all, and not READ itself, which places no call (Appendix J.1). Neither relay excludes $\mathtt{id}.P = Z$, unlike Appendix J.2's: $\mathcal{F}_{\mathsf{Net}}$'s fields follow $\mathcal{F}_{\mathsf{Sig}}$'s pattern and not the clock's, and nothing below reads any the worse for the root sending or fetching too. And authentication's verdict asks nothing of a sender, because FETCH returns none to ask about — the fetched pair is (m, msg) alone, and whether some m was ever sent, by whom, is exactly the fact no caller of $\mathcal{F}_{\mathsf{Net}}$ can otherwise recover, which is what the whole game is testing.

Proposition K.1 ($\mathcal{F}_{\mathsf{Net}}$ has liveness). *For every $\mathcal{Z} \in \mathbb{Z}_{\gamma,\gamma}$ and every occupant $\mathcal{X}$ of the adversary slot,* $\Pr[\text{WIN}] = 0$. *So $\{\mathcal{F}_{\mathsf{Net}}, \mathcal{G}_{\mathsf{Clock}}\}$ has liveness within 0 against $(\mathbb{Z}_{\gamma,\gamma}, \mathbb{X})$ in the sense of Definition 12.8, for $\mathbb{X}$ the class of all occupants.*

Proof. Suppose $(\text{SEND}, P, m, \tau_s, msg) \in \mathtt{tr}$, written at some activation of the SEND relay. Its own clock read and $\mathcal{F}_{\mathsf{Net}}$.SEND's internal one (Appendix K.1) answer the same value, by the argument just given, so the core stores exactly (m, P, msg, τ_s) in $\mathtt{buf}$.

Now suppose also $(\text{FETCH}, Q, \tau, M) \in \mathtt{tr}$ with $\tau \geqslant \tau_s + \Delta$. $\mathcal{F}_{\mathsf{Net}}$.FETCH's core returns every pair (m', msg') with $(m', P', msg', \tau_{s'}) \in \mathtt{buf}$ and $\mathtt{rel}[m', Q] = 1 \vee \tau_{s'} + \Delta \leqslant \tau$ (line 20), τ its own reading of the clock and, again, the relay's. Taking $m' = m$: the buffer entry is (m, P, msg, τ_s) and $\tau_s + \Delta \leqslant \tau$ holds by hypothesis, so the second disjunct is met whatever $\mathtt{rel}[m, Q]$ is and whatever the slot answered at line 12, which settles only S and I and neither disjunct. Hence $(m, msg) \in M$, and line 18 gives 0. □

Proposition K.2 ($\mathcal{F}_{\mathsf{Net}}$ does not have authentication). *There are $\mathcal{Z}^* \in \mathbb{Z}_{\gamma,\gamma}$ and an occupant $\mathcal{X}^*$ of the adversary slot with* $\Pr[\text{WIN}] = 1$: *no $\varepsilon < 1$ bounds $\{\mathcal{F}_{\mathsf{Net}}, \mathcal{G}_{\mathsf{Clock}}\}$'s authentication against any class containing them.*

Proof. Let $\mathcal{X}^*$ have $\mathcal{A}$'s fields but place no call. On the query $\mathtt{id}.A(\mathtt{id}''.\text{FETCH}, \tau)$ with $\mathtt{id}''.F = \mathcal{F}_{\mathsf{Net}}.pid$ — the shape of line 12 — it returns $(\varnothing, \{msg^*\})$ for one fixed $msg^* \in \mathcal{M}$; on anything else it returns $\perp$. Placing no call, it is never refused by RESPOND at either of $\mathcal{F}_{\mathsf{Net}}$'s slot calls, unlike the dummy, whose only way to answer is to relay to $\mathcal{Z}$ (line 5), a call RESPOND does refuse (Remark F.4). This is exactly where $\mathcal{X}^*$ differs from $\mathcal{D}$, and exactly why Propositions H.1, J.2 and K.1 are stated over every occupant of the slot and not the dummy alone — the default $\text{ADV}_{\sigma}^{\mathcal{G}m}(\mathcal{Z})$ of Definition 12.8 would read $\mathcal{G}m_{\text{AUTH}}$ as satisfied, and only vacuously, $\mathcal{D}$ being unable to inject at all.

Let $\mathcal{Z}^*$ claim $(\mathcal{G}m.pid, Q)$ for $Q \in \mathcal{F}_{\mathsf{Net}}.\mathbf{P}$, call FETCH() once, then claim $(\mathcal{G}m.pid, Z)$ and call FINALIZE(). It corrupts no one, hosts nothing, and claims only identities of $\mathcal{G}m$'s own name, so $\mathcal{Z}^* \in \mathbb{Z}_{\gamma,\gamma}$.

At that fetch, $I = \{msg^*\}$ survives sanitizing ($I \leftarrow I \cap \mathcal{M}$), so the loop of line 16 adds some fresh $(m, \perp, msg^*, \infty)$ to $\mathtt{buf}$ and sets $\mathtt{rel}[m, Q] \leftarrow 1$; (m, msg^*) is returned, and $(\text{FETCH}, Q, \tau, M) \in \mathtt{tr}$ with $(m, msg^*) \in M$. No SEND relay ever ran, so $\mathtt{tr}$ carries no entry $(\text{SEND}, \cdot, m, \cdot, msg^*)$ — indeed no SEND entry naming m at all, m being fresh to the injection. $Q \notin \mathbf{C}$ throughout, and line 5 gives 1. □

As with $\mathcal{G}_{\mathsf{Clock}}$, liveness transfers by Corollary 12.11 only if the cores of the game system place no responsive call, and both of $\mathcal{F}_{\mathsf{Net}}$'s do, lines 8 and 12; a system π put in place of $\{\mathcal{F}_{\mathsf{Net}}, \mathcal{G}_{\mathsf{Clock}}\}$ must then have the transfer argued directly, for the reason Remark F.4 gives,

exactly as the clock's own notification cost Appendix J.2 the same thing. Tightness, note, did no work in either proof above beyond licensing γ as a game system: unlike correctness and unforgeability, neither verdict here needs to rule out a message entering `buf` by some route $\mathcal{G}\mathfrak{m}$ never sees, only to read off what happens to what $\mathcal{G}\mathfrak{m}$ itself placed — so both propositions would survive a weaker dependence, were one wanted, though nothing above asks for it.

Authentication transfers nothing, having nothing to transfer: Proposition K.2 is not a bound to inherit but a fact about the specification, exactly the fact Appendix K.1 already asserted in prose — proved now rather than merely said, and by an adversary no stranger than one that ignores its mail. No replacement of $\{\mathcal{F}_{\mathsf{Net}}, \mathcal{G}_{\mathsf{Clock}}\}$ is asked to improve on it. What does have the analogous property is Appendix L's $\mathcal{F}_{\mathsf{AC}}$: a corrupt sender's traffic there enters through SEND itself, under its own identity, rather than through an unaccountable slot answer (Appendix L, "Authentication is the adversary's own interface"), so the fetched pair carries a sender for a verdict to name, and there the transplanted win condition — fetched, addressed, and never sent — is not met.

L Δ-Delayed Authenticated Channel

L.1 Functionality

$\mathcal{F}_{\mathsf{Net}}$ diffuses and nobody signs. The complementary object is the one where every message arrives with its sender attached and none arrives from nobody — the authenticated channel of [2], given the same Δ and the same clock. $\mathcal{F}_{\mathsf{AC}}$ below is that functionality, and the two are worth reading side by side: they differ in a good deal more than the sender field, and the difference is where authentication actually lives.

What it does is quickly said. SEND takes a recipient as well as a message, stamps the pair by the clock, files it under a fresh index, and tells the adversary — authentication is not secrecy, and this channel hides nothing at all. FETCH gathers everything addressed to the fetching party that has come of age, asks the adversary which of the rest to release early, answers the union, and *empties* the entries it answered. What the receiver gets is the whole record: sender, recipient, send time, message. The division of power is $\mathcal{F}_{\mathsf{Net}}$'s — the adversary chooses when within Δ and in what order — and so is the bound: an entry Δ old is in the next fetch of the party it names, whatever the slot answers.

The fields. They are $\mathcal{F}_{\mathsf{Net}}$'s, for $\mathcal{F}_{\mathsf{Net}}$'s reasons. *pid*, **P** and **N** are parameters, one instance per session, with **N** naming the protocol served in the local pattern of Section 1, and *par* $:= \Delta$. The **U** field is $\{(\mathcal{G}_{\mathsf{Clock}}, \text{SERVES}), (\mathcal{A}, \text{SERVES})\}$, its clock entry met at the shared copy by the global disjunct of Definition 1.3. Both conjuncts of SERVES are needed and neither is a formality: the first asks the clock to serve every party the channel does, and without the second the channel's own name would not be admitted at the clock, so GUARD would refuse every read on line 4.

Authentication is the adversary's own interface. $\mathcal{F}_{\mathsf{Net}}$ needs line 16 because a diffusion channel gives adversarial traffic no name to enter under: the wire is anonymous, so forgery is modelled by letting the slot write on it directly. Here there is a name, and the full interface already delivers it. At a corrupt sender the adversary reaches the core — lines 3 and 4 admit $\mathcal{A}$ at a corrupt party and neither mediation hook fires on its calls, both testing $\mathtt{id}'.\mathsf{F} \neq \mathsf{A}$ — so corrupt traffic enters through SEND under the sender's own identity, timestamped by the clock like anyone's. $\mathcal{F}_{\mathsf{AC}}$ therefore has no injections and needs none, and the sender

field of an entry is not a report about the wire but a fact about which interface the call came through. That is what an authenticated channel is: forgery is impossible because there is nowhere to forge from.

The price is that adversarial traffic *ripens*. A message the adversary sends from a corrupt party is in the list with a real timestamp, so line 12 will deliver it by Δ and no later answer can retract it. $\mathcal{F}_{\mathsf{Net}}$ refuses this deliberately, filing injections at ∞ so the guarantee is for the honest and never conscripts the slot's own traffic. The difference is a consequence of addressing rather than a choice made against it: a message with a named sender and a named recipient is a message the sender is on the record for, and holding the adversary to the deadline it accepted when it spoke as a corrupt party is the reading that keeps the sender field meaning what it says.

Fetching empties. Each entry names one recipient, so answering it discharges it, and line 16 clears what line 12 and the slot between them selected. $\mathcal{F}_{\mathsf{Net}}$ cannot do this — a diffused message is owed to everyone — which is why it carries the sticky `rel` table instead and re-answers what it has already answered. The two guarantees are different in kind and neither implies the other: $\mathcal{F}_{\mathsf{Net}}$'s is that a party's view only grows, $\mathcal{F}_{\mathsf{AC}}$'s that a message is delivered exactly once. A protocol over $\mathcal{F}_{\mathsf{Net}}$ must deduplicate and one over $\mathcal{F}_{\mathsf{AC}}$ must not lose what it fetched, and each obligation is the other's absence.

Sanitizing the release. The slot's answer is a set of indices to release early, and the only thing wrong it can be is a set naming entries that are not the fetching party's — or not entries at all. Appendix G's procedure repairs exactly this kind of fault, so line 14 runs the answer through it under

$$\textsc{Clean}_{\textsc{addr}}(S;\, \texttt{list}, P) \;:\Longleftrightarrow\; S \subseteq \{\, i : \texttt{list}[i] = (\cdot,\, P,\, \cdot,\, \cdot)\,\},$$

with the list and the fetching party as the read-only context. Two things follow that are worth saying, because they are what makes San the right instrument here rather than an assertion. A good value always exists, so the footnote of Appendix G has nothing to prove in this case: $\varnothing$ satisfies $\textsc{Clean}_{\textsc{addr}}$ whatever the list holds, and being the empty set it is also the lexicographically first such, so a failing answer is repaired to no early release at all. And that repair costs the adversary nothing it could not have had, since an adversary wanting some particular good subset released may submit that subset and be returned it untouched. An assertion in San's place would read the same on good answers and hand the slot a denial of service on bad ones — one stray index and the honest party's fetch dies — which is the failure Appendix G exists to rule out.

Both slot calls are responsive. The clock's discipline, and here it is load-bearing in a way worth spelling out. Fetch reads the time on line 10 and asks the slot on line 13. Were that call not responsive the adversary would hold the token in between and could drive the clock forward before answering; τ would then be stale by the time line 12 used it, fewer entries would clear $\tau_i + \Delta \leqslant \tau$, and a message past its deadline would be lawfully withheld. Responsiveness is what makes the read and the test one moment. Send mutates before it notifies and needs no such argument, but is responsive too, for $\mathcal{F}_{\mathsf{Net}}$'s reason: nothing else may run inside either call.

Functionality $\mathcal{F}_{\mathsf{AC}}$
pid, **P**, **N**, $\mathbf{U} := \{(\mathcal{G}_{\mathsf{Clock}}, \textsc{serves}), (\mathcal{A}, \textsc{serves})\}$, $par := \Delta$

INITIALIZE():
1: $\mathtt{ctr} \leftarrow 0$
2: $\mathtt{list} : \mathbb{N} \to (\mathcal{F}_{\mathsf{AC}}.\mathbf{P}^2 \times \mathbb{N} \times \mathcal{M}) \cup \{\bot\}$
3: $\mathtt{list}[*] \leftarrow \bot$ *// entries* $(\mathrm{P}, \mathrm{Q}, \tau, msg)$

id.SEND(Q, *msg*) from id′
4: $\tau \leftarrow \mathrm{id}_{\mathcal{G}_{\mathsf{Clock}}}.\textsc{FullRead}()$ as id
5: $\mathtt{ctr} \leftarrow \mathtt{ctr} + 1$
6: $\mathtt{list}[\mathtt{ctr}] \leftarrow (\mathrm{id.P}, \mathrm{Q}, \tau, msg)$
7: $\mathcal{A}^{!}(\mathrm{id.SEND}, \mathtt{ctr}, \mathrm{Q}, \tau, msg)$
8: **return** OK

id.LEAK() from id′
9: **return** list

id.FETCH() from id′
10: $\tau \leftarrow \mathrm{id}_{\mathcal{G}_{\mathsf{Clock}}}.\textsc{FullRead}()$ as id
11: $\mathrm{R} \leftarrow \{\mathrm{m} : \mathtt{list}[\mathrm{m}] = (\cdot, \mathrm{id.P}, \tau_{\mathrm{m}}, \cdot)$
12: $\quad \wedge\ \tau_{\mathrm{m}} + \Delta \leqslant \tau\}$ *// come of age*
13: $\mathrm{S} \leftarrow \mathcal{A}^{!}(\mathrm{id.FETCH}, \tau)$ *// released early*
14: $\mathrm{S} \leftarrow \textsc{San}[\textsc{Clean}_{\textsc{addr}}](\mathrm{S}; \mathtt{list}, \mathrm{id.P})$
15: $\mathrm{M} \leftarrow \{\mathtt{list}[\mathrm{m}] : \mathrm{m} \in \mathrm{R} \cup \mathrm{S}\}$
16: $\mathtt{list}[\mathrm{m}] \leftarrow \bot$ for each $\mathrm{m} \in \mathrm{R} \cup \mathrm{S}$
17: **return** M

Three remarks on the code. The recipient is not checked against **P** on SEND: a message addressed outside the channel's parties is filed and never fetched, which is the same silence a party who declines to fetch would produce, and the guarantee below is a statement about entries someone fetches. The send time travels with the message, so a receiver can date what it receives and check the deadline itself — $\mathcal{F}_{\mathsf{Net}}$'s fetch returns no such thing, and the difference is that $\mathcal{F}_{\mathsf{AC}}$'s Δ is locally auditable where $\mathcal{F}_{\mathsf{Net}}$'s is only a fact about the functionality. And LEAK returns the list entire, the channel keeping no secrets: its contents were told to the adversary at line 7 as they arrived, and what has since been drained is what the slot itself watched leave.

L.2 Properties

The paragraph above named two claims: delivery by Δ, and the one $\mathcal{F}_{\mathsf{Net}}$ cannot make — every message fetched was sent, by the party it names. A third, no delivery before sending, again needs no game: an index not yet in `list`, or already drained, fails $\textsc{Clean}_{\textsc{addr}}$, so SAN repairs S to $\varnothing$ and nothing not yet real is released early either (Appendix K.2 made the same point for $\mathcal{F}_{\mathsf{Net}}$'s buffer). $\mathcal{G}\mathrm{m}_{\textsc{Live}}$ and $\mathcal{G}\mathrm{m}_{\textsc{Auth}}$ below formalize the other two, transplanting Appendix K.2's pair rather than starting over — the shell of relay, log, finalize does not change between a diffusion channel and an addressed one. What changes is which of the two $\mathcal{F}_{\mathsf{AC}}$ can keep: both, this time, and the second is the point of the whole design.

Both are played over the game system $\gamma := \{\mathcal{G}\mathrm{m}\} \cup \{\mathcal{F}_{\mathsf{AC}}, \mathcal{G}_{\mathsf{Clock}}\}$, with $\mathcal{F}_{\mathsf{AC}}.\mathbf{N} := \{\mathcal{G}\mathrm{m}.pid\}$ pinned and $\mathcal{F}_{\mathsf{AC}}.\mathbf{P} \subseteq \mathcal{G}\mathrm{m}.\mathbf{P}$ — the same tightness argument as Appendix K.2, and for the same reason, $\mathcal{F}_{\mathsf{AC}}$'s fields already following $\mathcal{F}_{\mathsf{Net}}$'s pattern (Appendix L.1). No caller but $\mathcal{G}\mathrm{m}$ passes GUARD at $\mathcal{F}_{\mathsf{AC}}$, so every entry SEND ever places in `list`, and every record FETCH ever returns, passes through the relays below and is recorded. The interface box states $\mathcal{G}\mathrm{m}.\mathbf{U}$ in full. Inside an interface at id we write $\mathrm{id}_{\mathcal{F}_{\mathsf{AC}}}$ for $(\mathcal{F}_{\mathsf{AC}}.pid, \mathrm{id.P})$ and $\mathrm{id}_{\mathcal{G}_{\mathsf{Clock}}}$ for $(\mathcal{G}_{\mathsf{Clock}}.pid, \mathrm{id.P})$, the second the notation of Appendix J.2.

Game shell over $\mathcal{F}_{\text{AC}}$ and $\mathcal{G}_{\text{Clock}}$, and the FINALIZE of $\mathcal{G}\text{m}_{\text{LIVE}}$
pid $:= (\mathcal{G}\text{m.F}, 0, 0)$, $\mathbf{P} := \mathcal{F}_{\text{AC}}.\mathbf{P} \cup \{Z\}$, $\mathbf{N} := \mathbf{Z}$, $\mathbf{U} := \{(\mathcal{F}_{\text{AC}}, \text{SERVES}_{\mathcal{G}\text{m}}), (\mathcal{G}_{\text{Clock}}, \text{SERVES})\}$, *par* $:= \bot$

INITIALIZE():
1: DONE $\leftarrow$ FALSE
2: tr $\leftarrow$ ( ) // *the trace, a sequence*

id.SEND(Q, *msg*) from id′
3: **require** ¬DONE
4: $\tau \leftarrow \text{id}_{\mathcal{G}_{\text{Clock}}}$.FULLREAD() as id
5: r $\leftarrow \text{id}_{\mathcal{F}_{\text{AC}}}$.FULLSEND(Q, *msg*) as id
6: tr $\leftarrow$ tr $\cdot$ (SEND, id.P, Q, τ, *msg*)
7: **return** r

id.FETCH() from id′
8: **require** ¬DONE
9: $\tau \leftarrow \text{id}_{\mathcal{G}_{\text{Clock}}}$.FULLREAD() as id
10: M $\leftarrow \text{id}_{\mathcal{F}_{\text{AC}}}$.FULLFETCH() as id
11: tr $\leftarrow$ tr $\cdot$ (FETCH, id.P, τ, M)
12: **return** M

id.LEAK() from id′
13: **return** $\bot$

id.FINALIZE() from id′ (liveness)
14: **require** ¬DONE
15: **require** id.P $= Z$
16: DONE $\leftarrow$ TRUE
17: **C** $\leftarrow$ (C, Z).FULLSTATUS() as id
18: $w \leftarrow 1$ if $\exists$ P, Q, τ_s, *msg*, τ, M :
19: P $\notin$ **C** $\wedge$ Q $\notin$ **C**
20: $\wedge$ (SEND, P, Q, τ_s, *msg*) $\in$ tr
21: $\wedge$ (FETCH, Q, τ, M) $\in$ tr $\wedge$ $\tau \geqslant \tau_s + \Delta$
22: $\wedge$ $\neg\exists\, \tau', M'$:
23: (FETCH, Q, τ', M′) $\in$ tr $\wedge$ (P, Q, τ_s, *msg*) $\in$ M′;
24: else $w \leftarrow 0$
25: **return** w

The FINALIZE of $\mathcal{G}\text{m}_{\text{AUTH}}$; the shell above is unchanged

id.FULLFINALIZE() from id′
1: **require** ¬DONE
2: **require** id.P $= Z$
3: DONE $\leftarrow$ TRUE
4: **C** $\leftarrow$ (C, Z).FULLSTATUS() as id
5: $w \leftarrow 1$ if $\exists$ Q, τ, M, P, τ_s, *msg* :
6: Q $\notin$ **C** $\wedge$ (FETCH, Q, τ, M) $\in$ tr $\wedge$ (P, Q, τ_s, *msg*) $\in$ M
7: $\wedge$ (SEND, P, Q, τ_s, *msg*) $\notin$ tr;
8: else $w \leftarrow 0$
9: **return** w

Three things about the code, two of them familiar. Both relays read the clock themselves for the same reason as Appendix K.2's: the two readings coincide because no core of γ runs between them, READ placing no call of its own. SEND's relay keeps $\mathcal{F}_{\text{AC}}$.SEND's own return, OK, having nothing else to report — unlike $\mathcal{F}_{\text{Net}}$'s, which had a message id to pass back and $\mathcal{F}_{\text{AC}}$ never hands out. And authentication's verdict needs no search over senders or times, unlike its namesake's: a fetched record already names P and τ_s, so the win condition is a direct check that the exact tuple was never sent, not an existential one over what might have produced it — the address $\mathcal{F}_{\text{AC}}$ carries is doing, in the verdict, exactly what it does at the interface.

Proposition L.1 ($\mathcal{F}_{\mathsf{AC}}$ has liveness). *For every $\mathcal{Z} \in \mathbb{Z}_{\gamma,\gamma}$ and every occupant $\mathcal{X}$ of the adversary slot,* $\Pr[\textsc{Win}] = 0$. *So $\{\mathcal{F}_{\mathsf{AC}}, \mathcal{G}_{\mathsf{Clock}}\}$ has liveness within 0 against $(\mathbb{Z}_{\gamma,\gamma}, \mathbb{X})$ in the sense of Definition 12.8, for $\mathbb{X}$ the class of all occupants.*

Proof. Suppose $(\textsc{send}, P, Q, \tau_s, msg) \in \mathtt{tr}$, written at some activation of the SEND relay (line 5). Its own clock read and $\mathcal{F}_{\mathsf{AC}}$.SEND's internal one (line 4) agree, by the argument of Appendix K.2: no core of γ runs between them, $\mathcal{F}_{\mathsf{AC}}$'s guard and corruption check finding $\mathtt{id}.P$ honest since the relay ran at all, and READ itself placing no call. So the core wrote some fresh m with $\mathtt{list}[m] = (P, Q, \tau_s, msg)$ at that moment.

Suppose too $(\textsc{fetch}, Q, \tau, M) \in \mathtt{tr}$ with $\tau \geqslant \tau_s + \Delta$. If $\mathtt{list}[m]$ still holds (P, Q, τ_s, msg) when that fetch's core runs, then $m \in R$ by line 12 — the entry names Q, and $\tau_s + \Delta \leqslant \tau$ by hypothesis, for τ its own reading of the clock and, by the same argument again, the relay's — so $(P, Q, \tau_s, msg) \in M$ whatever the slot answers at line 13 and whatever SAN does with it, neither touching R. If instead $\mathtt{list}[m]$ had already been cleared, that happened only at line 16 of some earlier fetch by Q — the one line that ever clears an entry — and only on indices already placed in that fetch's own $R \cup S$, hence already present in that earlier fetch's own M. Either way (P, Q, τ_s, msg) appears in some fetch of Q's, and line 18 gives 0. □

Proposition L.2 ($\mathcal{F}_{\mathsf{AC}}$ has authentication). *For every $\mathcal{Z} \in \mathbb{Z}_{\gamma,\gamma}$ and every occupant $\mathcal{X}$ of the adversary slot,* $\Pr[\textsc{Win}] = 0$. *So $\{\mathcal{F}_{\mathsf{AC}}, \mathcal{G}_{\mathsf{Clock}}\}$ has authentication within 0 against $(\mathbb{Z}_{\gamma,\gamma}, \mathbb{X})$ in the sense of Definition 12.8, for $\mathbb{X}$ the class of all occupants — the property Appendix K.2 proved $\mathcal{F}_{\mathsf{Net}}$ lacks, held here instead.*

Proof. Suppose $(\textsc{fetch}, Q, \tau, M) \in \mathtt{tr}$ with $Q \notin \mathbf{C}$ and $(P, Q, \tau_s, msg) \in M$. By $\mathcal{F}_{\mathsf{AC}}$.FETCH's code, $M = \{\mathtt{list}[m] : m \in R \cup S\}$, and both index sets draw only on $\mathtt{list}$ as it already stands: R by its own pattern match (line 12), and S after sanitizing, since $\textsc{Clean}_{\textsc{addr}}$ admits only indices i with $\mathtt{list}[i] = (\cdot, Q, \cdot, \cdot)$ already and line 3 (Appendix G) picks among values meeting that predicate — neither manufactures an index $\mathtt{list}$ does not hold. So $(P, Q, \tau_s, msg) \in M$ only if $\mathtt{list}[m] = (P, Q, \tau_s, msg)$ for some m at some point.

The only line that ever writes $\mathtt{list}[m]$ to a value other than $\perp$ is SEND's own core, and by tightness — $\mathcal{F}_{\mathsf{AC}}.\mathbf{N} = \{\mathcal{G}m.pid\}$ pinned — that core runs only on calls $\mathcal{G}m$'s relay places: not the environment, not the adversary, not a machine hosted at an invented instance of the game's name. So the write producing (P, Q, τ_s, msg) happened at some activation of the SEND relay, and by the coinciding-read argument of the previous proof it was logged there as exactly $(\textsc{send}, P, Q, \tau_s, msg)$. Hence $(\textsc{send}, P, Q, \tau_s, msg) \in \mathtt{tr}$, and line 5 gives 0. □

As with $\mathcal{G}_{\mathsf{Clock}}$ and $\mathcal{F}_{\mathsf{Net}}$, both properties transfer by Corollary 12.11 only if the game system's cores place no responsive call — and here, as there, neither does: Appendix L.1's "Both slot calls are responsive" covers SEND's notification and FETCH's query alike. A system π put in place of $\{\mathcal{F}_{\mathsf{AC}}, \mathcal{G}_{\mathsf{Clock}}\}$ must then have both transfers argued directly, for the reason Remark F.4 gives, exactly as $\mathcal{F}_{\mathsf{Net}}$'s own notification cost Appendix K.2 the same thing.

Unlike the network, nothing here is left standing only as a negative result. Proposition L.2 is the win condition Appendix K.2 transplanted from $\mathcal{F}_{\mathsf{Net}}$ and could not close, closed here instead — and the two proofs, laid side by side, locate the whole difference in one fact: whether tightness can trace a fetched record back to a SEND that produced it. For

$\mathcal{F}_{\mathsf{AC}}$ it always can, `list` having no other way to gain an entry; for $\mathcal{F}_{\mathsf{Net}}$ it cannot, line 16 being exactly the other way in. Two appendices, one game transplanted twice, and the address is the entire difference between them.